

MOBILE SECURITY AND PRIVACY:
ATTACKS AND DEFENSES

A DISSERTATION
SUBMITTED TO THE DEPARTMENT OF ELECTRICAL
ENGINEERING
AND THE COMMITTEE ON GRADUATE STUDIES
OF STANFORD UNIVERSITY
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE DEGREE OF
DOCTOR OF PHILOSOPHY

Yan Michalevsky
December 2017

Preface

Smartphones are ubiquitous and hold much of our private personal data. Many smartphones are equipped with a multitude of sensors: cameras, microphones, motion sensors, magnetometers, GPS receivers and more. This dissertation explores ways in which smartphone sensors can be used for unintended applications. We show how to use the accelerometer and the microphone to obtain a robust hardware fingerprint that can be used to track a user across web sites; we show how to use the gyroscope to eavesdrop on speech; and we show how to use the power meter to learn the location and movements of the owner. This work demonstrates that a combination of signal processing, machine learning and the physical properties of mobile hardware, can reveal unexpected information about the device and its owner. These results are intended to inform the community, and smartphone vendors, about the risks of giving sensor data to third party smartphone applications.

Acknowledgments

First, I would like to thank my advisor, Prof. Dan Boneh, for the guidance and mentoring through my PhD studies, as well as for co-authoring some of the publications that this thesis is based on. He was an inspiration and a role-model due to his combination of professional qualities and personality. In particular I admired his practical approach, clarity of expression, and commitment to helping improve security in practice. I feel grateful for having the chance to learn from him, and lucky to have had his guidance.

Thanks to Prof. Sachin Katti, I had a supportive environment during my first year at Stanford, that enabled me to explore my interests. Thanks to being part of his lab I was exposed to the work of Aaron Schulman that eventually inspired me to think about the PowerSpy attack presented here. Eventually, important parts of this thesis ended up revolving around different aspects of wireless communication and signal processing.

Thanks to Hristo Bojinov, who was finishing his PhD around the time I came to Stanford. Hristo was the first author and the main contributor to the paper on sensor fingerprinting that constitutes the first chapter of my dissertation. His work contributed to the basis on which my research kept on building.

I cannot express enough gratitude to my friend and co-author, Gabi Nakibly. I was lucky to run into him on campus during my first year at Stanford, while he was a visiting researcher there. He told me about the research he was doing on sensor fingerprinting and I started helping with it. It eventually turned out to be the seed to this dissertation, and evolved into a fruitful collaboration on several more works.

I would like to thank Suman Nath and Jie Liu, my mentors at Microsoft Research

where I spent the summer of 2015 and worked on exploring the application of secret handshakes for mobile devices.

Towards the end of my PhD program I collaborated with Marc Joye, and had him as a mentor during my internship at NXP Semiconductors. It was extremely fun working with him and learning cryptography from him, benefitting from his vast knowledge and experience in the field. Always helpful, attentive and kind, Marc is a great colleague who I hope to have a chance to work with again.

I'd like to thank Valeria Nikolaenko for being a great office-mate, with her cheerful personality, politeness and the willingness to explain cryptography whenever there was something I wanted to understand. I'm also grateful for our collaboration on the Secure-Reduce project.

I would like to thank my peers in the lab along the course of my PhD: Ananth Raghunathan, Hart Montgomery, Mark Zhandry, Joe Zimmerman, Kevin Lewi, Ben Fisch, Sam Kim, Benedikt Bünz, Dima Kogan, Florian Tramèr, Nathan Manohar and Saba Eskandarian. Special thanks to Henry Corrigan-Gibbs for reviewing and commenting on some of my drafts and for insightful discussions about security, and to David Wu with whom I had the pleasure to work on a proposal for the Qualcomm Innovation Fellowship, and who was always helpful in explaining concepts in cryptography that I was unfamiliar with.

Thanks to the former students in Sachin Katti's lab from whom I learned, and who had been good friends during my first year at Stanford: Aditya Gudipati, Dinesh Bharadia, Rakesh Mishra, Emily McMillin, Steven Hong and Kanthi Nagaraj. Special thanks to Manu Bansal whom I worked with closely, and to my friend Asaf Cidon who inspired me to take on this journey.

I would like to thank the members of my dissertation defense committee, whom I have not mentioned yet: Prof. Tsachy Weissman, Prof. Monica Lam and Prof. Mykel Kochenderfer. I am grateful for their willingness to help on a fairly short notice, as well as for the feedback provided following the oral examination.

Finally, thanks to my family and friends: my parents, Vladimir and Violetta, my sisters Beatrice and Veronica, my grandparents Sofia, Alexander and Claudia, my in-laws Gerlind, Joachim, Karen, Oma Gerda, Oma Ursel and Opa Johannes, my

pretty well-behaved and sweet dog Emma, and to my dear, beloved and wonderful wife, Katharina Rösler, who, on top of her own demanding PhD program, had the kindness and empathy to be happy for me on the good days, and support me on the tougher ones. My dear friends and extended family in Israel, it was sad to part from you when I headed to Stanford, and you were very often on my mind.

Contents

Preface	iv
Acknowledgments	v
1 Introduction	1
2 Sensor Fingerprinting	4
2.1 Threat Model	6
2.2 Background	6
2.2.1 Android	7
2.2.2 iOS	7
2.3 Use of Sensors for Identification	8
2.3.1 Common Bias Types	8
2.3.2 Common Sensor Types	9
2.4 Device Identification via the Microphone	10
2.4.1 Fingerprinting Scheme	10
2.4.2 Experiment: L2-Distance Classification	13
2.4.3 Experiment: Maximum-Likelihood Classification	14
2.4.4 Improving Measurement Stealth	16
2.5 Device ID Using the Accelerometer	17
2.5.1 Fingerprinting Scheme	17
2.5.2 Using a Web Page to Profile the Accelerometer	18
2.5.3 Experiment: Initial Evaluation	20
2.5.4 Experiment: Large-Scale Accelerometer Fingerprinting	21

2.6	Estimating All Six Bias Parameters of the Accelerometer	25
2.6.1	Experiment: Lab Droids in 3D	27
2.7	Threat Mitigation	27
2.8	Difficulties with using some sensors for identification	28
2.9	Related Work	30
2.10	Future work	32
3	Recognizing Speech from Gyroscope Signals	33
3.1	Gyroscope as a microphone	35
3.1.1	How does a MEMS gyroscope work?	35
3.1.2	Acoustic Effects	37
3.1.3	Characteristics of a gyro as a microphone	38
3.2	Speech analysis based on a single gyroscope	43
3.2.1	Speech processing: features and algorithms	44
3.2.2	Speaker identification algorithm	46
3.2.3	Speech recognition algorithm	47
3.2.4	Experiment setup	47
3.2.5	Sphinx	49
3.2.6	Custom recognition algorithms	50
3.2.7	Further improvement	53
3.3	Reconstruction using multiple devices	54
3.3.1	Reconstruction algorithm	55
3.4	Further Attacks	59
3.5	Defenses	60
3.6	Gyrophone Attack Conclusion	61
4	Location Tracking using Power Analysis	62
4.1	Threat Models	65
4.2	Background	66
4.2.1	Location affects signal strength and power consumption	67
4.2.2	Power usage can reveal location	67
4.2.3	Hysteresis	70

4.2.4	Background summary and challenges	70
4.3	Route distinguishability	71
4.4	Real-time mobile device tracking	72
4.4.1	Tracking via Dynamic Time Warping	73
4.4.2	Improved tracking via a motion model	73
4.4.3	Tracking using Optimal Subsequence Bijection	73
4.5	Inference of new routes	74
4.5.1	Particle Filter	75
4.6	Experiments	77
4.6.1	Data collection	77
4.6.2	Assumptions and limitations	77
4.6.3	Route distinguishability	78
4.6.4	Real-time mobile device tracking	80
4.6.5	Inference of new routes	82
4.7	Future directions	87
4.7.1	Power consumption inference	87
4.7.2	State of Discharge (SOD)	88
4.7.3	Choice of reference routes	88
4.7.4	Collecting a massive dataset	89
4.8	Defenses	89
4.8.1	Non-defenses	89
4.8.2	Risky combination of power data and network access	89
4.8.3	Secure hardware design	90
4.8.4	Requiring superuser privileges	90
4.8.5	Power consumption as a coarse location indicator	90
4.9	Related work	91
4.9.1	Many sensors can leak location	91
4.9.2	Other private information leaked from smartphone sensors	92
4.10	PowerSpy Attack Conclusion	92

5	Mobile Applications of Secret Handshakes	94
5.1	Background	98
5.1.1	Bluetooth Low Energy	98
5.1.2	Cryptographic secret handshakes	101
5.2	Secret handshakes over BLE	108
5.2.1	Challenges	108
5.2.2	Choice of the underlying SH scheme	109
5.2.3	A new pairing mode	110
5.2.4	Advertiser-scanner interaction	111
5.3	Implementation	113
5.3.1	Implementation Challenges	114
5.3.2	Windows Phone	114
5.3.3	Evaluation	121
5.4	Further improvements	124
5.4.1	Speeding up computation	124
5.4.2	Using BLE identifiers as pseudonyms	125
5.4.3	Organizational role authentication	125
5.4.4	Additional applications of PbcProxy and ported PBC library	125
5.4.5	Usability	126
5.5	Related work	126
5.6	MASHaBLE Conclusion	128
6	Conclusions	129
A	Gyrophone supplementary materials	131
A.1	Signal reconstruction from Recurrent Non-Uniform Samples	131
A.2	Code for sampling a gyroscope via a HTML web-page	132
A.3	Gyroscope rate limitation on Android	132
A.4	Code Release	133

B PowerSpy supplementary materials	134
B.1 Formal model of new route inference	134
B.2 Choosing the best inferred route	135
Bibliography	137

List of Tables

2.1	Common mobile device sensors along with a description of their im- perfection	9
2.2	Frequencies at which we measure the feedback ratio for each device .	12
2.3	Performance of simple L2 distance-based classification of data from locations 2 and 3.	14
2.4	Performance of MLE-based classification	16
3.1	Maximum sampling frequencies on different platforms	39
3.2	Sensed amplitude for every direction	42
3.3	Speaker's gender identification results	50
3.4	Speaker identification results	50
3.5	Speaker-independent case – isolated words recognition results	50
3.6	Speaker-dependent case – isolated words recognition for a single speaker	51
3.7	Evaluation of the method of reconstruction from multiple devices. . .	59
4.1	Route distinguishability evaluation results.	79
4.2	Test Routes	84
4.3	Destination localization	85
4.4	Levenshtein distance	86
4.5	Exact full route fit	87

List of Figures

2.1	A typical frequency response curve for a microphone	11
2.2	Diagram of sound feedback analysis	11
2.3	Comparison of the first harmonic feedback ratio curves obtained for three devices	12
2.4	Comparison of second-harmonic curves for all devices	13
2.5	Four overlaid frequency response curves at a different location.	15
2.6	Scatter plot of data obtained by accessing the accelerometer profiling web page from 17 iOS devices.	20
2.7	Percent correctly identified devices as a function of M_{S_z}	22
2.8	Different OS platforms as identified from the User-Agent string	23
2.9	Devices grouped by the number of their submissions to our server.	24
2.10	Compass readings taken by placing two different devices at the same location on a flat surface	29
3.1	STMicroelectronics 3-axis gyro design	37
3.2	InvenSense 3-axis gyro design	37
3.3	Examples of aliasing on a mobile device	41
3.4	Coordinate system of Android and iOS.	42
3.5	Experimental setup	48
3.6	Speaker independent word recognition using DTW: confusion matrix	51
3.7	Speaker dependent word recognition using DTW: confusion matrix	51
4.1	Signal strength profiles measured on two different days are stable.	68

4.2	For two phones of the same model, power variations on the same drive are similar.	69
4.3	For two different phone models, power variations on the same drive are similar.	69
4.4	Power profile with a phone call	78
4.5	Location estimation error for online tracking.	81
4.6	Estimation errors distribution for real-time tracking.	81
4.7	Comparison of DTW and OSB for real-time tracking.	82
4.8	Map of area and intersections for route inference.	83
4.9	Examples of estimation errors and their corresponding distances . . .	87
5.1	Structure of a BLE advertisement packet	100
5.2	Secret handshake protocol.	103
5.3	Secret handshake as a new pairing mode	112
5.4	Component dependencies	117
5.5	Class diagram of the handshake prototype implementation	118
5.6	Secret handshake on top of Advertiser-Scanner interaction	119
5.7	State machines for Advertiser and Scanner.	120
5.8	Battery drain in different modes after 3 hours	123
A.1	Filterbank reconstruction scheme	132
A.2	Recording audio at 200 Hz using JavaScript	133

Chapter 1

Introduction

Smartphones have become an indispensable tool, and are a hub for personal and often sensitive data. As such, there is a pressing need to better understand their privacy and security characteristics. This dissertation studies side-channel attacks on smartphones: attacks that expose information via unanticipated channels that were overlooked at design time.

Let us begin with an analogy that symbolizes where side-channel research lies within the broader field of security and cryptography. Imagine a classified facility where the entrance is reasonably secured, has RFID based access control, automatic gates that would prevent an unauthorized vehicle from breaking in, and so on. However, at the same time, there is ongoing construction in the back. Since construction workers have to come and go, there is literally a hole in the fence, with a little gate. We can think of it as a metaphor for secure systems design vs. side-channels research and hacking. While the former aims to provide solid security guarantees, rooted in formal reasoning, the latter tries to find those holes in the fence.

Smartphone devices have a multitude of sensors on them: cameras, microphones, a magnetometer, an ambient light sensor and motion sensors such as accelerometers and gyroscopes. They enable rich functionality and connect the physical world to the digital. At the same time, however they expose us to new threats. Taking control of a camera or a microphone of on a smartphone can obviously enable eavesdropping or spying on the owner. Access to the GPS sensor can clearly enable tracking her

location. These are obvious threats that were taken into account early on, and are addressed by an access control system that requires the user’s approval when giving applications access to sensitive data.

In a series of works [120, 122, 47], we demonstrated that seemingly benign sensors, that are freely accessible to applications and even web-browsers, can expose users to unexpected threats. In particular, access to the motion sensors, accelerometers and gyroscopes, is not considered sensitive, however, it enabled two of the side-channels presented here. This dissertation aims to convince that security is not modular, in the sense that it is hard to build a secure system by composing different, seemingly secure blocks. Eventually, we need to take a look at the system as a whole and examine it for flaws and security breaches.

There is an interesting duality that often applies to side-channels. When we find new ways to use some information, it often enables additional constructive applications. Each one of the techniques this thesis presents can be seen both as threat, and as a feature. For instance, the fingerprinting techniques presented in chapter 2 can serve for web-tracking and de-anonymization of users on one hand, but also for detecting fraudulent login attempts from unauthorized devices on the other. Gyrophone-like techniques described in chapter 3 can be used for malicious eavesdropping on a user. However, as shown in [161], they can also be used for energy-efficient recognition of hotwords. Similarly, the PowerSpy technique in chapter 4 that enables adversarial location tracking, can be harnessed to provide an energy-efficient location discovery service, without the need to use the highly power-consuming GPS sensor. Here, we mainly focus on the offensive potential of the techniques.

Finally, this thesis addresses the problem of private interaction and communication between smartphones, by exploring a protocol for secret handshakes implemented on top of Bluetooth LE [121]. In a way, designing a privacy-preserving protocol was a redemption attempt after spending considerable time showing how broken things are.

The results in this thesis are based on the following publications:

- Mobile Device Identification via Sensor Fingerprinting [47]
- Gyrophone: Recognizing Speech from Gyroscope Signals [120]
- PowerSpy: Location Tracking using Mobile Device Power Analysis [122]
- MASHaBLE: Mobile Applications of Secret Handshakes over Bluetooth LE [121]

Chapter 2

Mobile Device Identification via Sensor Fingerprinting

Many Internet services need reliable identifiers to identify repeat visitors. The simplest identifier, a Web cookie, works well, but is unreliable in case users clear cookies, block 3rd party cookies, or use private browsing mode. This lead to the development of stronger identifiers such as supercookies. The Panopticlick project [70] showed that desktop browsers are sufficiently different to be identified. However, the project noted that mobile browsers, especially on iOS, are too similar for this approach to work.

The need for robust identifiers is even stronger on mobile devices. First, as above, Web sites wish to identify repeat visitors. Second, cloud-based services who develop mobile applications often need a robust phone identifier. Consider the following scenario: a user installs a cloud-based app and the app installs an identifier on the device. Later the user resets the device to its factory settings thereby deleting the app and its stored identifier. The user then re-installs the app and connects to the cloud service. At this point the service cannot tell whether it has already seen the device before. This simple trick may allow a misbehaving user whose account was blocked to reconnect to the service using a different identity.

More generally, online device identification is a topic of much interest to advertising networks and organizations providing security services. The need to identify

remote peers is always pitted against concerns that identifying information may be misused. Rapidly evolving mobile technologies pose new challenges to preserving user privacy and one of our goals in this chapter is to explore these challenges, and inform the design of future mobile device platforms.

To obtain a robust identifier for mobile devices many app developers have turned to a hardware ID that survives a device reset to factory settings. A recent study shows that 8% of Android apps use the International Mobile Equipment Identity (IMEI) as a hardware device ID [71]. This type of practice is frowned upon to the point that Apple disallows apps who read the iOS Universal Device ID (UDID) on their app store [99].

In this chapter we show that the multitude of sensors on a modern smartphone can be used to build a robust device ID, or fingerprint, that is independent of the software state and survives a hard reset. Our results show a unique device fingerprint can be computed without accessing traditional hardware identifiers such as the IMEI or UDID. Consequently, simply disallowing app access to the device UDID is an ineffective privacy policy.

We experiment with fingerprinting using two sensors:

- **The speakerphone-microphone system:** the fingerprinting system uses the speakers to emit a sequence of sounds at different frequencies and records the resulting signals using the microphone. The fingerprint is computed by looking at amplitude and frequency distortions in the recorded signals.
- **The accelerometer:** the accelerometer measures forces in each of the three dimensions. Imprecisions in accelerometer calibration result in a device-specific scaling and translation of the measured values. By repeatedly querying the accelerometer we estimate these calibration errors by solving an optimization problem and using the resulting six values (two for each dimension) as a fingerprint.

By collecting sensor measurements from over 10,000 mobile devices we show that the resulting fingerprints are robust and survive a hard device reset. Moreover there is

sufficient entropy in the fingerprint to reliably identify the device among thousands of devices.

Recently, [67] proposed a method to fingerprint an accelerometer while it is vibrating (e.g. during an incoming call or message). This method depends on the surface on which the phone lays and the case in which it is enclosed. In contrast, our method is oblivious to these factors. Ref. [58] and [61] proposed methods to fingerprint loudspeakers. Our method differs by allowing to fingerprint the combination of loudspeaker and microphone thus yielding more fingerprint entropy.

2.1 Threat Model

Device identification may be used for both malicious and benevolent purposes. Here we focus on the offensive potential of sensor fingerprinting.

A *malicious website* may wish to track its users without resorting to any browser storage such as cookies, and without triggering any permissions warnings. Each device that comes into contact with the website is fingerprinted and classified as one of the already known devices or as a new one—if its fingerprint differs sufficiently from those in the database. The malicious website is assumed to be accessed by the user for a long enough time interval, during which the user may possibly leave the device unattended or unused. The user does not suspect that fingerprinting is taking place and does nothing aimed specifically at disrupting the process.

2.2 Background

There are a few standard methods for mobile device identification that can be used by a native (but not browser-based) application. These methods use information exposed by the operating system, however all of them are either applicable to only some versions of an OS or the identifying information reported by the OS can be easily changed by the owner of the device thereby evading identification. We now list some of these standard identification methods in the context of the two most popular mobile operating systems: Android and iOS. For each method we detail its

most important features and restrictions. The following is primarily based on [37].

2.2.1 Android

Device ID: The method `getDeviceId` of the `TelephonyManager` class returns the unique ID for a phone, for example, the IMEI for GSM phones and the MEID or ESN for CDMA phones [36]. However, no such ID exists for mobile device which do not have telephony capabilities.

MAC address: One can retrieve the MAC address of one of the device's network interfaces (e.g. WiFi and Bluetooth). However, the owner of a device is able to change the MAC address of the device. For example, see the "MAC Address Ghost" application [81].

Serial number: The field `SERIAL` of the `Build` class contains a hardware serial number, if one is available on the device [33]. This field is only available on version 2.3 and later.

ANDROID_ID: The constant `ANDROID_ID` of the `Settings.Secure` class is a 64-bit number that is randomly generated on the device's first boot and remains constant for the lifetime of the device [35]. However, the value may change if a factory reset is performed.

2.2.2 iOS

UDID: The Unique Device Identifier (UDID) is the primary method for the identification of an iOS device. It is retrieved using the `uniqueIdentifier` property of the `UIDevice` class. However, since iOS 5 it has been deprecated [94].

identifierForVendor: This is a property of the `UIDevice` which is only available in iOS 6 and later. It is an alphanumeric string that uniquely identifies a device to the applications vendor, i.e., different application vendors will retrieve different identifiers [94]. However, the value of this identifier is deleted once the user uninstalls the last application from a particular vendor. If he later reinstalls an application from that vendor, the a new identifier value will be generated.

advertisingIdentifier: This is a property of the `ASIdentifierManager` class which is only available in iOS 6 and later. It is an alphanumeric string unique to each device and is intended to be used by advertisers [93]. However, the value of this identifier is reset once the device is erased by the user.

MAC address: As in Android, a MAC address may be forged by the owner of the device.

2.3 Use of Sensors for Identification

The key observation behind this work is that different hardware instances of a particular sensor are quite different, mainly due to imperfections in the manufacturing and assembly process. These variations introduce biases into the sampled data read from the sensor that are unique to the specific sensor. Moreover, these variations are persistent throughout the life of the sensor. By measuring the imperfections we can consistently identify devices carrying these sensors.

We start by defining different types of sensor biases, and then follow with an overview of commonly encountered sensor types.

2.3.1 Common Bias Types

Linear bias: For many sensor types the value of their measurements can be approximated as a linear function of the true value they measure, i.e. $v_m = v_t S + O$. Here v_m and v_t are the measured and true values, while S and O are the sensitivity and offset of the sensor (in other words S and O are the calibration parameters specific to the sensor). Ideally, the parameter values should be $S = 1$ and $O = 0$. Approximate linear bias can be found in accelerometers [69], gyroscopes [44], magnetometers [136], and camera pixels [88]. We note that in most sensors a linear bias is only an approximation of the actual bias. Some sensors also manifest random and quantization noises, while other sensors exhibit cross-dimensional effects, where the measured value in one dimension affects the measurements in the other dimensions. Another example of a linear bias is clock drift. The absolute difference between a

clock's time and the true time increases linearly as time goes by. Here the clock's sensitivity is also referred to as the clock skew.

Tolerance: For some sensors their measurements can not be modeled as a simple linear function of the true value. In such cases the measurements will be within a predetermined range relative to the true value. For example, the output gain of a microphone for a specific frequency may vary with $\pm 2\text{db}$ within the actual input power.

Timing: In addition to the bias of the actual measurements a sensor may exhibit variance in the time it takes to produce measurements. Sensor data is often gathered when the hardware triggers an interrupt, signaling that there are new readings available. The timing of this interrupt may vary across devices, and then it can be used as part of an identification scheme. Interrupt timing is relatively difficult to access from application code, but still it may be a viable component in a larger fingerprinting scheme.

2.3.2 Common Sensor Types

Our goal is carrying out a comprehensive survey of sensors that are commonly available on mobile devices, designing specific identification techniques wherever possible.

Sensor	Imperfection	Comment
Audio	tolerance gain	Section 2.4
Accelerometer	linear bias	Section 2.5
Gyroscope	linear bias	no baseline
Magnetometer	linear bias	variability, hysteresis
Ambient light	linear bias	no baseline, sporadic data
GPS	clock skew	not observable
Touch screen	misalignment	no baseline
Camera	pattern noise	see [113]

Table 2.1: Common mobile device sensors along with a description of their imperfection. Non-covered sensors show the reason why use for identification is difficult.

Table 2.1 lists the different sensors we have looked at. While in theory most sensors have some sort of measurable bias, in practice the defect may not be readily exposed under “normal” conditions, consistent with the threat model that we have outlined, or no baseline measurement is available to calculate the bias.

This chapter describes two successful attempts for sensor fingerprinting: audio (microphone/speaker) in section 2.4 and accelerometer in section 2.5. In appendix 2.8 we sketch some of the difficulties that hindered identification using other sensors listed in Table 2.1.

2.4 Device Identification via the Microphone

The main specification of a microphone and a loudspeaker is the frequency response graph. A microphone’s frequency response is its normalized output gain over a given frequency range. Conversely, a loudspeaker’s frequency response is its normalized output audio intensity over a given frequency range. Ideally for both devices, the frequency response should be the same for all frequencies in the range. However, a typical microphone or loudspeaker has a response curve that varies across different frequencies. These variations are dependent on the design of the audio device. Figure 2.1 depicts a typical frequency response curve for a microphone.

Due to manufacturing inconsistencies the frequency responses of each instance of a microphone or a loudspeaker are not identical even if they are of the same model. A device’s response for each frequency has a tolerance relative to its response specified by the manufacturer. A typical tolerance for commodity microphone and loudspeakers is $\pm 2\text{db}$. This frequency response variance gives rise to our first fingerprinting scheme.

2.4.1 Fingerprinting Scheme

When it comes to fingerprinting, the sound domain is unique because mobile devices have the ability to both transmit (via the speaker) and receive (via the microphone). This in turn allows us to build a completely self-contained fingerprinting scheme, dependent only on the device being in a relatively quiet environment (to minimize

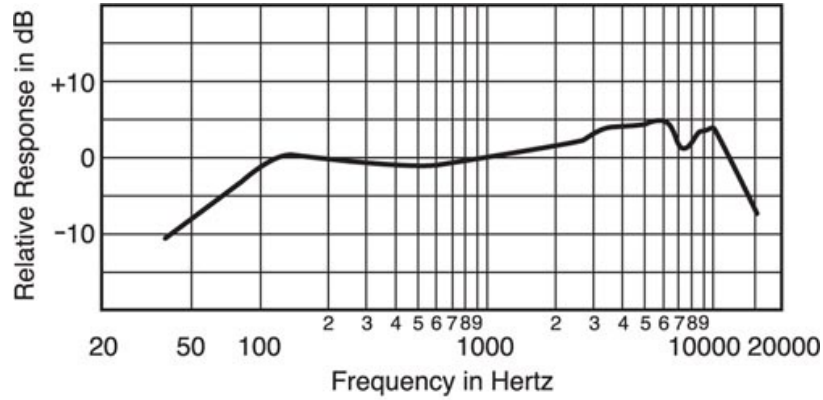


Figure 2.1: A typical frequency response curve for a microphone. Note that for some frequencies the audio is exaggerated (larger than 0 dB) while for others it is attenuated (smaller than 0 dB).

signal noise). In our scheme a device's audio fingerprint is the composed frequency response of the device's speaker and microphone. In a nutshell, we play using the speaker an audio signal at a given intensity and we record it using the microphone. We divide the recorded intensity by the original intensity. We refer to this as the *feedback ratio*. We measure the feedback ratio for several different frequencies. Figure 2.2 illustrates this process.

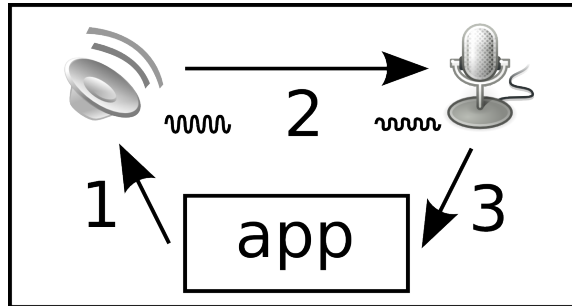


Figure 2.2: Diagram of sound feedback analysis. Within the confines of the device, the application emits sound via the built-in speaker (1), the sound reaches the microphone in a distorted and attenuated form (2), and the application records the microphone signal and analyzes it (3).

The signal recorded by the microphone is processed in the following way. We

isolate the main frequency, as well as its harmonics, by computing Fourier coefficients. Specifically, for each played frequency f_i , we record 1 second of samples (8000 integers at the typically supported sampling frequency) in the vector R_i , and calculate the j -th harmonic system response (for $j = 1, 2, \dots$) as follows:

$$r_{ij} = \sqrt{C(i, j) \cdot R_i + S(i, j) \cdot R_i}$$

Here $C(i, j)$ and $S(i, j)$ are vectors of 8000 samples of the reference signal as a cosine and sine function at frequency jf_i .

Hz	220	330	440	550	660	880	1320
----	-----	-----	-----	-----	-----	-----	------

Table 2.2: Frequencies at which we measure the feedback ratio for each device. The range and granularity of measurements can be extended, resulting in more data about each device. We have tried to stay below 2000Hz in order to be able to measure at least the second harmonic response at each frequency.

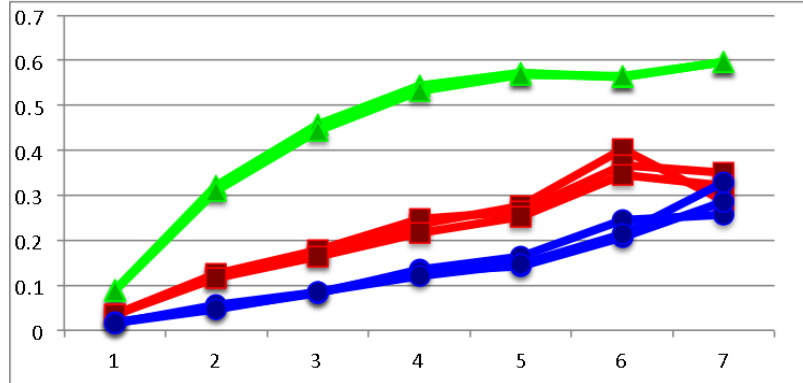


Figure 2.3: Comparison of the first harmonic feedback ratio curves obtained for three devices, each placed in three different locations (the same three locations used by all devices). Each device's curve is labeled with a distinct marker type and color. Feedback ratios (y-axis) are calculated at seven different frequencies (x-axis).

We repeat the playback, recording, and analysis at each frequency (Table 2.2), obtaining 7 floating-point numbers (r_{ij} 's) for each harmonic. Our analysis focused on the first two harmonics ($j = 1, 2$). Figure 2.3 shows a comparison between the

first harmonic measurements obtained from three different devices when measured at three different, fixed physical locations each. We note that the feedback ratio for a device is similar but not identical across locations. For example, a device’s feedback ratio is dependent on the acoustic properties of surface on which the device lies and the acoustic properties of the device’s surroundings. This poses a potential difficulty for this fingerprinting scheme.

2.4.2 Experiment: L2-Distance Classification

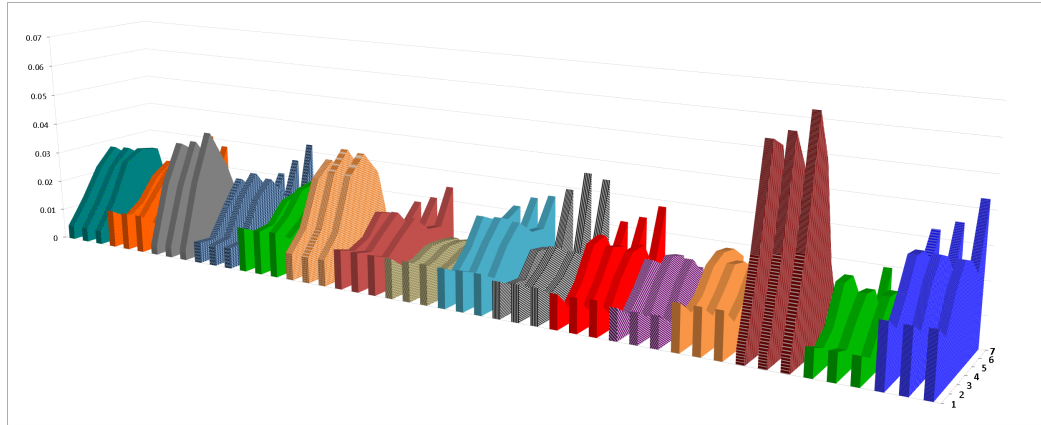


Figure 2.4: Comparison of second-harmonic curves for all devices. Each device is represented by three adjacent curves that have the same color and fill pattern.

We used 16 identical Motorola Droid devices to assess the feasibility of a fingerprinting scheme based on sound feedback analysis. We selected three locations (one on a wooden desk, one on a metal filing cabinet, and one on a composite wood windowsill). Each smartphone was placed at each location and measurements were taken. The measurements were performed by an Android application that we side-loaded onto each phone. The application requires only the `RECORD_AUDIO` and `MODIFY_AUDIO_SETTINGS` permissions, and once launched it adjusts the sound volume to a fixed, medium level and then plays a three-second sound at each frequency, recording the middle one second back on a separate Java thread. That one-second recording is used as the vector R in subsequent analysis.

Figure 2.4 juxtaposes the processed data obtained from all 16 devices. The similarity of the data from the same device across locations is evident, as are the differences between different devices.

Test Location	Distance Metric			
	A	B	B'	B''
2	68.8%	100%	62.5%	56.3%
3	43.8%	75%	50%	37.5%

Table 2.3: Performance of simple L2 distance-based classification of data from locations 2 and 3 (location 1 data were used as the training set). When first harmonic (Distance A), and second harmonic (Distance B) feedback are compared, second harmonic is almost always more reliable and performs better than first harmonic. Using the second harmonic’s first and second derivatives only appears to have a negative overall impact.

In order to estimate the amount of information that can be derived from this type of signal metric we designed a simple algorithm which uses one location’s measurement from each device to “learn” its fingerprint, and then tries to match the data from the remaining two locations to the device for which the learned fingerprint is the closest. If a data set is matched to the device it originated from, we count it as a correct detection (Table 2.3).

2.4.3 Experiment: Maximum-Likelihood Classification

From the initial set of results it was clear that using the response amplitude at the second harmonic frequency gives the best results, however the classification accuracy clearly left something to be desired. We performed a second experiment with the same batch of Droid devices (excluding one which was faulty¹).

In the second experiment, we measured response amplitudes at 13 different frequencies, from 100Hz to 1300Hz at 100Hz increments. We measured the response of each device at the same three locations we used in the previous experiment, however we ran the measurement four times at each location. Figure 2.5 illustrates why

¹The device we excluded was not misclassified in the first experiment, so its exclusion did not contribute to the improved results.

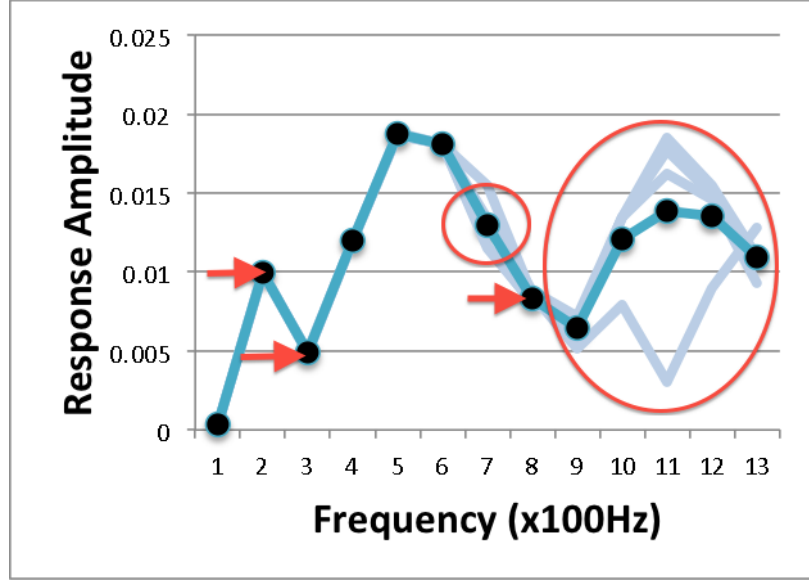


Figure 2.5: Four overlaid frequency response curves at a different location.

the simple Euclidean distance-based classification approach was not as precise as we wanted: at some frequencies, on some surfaces, there is simply too much variation—which results in a large penalty when calculating distances. Other frequencies offer much smaller tolerances and can be used to identify the device more accurately. Fortunately, the *maximum-likelihood estimation* method is designed to deal with just this type of situation.

Our new scheme is based on the following simple setup (all frequency response measurements are at the second harmonic frequency—this will be implied for the rest of this section): for each device we assume that its response at a certain frequency is a normally distributed random variable. Given enough samples, we can estimate the mean and variance of the distribution. Later, when we encounter a device, we can calculate the likelihood that it matches any of the known devices in our database—and select the one that maximizes the likelihood. Formally, based on the training data from device D_k , at each frequency f_i we estimate the mean of the response $\mu(k, i)$ as well as the variance $\sigma^2(k, i)$. Then we select the device $D_{k_{max}}$ that was most likely to produce this measurement:

$$k_{max} = \arg \max_k \sum_{i=1}^{13} -\frac{(v_i - \mu(k, i))^2}{\sigma^2(k, i)} \quad (2.1)$$

Equation 2.1 follows from the formula for the joint probability of normally distributed, independent variables after taking a logarithm (to turn the product into a sum) and maximizing the resulting sum.

Test Location	Frequency	
	Main	2 nd Harmonic
1	100%	100%
2	100%	100%
3	100%	100%
1	93.3%	100%
2	100%	100%
3 (untrained)	73.3%	80%

Table 2.4: Performance of MLE-based classification. The first run provides training data from all three locations, then tests at them. The second run omits Location 3 from the training data, yielding worse device recognition rates at the unknown location. The experiment involves 15 devices. For test location 3, the results are similar (symmetric) if another location is omitted from the training data instead.

The results from using maximum-likelihood estimation are shown in Table 2.4.

2.4.4 Improving Measurement Stealth

According to the method for microphone and loudspeaker characterization described so far we played each frequency separately. In order to achieve more stealth and shorter sound playback times we examined another method. We choose several frequencies that are not harmonics one of the other, and play them simultaneously. We compute the FFT of the recording and extract the second and third harmonics of the chosen base frequencies. We use the energies corresponding to those frequencies to construct the feature vector corresponding to the device. We obtained feature vectors from 17 devices using this method and tested K-NN classification performance

on this data using 10-fold cross validation, achieving correct identification percentage of approximately 95%.

2.5 Device ID Using the Accelerometer

An accelerometer measures the acceleration force that is applied to a device along all three physical axes. Recall that the sensor’s reading, v_m , along the v axis is related to the actual device acceleration, v_t , at that axis as follows: $v_m = v_t S_v + O_v$ [69]. Here S_v and O_v are the sensitivity and offset parameters of the accelerometer—note that for a three dimensional sensor there are 6 such parameters. We can use a well known acceleration baseline to measure the accelerometer’s offset—Earth’s gravity (denoted by g)². At rest the phone experiences an acceleration with a true magnitude of exactly g . The orientation of that acceleration depends on the relative orientation of the phone to the Earth’s surface.

2.5.1 Fingerprinting Scheme

The accelerometer is convenient to fingerprint for several fundamental reasons: the user often leaves the device still—for instance on a desk, or in a purse; as noted above, when the device is not moving the magnitude of the acceleration vector on the device equals g ; finally, acceleration can be measured by an Android application that does not require any permissions [34], and what is more, iOS as well as Android browsers expose this functionality to websites without notifying the user.

In contrast to audio-based fingerprinting, there is no good way to feed a signal into the accelerometer, namely exert a known acceleration force; instead we take an approach of performing background measurements and waiting until there is enough data to estimate the accelerometer calibration parameters. We perform a measurement every time the phone is at a resting position, or more precisely, the phone is at a constant velocity (no acceleration). Note that in most reasonable cases it is very unlikely that a phone will not be at rest for an extended period of time. Detecting

²Earth’s gravity indeed varies a little depending on location, however even these small variations can be predicted.

the phone is at rest is relatively straightforward: the measured acceleration vector should be static and its magnitude should roughly³ be equal to g .

Estimating O_z and S_z

Let's assume that throughout our measurements the device will be lying flat and still on a table and thus the Z axis will register practically all the acceleration due to Earth's gravity. If the sensitivity parameter S_z of the sensor is known, then it is easy to estimate the offset from a single measurement z_m : $O_z = z_m - gS_z$. Unfortunately, given an arbitrary device S_z is unknown and thus we need two measurements—one with the device facing up (z_{m+}) and one facing down (z_{m-}). Using these two numbers one can calculate the two bias parameters for the Z axis⁴.

$$\begin{aligned} S_z &= (z_{m+} - z_{m-})/2g \\ O_z &= (z_{m+} + z_{m-})/2 \end{aligned}$$

We will see that this method yields very satisfactory results in a variety of experimental settings even if the surface on which the phone lies is not perfectly level. This method may be exercised without user cooperation since it is quite plausible that the user will sometimes leave their device facing up, and sometimes facing down on a table or a desk. We note that by gathering more data and applying more sophisticated processing, all six accelerometer parameters can be estimated at the same time, with no restrictions on device orientation during measurement (Appendix 2.6).

2.5.2 Using a Web Page to Profile the Accelerometer

We started out by building an Android application to track accelerometer readings, filter out any instability, and carry out the simple calculations required to establish S_z and O_z for a given smartphone.

³Note that due to the accelerometer defects we are measuring, it will most likely not be exactly equal to g .

⁴For further details see [2].

While the application did not require any permissions to access the accelerometer (it is not considered to be a sensitive module by the Android framework), it would need the INTERNET permission to report its findings. We therefore instead built a light-weight JavaScript implementation that runs entirely in the mobile browser.

An implementation contained within a web page has the advantage that the user doesn't need to install anything—assuming the user can be convinced to leave the device facing up and then down for some time, a single visit to a website can result in a fingerprint being calculated.

To collect accelerometer data in JavaScript we implement a function to handle `window.ondevicemotion` events:

```
window.ondevicemotion = function(event) {  
    var x = event.accelerationIncludingGravity.x;  
    var y = event.accelerationIncludingGravity.y;  
    var z = event.accelerationIncludingGravity.z;  
    ...  
}
```

The first time when our function is called we clear all state and set a text message that instructs the user on how to proceed. Every accelerometer reading results in a new call, where we gather data in batches that are relatively uniform (to avoid noise) and point in a direction that we need (either the positive or negative Z axis, that is, orthogonal to the device screen). When we gather enough positive or negative Z data we instruct the user to flip the device so that we can complete the gathering process.

Finally, when enough data is collected, we estimate S_z and O_z for the device and post the result to our web site for future analysis. We also generate a random number and set a cookie with it in the user's browser. The cookie is also posted to our server to help us correlate different submissions while assessing the error rate for our fingerprinting algorithm.

2.5.3 Experiment: Initial Evaluation

We carried out our first accelerometer profiling experiment on a group of 17 iPhone and iPod Touch devices, obtaining multiple measurements from each.

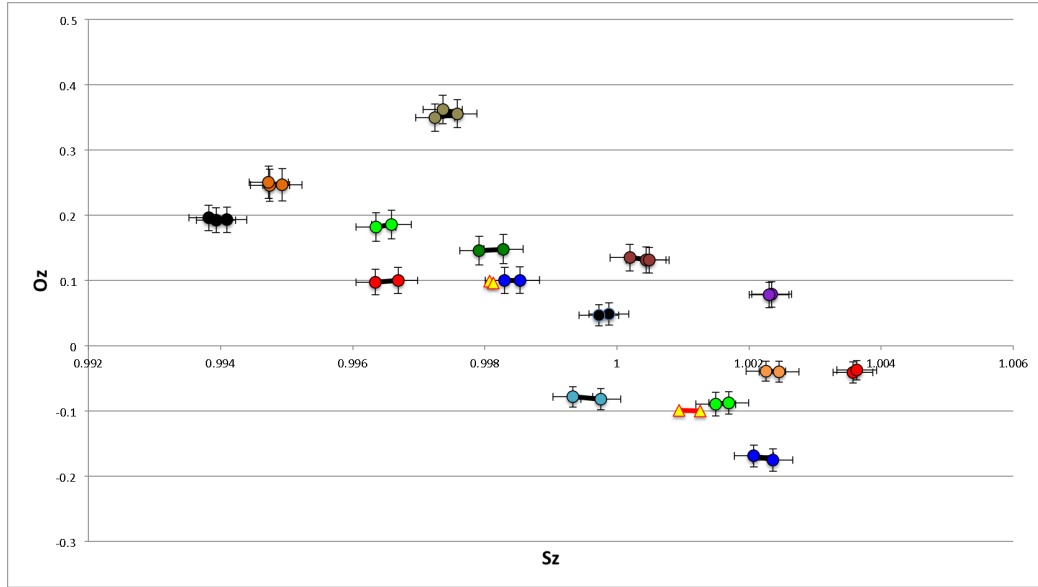


Figure 2.6: Scatter plot of data obtained by accessing the accelerometer profiling web page from 17 iOS devices.

Figure 2.6 presents a scatter plot of the estimated accelerometer parameters for all devices in the experiment. There were only two pairs of devices whose measurements were “too close”—one of the devices in each pair uses triangular markers instead of the usual round ones.

In our lab, using the batch of identical Motorola Droid devices from the audio experiments in Section 2.4 we were able to double the amount of data available. Because the Droid devices were too old and did not support JavaScript access to the accelerometer, we implemented an Android application which gathers the necessary data and processes it according to the same algorithm used by the web-based implementation described earlier. The application does not require any special permissions to install, as accelerometer readings are not considered to be significant for preserving user privacy.

The Droid data was also promising: from the 16 devices under study, there was only one near-collision. We combined the data obtained from the two initial accelerometer experiments in order to have the largest possible set of devices and thus assess realistically how feasible it is to identify handsets.

Our identification algorithm took two data samples from every device (most iOS devices only had two data points each anyway, due to constraints at the time of gathering; Droids had four samples each and we experimented with picking the first two, last two, and the first and last—Figure 2.7). We used the first of these samples as the device fingerprint, and the second one as a datapoint to be mapped to the closest of the known fingerprints. We used the square of the Euclidean distance between these points in the plane: $(O_{z_2} - O_{z_1})^2 + M_{S_z}(S_{z_2} - S_{z_1})^2$. Here M_{S_z} is a scaling factor used to reflect the different scale of O_z vs. S_z distances: while O_z typically ranges between -0.5 and 0.5 (Figure 2.6), S_z (being a multiplicative parameter in our model) is much more narrowly spread between 0.99 and 1.04 . This suggests a scale of about 10 for the values and about 100 for the squares. Indeed, the plot for the rate of correct identification as a function of this scaling factor is shown in Figure 2.7 and confirms this estimate, showing 100% recognition for M_{S_z} values between about 200 and 1000.

2.5.4 Experiment: Large-Scale Accelerometer Fingerprinting

Armed with insight from smaller-scale experiments in accelerometer fingerprinting we set out to expand the scope of the inquiry. Our goal was to gather data from a large number of devices in order to prove that accelerometers can be fingerprinted robustly in the context of a web application

We built an experimental public website [23] and publicized it in online as well as in printed media. Over the course of two weeks we gathered more than 16,000 submissions.

Website design. Our experiment website consists of a data-gathering page, and a dynamically generated chart page that the user can navigate to after submitting his

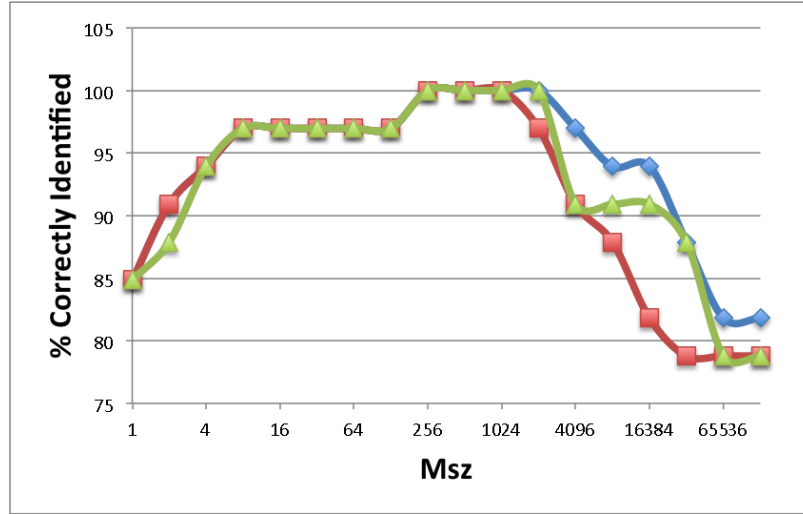


Figure 2.7: Percent correctly identified devices as a function of M_{S_z} in the distance formula. The red, green, and blue lines correspond to different ways of picking the training and test samples for Droid devices—the optimal values for M_{S_z} fall in the same range in all three cases.

or her device’s readings. The chart page displays a scatter plot of all data points that have been recorded so far, and shows the current user’s submission in a distinct color (Appendix ??). Effectively, the coordinates of the user’s device on the grid comprise the accelerometer’s Z-axis fingerprint.

The user is encouraged to go back to the data-gathering page, repeat the process, and see if the second identifier displayed matches the first one. Our data-gathering web page plants a cookie⁵ in the user’s browser, which makes it possible to correlate data points coming from the same device⁶.

The web site is implemented completely in JavaScript: no native code or Flash. We found that this method works reliably on a broad range of Android and iOS devices.

Data breakdown. We looked at several general properties of the data collected in order to ensure it made sense and matches our expectations in terms of quality. In

⁵Containing a large random number—a unique ID.

⁶Unless the browser application exited and deleted the cookie; we will ignore this type of scenario here: its presence will strictly degrade our results, so the analysis we report here is conservative.

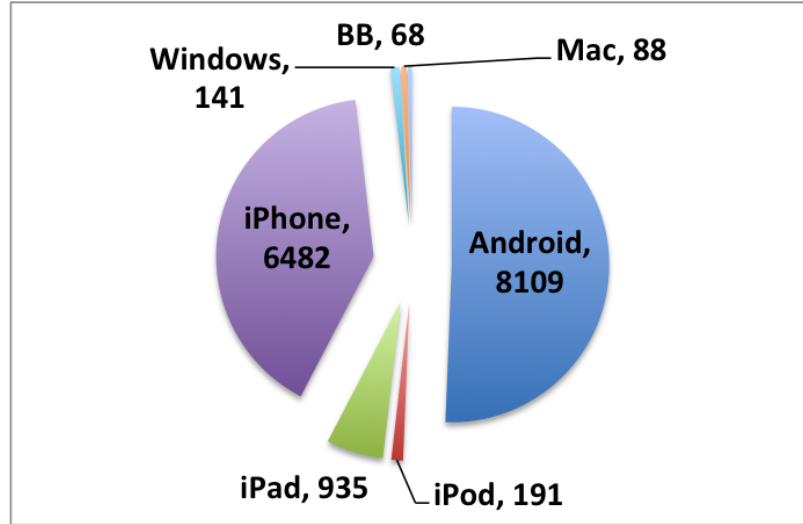


Figure 2.8: Different OS platforms as identified from the User-Agent string in device data submissions. In this chart multiple submissions from the same device are counted as separate entries.

Figure 2.8, we show the relative presence of different platforms in our data set. As expected, Android and iOS are the two dominant platforms, with a variety of others present in negligible numbers.

Figure 2.9 slices the data by the number of submissions we received from a devices. A large number of devices only ran the experiment once, another large group ran it twice (as our web page suggests), with diminishing numbers submitting three, four, and more times.

Device fingerprinting results. Our large-scale accelerometer fingerprinting experiment yielded enough data to make a direct calculation feasible. In a pre-processing scan, we create a list of all O_z and S_z distances between the two submissions of two-submission devices. We find that the 95th percentile for O_z distance is 0.045 (in other words, 95% of two-submission devices had a smaller distance than that). For S_z this distance is 0.0037.

When we used these 95th percentile distances to divide the O_z - S_z scatter plot into blocks of equal size—and counted the data points in each block (C_{xy})—we were able to calculate overall entropy of the distribution as follows:

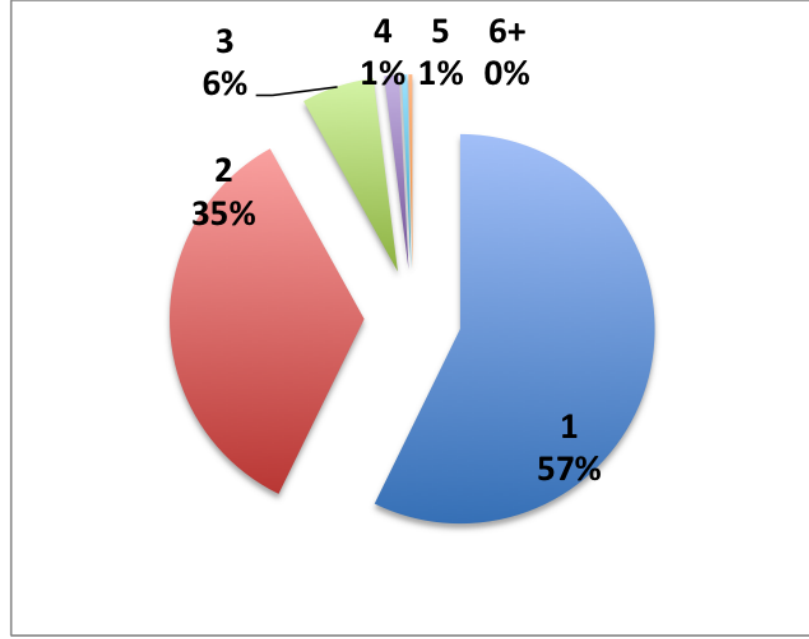


Figure 2.9: Devices grouped by the number of their submissions to our server. This chart reflects actual devices (we have collapsed multiple submissions from the same device into one entry for counting purposes).

$$P_{xy} = \frac{C_{xy}}{\sum_{x,y} C_{xy}} \quad (2.2)$$

$$H_{direct} = - \sum_{x,y} P_{xy} \log_2 P_{xy} = 7.498 \quad (2.3)$$

We verified that small variations of the grid origin had minimal effects on the entropy estimate (specifically, we saw less than 0.01 bit of entropy difference between the smallest and largest estimated value, 7.493 vs. 7.502). We consider this to be a confirmation that the result is robust.

Note that this entropy measurement is based on the parameters of just a single axis (Z-axis). The identifier bits from the accelerometer's X and Y axes are not included in this experiment. If the parameters of these axes can be measured reliably then we would gain additional entropy, allowing the identification of millions of devices.

For identification, we focused on two-submission devices, and asked whether the second data point for such devices is closest to the first data point from the same device, or to some other data point in the set. If the closest data point is from the same device, we count this device as correctly recognized. Note that this is a rather restrictive protocol because we permit all of the data points collected to impact the recognition, including data points from one-submission devices which may or may not have been carefully measured.

In this most restrictive setup, we managed to correctly identify 298 of the 3583 two-submission devices, a success rate of 8.3%. When we only looked at two-submission devices and further eliminated those whose distance in either O_z or S_z was above the 95th percentile, we were able to identify 543 devices, or 15.1%. While these numbers may look weak, we have to recall that in this identification procedure we are only using the accelerometer, in only one of its dimensions.

Improving identification rate via the User-Agent string. The User-Agent string identifies the type of device connecting to a web site. The User-Agent data, with no other signals, permitted direct identification of 544 devices out of the set. The combination of User-Agent and accelerometer inputs, however, increased the number of correctly identifiable devices to 1900 devices or 53%. Removing 95th percentile devices further increased the correct recognition rate to 58.7%. This shows that the accelerometer fingerprint can be quite effective at distinguishing devices with identical User-Agent strings.

2.6 Estimating All Six Bias Parameters of the Accelerometer

Using the single-axis (Z only) calibration method one could theoretically estimate the sensor parameters in the X and Y dimensions, however it is very unlikely that a user will position the device on its narrow sides. An alternative approach for estimating the bias parameters for all three dimensions is to gather accelerometer measurements at 6 different and arbitrary resting positions. Note that in this approach although we need

to gather measurements for more resting positions we do not assume anything about the device's orientation at those positions. We know that the true acceleration along the v dimension is given by $v_t = \frac{v_m - O_v}{S_v}$. Therefore we have for each measurement m the following equation:

$$\left(\frac{x_m - O_x}{S_x}\right)^2 + \left(\frac{y_m - O_y}{S_y}\right)^2 + \left(\frac{z_m - O_z}{S_z}\right)^2 = g^2$$

In principle, six such equations (for six different resting positions) would have allowed us to calculate the six unknown bias parameters. However, due to quantization errors and other random noises the above equation should be turned into an inequality of the following form:

$$\left(\frac{x_m - O_x}{S_x}\right)^2 + \left(\frac{y_m - O_y}{S_y}\right)^2 + \left(\frac{z_m - O_z}{S_z}\right)^2 - g^2 \leq \epsilon$$

where ϵ is some unknown measurement error. To find the accelerometer deviation parameters in presence of noise we collect multiple measurements $x_m^{(i)}, y_m^{(i)}, z_m^{(i)}$ (as many as it is practical to have) and solve the following optimization problem

Minimize $\sum_i \epsilon_i^2$

subject to

$$\left(\frac{x_m^{(i)} - O_x}{S_x}\right)^2 + \left(\frac{y_m^{(i)} - O_y}{S_y}\right)^2 + \left(\frac{z_m^{(i)} - O_z}{S_z}\right)^2 - g^2 = \epsilon_i$$

where i is the measurement index.

This is not a convex problem in general (depending on the measurements) and therefore we chose to use a numerical gradient descent method to find the parameters $O_x, O_y, O_z, S_x, S_y, S_z$ that minimize the error. Assuming we have a smart guess for the initial point and given the constraints on the reasonable parameter values we expect that solutions for different sample sets will all converge to the same local minima. We take $O_{3 \times 1} = 0$ and $S_{3 \times 1} = 1$ as the initial point for the algorithm since these are the ideal values from which the device can deviate by a small fraction.

We apply this algorithm to multiple sets of measurements for every device, and

obtain labeled samples in a 6-dimensional space. To identify a device given a new set of measurements we repeat the algorithm and obtain an unlabeled sample. We then use nearest neighbor matching (KNN) to associate the sample with a labeled cluster. Cross-validation of KNN classification over this data yielded a correct classification percentage of 81.3%.

2.6.1 Experiment: Lab Droids in 3D

We evaluated the algorithm first for 5 devices and then for 16, performing both unsupervised clustering and supervised classification. K-means clustering for the setup of five devices resulted in a perfect identification of samples obtained from the same device. With 16 devices we obtained good clustering, however we did observe some errors. Supervised classification with 16 devices yields a correct classification percentage in 81.25% of the cases, indicating that this method could be a significant contribution to the overall identification process in combination with the other methods.

2.7 Threat Mitigation

Device identification via sensor fingerprinting has benign as well as malicious uses. In the context of privacy violation for example, it is worth considering the possible methods for mitigating this threat to mobile users. For any particular sensor, the feasibility of fingerprinting can be practically eliminated by calibrating the sensor at the time of manufacturing. A different, software-only approach can be to add a random value to the sensor output at the OS level. This value can remain constant during continuous use of the device, allowing software such as mobile games to calibrate the sensor if needed. During periods of long inactivity, the random value can change—which would invalidate any device fingerprint that may have been collected already.

We also believe that we have also made a good case for re-evaluating the status of sensor data conferred by browsers and mobile operating systems. Until now, sensor streams such as accelerometer readings have not been considered sensitive

information—yet we have demonstrated that they can be used to identify and track devices. As smartphone operating systems and browser technologies mature further, we expect to see more uniform access controls on device sensors.

2.8 Difficulties with using some sensors for identification

In this section we briefly discuss the difficulties we had in using certain sensors listed in Table 2.1 for device fingerprinting.

Gyroscope: Measuring the offset and sensitivity of the gyroscope would require subjecting the device to constant angular velocity rotation at different speeds—an experiment that is difficult to carry out even in a lab.

Magnetometer: We carried out some magnetometer experiments which convinced us that although compass readings are a possible source of identification data, the peculiarities of the sensor make practical use next to impossible. Consider for example Figure 2.10: while sensitivity and offset are evident from the geometry of magnetic field measurements in multiple directions, there are also clear memory effects which can disrupt the estimates. In addition, the variability of the magnetic field can be sometimes significant (e.g. near metallic objects), and sometimes subtle, making corrections difficult and error-prone.

Light: Light sensors only provide erratic measurements which are subject to noise due to partial obstruction of the sensor; thus, light measurements are difficult to put in a context that allows for estimation of the imperfections.

GPS: A GPS receiver triangulates the location of a phone by calculating its distance to at least 3 GPS satellites. The distances are calculated by measuring the time a signal travels from a satellite to the GPS receiver. The travel time is measured using an inaccurate clock built into the GPS receiver. Previous work [102] has shown that a clock’s skew can identify the clock. However, modern GPS receivers utilize a 4th satellite measurement which allows to take this bias into account. Therefore, the clock’s bias does not affect the calculated location.

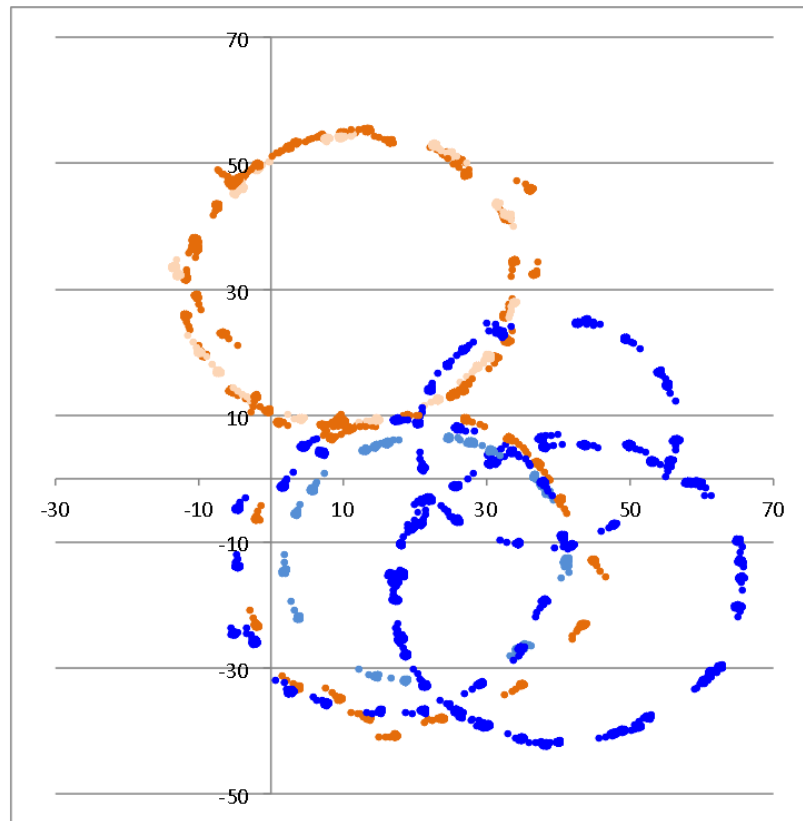


Figure 2.10: Compass (magnetometer) readings taken by placing two different devices (in blue and orange) at the same location on a flat surface, in a variety of orientations. The oval shapes are created due to the varying orientations of the device (with a stable Z component—note that the same device sometimes produces a smaller oval, consistent with a varying Z value across runs, even at the same location). Offset is evident from the offset centers of the ovals, and sensitivity is reflected in the ovals’ shapes. Different runs even at the same physical location may produce different results, which demonstrates memory effects that are difficult to correct for.

Touch screen: The touch screen sensor is mounted over the phone’s display. Inaccurate assembly process may cause the touch screen to be misaligned with the display. This may cause the user to erroneously tap on locations adjacent to the intended target. This misalignment may serve for identification. However, since it is usually very small and mostly goes unnoticed by the user, it is difficult to measure it. One possible direction to measure it would be, for example, to record the exact

locations of the user’s taps on the keyboard display. Averaging the tap locations for each key and comparing it against the actual key’s location at the keyboard display may allow one to calculate the misalignment. Nonetheless, we expect this method to be highly dependent on the user as much as the touch screen misalignment.

Camera: There are a few works that deal with camera identification using the pixels’ bias. The output gain of each pixel is a linear function of the actual intensity of light hitting that pixel. This linear bias is commonly called pattern noise; [113] proposes a method to determine a camera’s reference pattern noise using a 300 pictures taken by that camera. This serves as a unique fingerprint for the camera. It is shown that this enables to associate with good probability a new picture with the camera that took it. The study was done using 9 cameras. However, no effort was done to assess the expected number of cameras that can be distinguished using this method, and what is more, most of the cameras differ in either model or manufacturer which tends to make identification easier (some evidence is presented however that the results can carry over to identical cameras as well).

2.9 Related Work

Sensor fingerprinting has received significant attention in recent years, primarily in the context of de-anonymizing photos by correlating them to images with a known source. In [113], images taken by different cameras are processed to derive a reference noise pattern that is specific for each sensor. Based on this pattern, additional images are associated with their most likely source. Noise extraction algorithms are a critical part of this approach, and [110] proposes some further enhancements. The image capture pipeline is investigated in [57], where different stages of the process are revealed to introduce distinct artifacts. These artifacts can be used to design more robust identification algorithms.

Flash (solid state) storage has also been shown to contain unique defects that can be fingerprinted. Both coupling and timing effects are considered by [132], and shown to yield feasible identification mechanisms. The main difficulty of applying flash fingerprinting to the mobile device domain is the consistent move towards using

eMMC-style flash chips [59], which hide much of the raw data by building in complex wear-leveling logic.

There are some works that aim to fingerprint a device via the web that go beyond the standard HTTP cookies. Such works are based on software-related features rather than hardware related. Ref. [70] showed that parameters of system configuration such as screen resolution, browser plugins and system fonts as well as the contents of HTTP headers – User-Agent and Accept – allow to fingerprint a device. Ref. [157] also showed that good device identification can also be achieved using the values of User-Agent, IP address, cookies and login IDs. These values can be achieved using standard logs of web traffic.

In the past several years it has been shown [42] that many web sites identify a web client based on “super-cookies”. These are identifier which are stored on the local host in various persistent ways outside the control of a browser, hence the browser can not impose that standard restriction as of HTTP cookies.

Some works deal with remote hardware-based fingerprinting. The most well-known example is [102] which showed how to measure a device’s clock skew using ICMP and TCP traffic. The clock’s skew is shown as a good device identifier. There is also a body of work that propose remote fingerprinting methods based on wireless traffic, for example, radiometric analysis of IEEE 802.11 transmitters [52], signal phase identification of bluetooth transmitters [84], or timing analysis of 802.11 probe request frames [66].

There are a few recent works which independently proposed methods to fingerprint accelerometers and loudspeakers. In [67], it is suggested to fingerprint a smartphone device using its accelerometer. The proposed fingerprinting method is based on accelerometer output while the phone is vibrating (e.g. during an incoming call or message). Then machine learning algorithms are used to identify a phone based on general features extracted from the accelerometer output, such as mean, std. dev., and skewness. These features are indirectly based on the offset and sensitivity of the accelerometer. This method requires about 30 seconds of accelerometer recording during vibration, which may be hard to obtain if the phone is not set to vibrating mode. Furthermore, the method proposed in [67] is influenced by the surface on which

the phone lays and the case in which it is enclosed, while our method is oblivious to these since we fingerprinting the accelerometer while it is at rest.

[58] and [61] propose to fingerprint loudspeakers. The schemes proposed in these works focus solely on fingerprinting the loudspeakers; in contrast, our method allows to fingerprinting the loudspeaker and microphone combined, thus potentially allowing for more fingerprint entropy. Moreover, our use of the device’s microphone removes the need for an external microphone during the fingerprinting process and allows for a more practical attack scenario. Our scheme relies on short synthesized sounds that can be generated at the appropriate timing. [61] relies on recording ring-tones and therefore the attacker has to wait for an incoming call (or other event) to trigger the sound.

Finally, a concurrent work by Das et al. [62] studied the feasibility and de-facto usage of smartphone fingerprinting via motion sensors.

2.10 Future work

We presented a new approach to mobile device identification which allows for devices to be recognized without relying on soft identifiers (which may be lost after a device reset). Our fingerprinting method exploits sensor calibration variations in the speaker-microphone system and in the accelerometer. Accelerometer-based identification is particularly noteworthy because it can be performed by untrusted web code running within a mobile browser. We hope that our results illustrate the potential risk of granting untrusted code access to seemingly benign hardware.

This work raises several interesting open problems. What other types of mobile hardware can be leveraged for device fingerprinting? Can this be done using imperfections in the baseband processor? In other sensors? How much entropy can be extracted overall and, based on additional data from a larger set of identical devices, can we obtain a high-confidence estimate of the distributions of the measured calibration parameters? Is there sufficient entropy in sensor-based fingerprinting to generate a hardware-based cryptographic key? We hope these questions can be answered by future work.

Chapter 3

Gyrophone: Recognizing Speech from Gyroscope Signals

Modern smartphones and mobile devices have many sensors that enable rich user experience. Being generally put to good use, they can sometimes unintentionally expose information the user does not want to share. While the privacy risks associated with some sensors like a microphone (eavesdropping), camera or GPS (tracking) are obvious and well understood, some of the risks remained under the radar for users and application developers. In particular, access to motion sensors such as gyroscope and accelerometer is unmitigated by mobile operating systems. Namely, every application installed on a phone and every web page browsed over it can measure and record these sensors without the user being aware of it.

Recently, a few research works pointed out unintended information leaks using motion sensors. In Ref. [117] the authors suggest a method for user identification from gait patterns obtained from a mobile device's accelerometers. The feasibility of keystroke inference from nearby keyboards using accelerometers has been shown in [119]. In [53], the authors demonstrate the possibility of keystroke inference on a mobile device using accelerometers and mention the potential of using gyroscope measurements as well, while another study [32] points to the benefits of exploiting the gyroscope.

All of the above work focused on exploitation of motion events obtained from

the sensors, utilizing the expected kinetic response of accelerometers and gyroscopes. In this chapter we reveal a new way to extract information from gyroscope measurements. We show that gyroscopes are sufficiently sensitive to measure acoustic vibrations. This leads to the possibility of recovering speech from gyroscope readings, namely using the gyroscope as a crude microphone. We show that the sampling rate of the gyroscope is up to 200 Hz which covers some of the audible range. This raises the possibility of eavesdropping on speech in the vicinity of a phone without access to the real microphone.

As the sampling rate of the gyroscope is limited, one cannot fully reconstruct a comprehensible speech from measurements of a single gyroscope. Therefore, we resort to automatic speech recognition. We extract features from the gyroscope measurements using various signal processing methods and train machine learning algorithms for recognition. We achieve about 50% success rate for speaker identification from a set of 10 speakers. We also show that while limiting ourselves to a small vocabulary consisting solely of digit pronunciations ("one", "two", "three", ...) and achieve speech recognition success rate of 65% for the speaker dependent case and up to 26% recognition rate for the speaker independent case. This capability allows an attacker to substantially leak information about numbers spoken over or next to a phone (i.e. credit card numbers, social security numbers and the like).

We also consider the setting of a conference room where two or more people are carrying smartphones or tablets. This setting allows an attacker to gain simultaneous measurements of speech from several gyroscopes. We show that by combining the signals from two or more phones we can increase the effective sampling rate of the acoustic signal while achieving better speech recognition rates. In our experiments we achieved 77% successful recognition rate in the speaker dependent case based on the digits vocabulary.

This chapter is structured as follows: in section 3.1 we provide a brief description of how a MEMS gyroscope works and present initial investigation of its properties as a microphone. In section 3.2 we discuss speech analysis and describe our algorithms for speaker and speech recognition. In section 3.3 we suggest a method for audio signal recovery using samples from multiple devices. In section 3.4 we discuss more

directions for exploitation of gyroscopes' acoustic sensitivity. Finally, in Section 4.8 we discuss mitigation measures of this unexpected threat. In particular, we argue that restricting the sampling rate is an effective and backwards compatible solution.

3.1 Gyroscope as a microphone

In this section we explain how MEMS gyroscopes operate and present an initial investigation of their susceptibility to acoustic signals.

3.1.1 How does a MEMS gyroscope work?

Standard-size (non-MEMS) gyroscopes are usually composed of a spinning wheel on an axle that is free to assume any orientation. Based on the principles of angular momentum the wheel resists to changes in orientation, thereby allowing to measure those changes. Nonetheless, all MEMS gyros take advantage of a different physical phenomenon – the Coriolis force. It is a fictitious force (d'Alembert force) that appears to act on an object while viewing it from a rotating reference frame (much like the centrifugal force). The Coriolis force acts in a direction perpendicular to the rotation axis of the reference frame and to the velocity of the viewed object. The Coriolis force is calculated by $F = 2m\vec{v} \times \vec{\omega}$ where m and v denote the object's mass and velocity, respectively, and ω denotes the angular rate of the reference frame.

Generally speaking, MEMS gyros measure their angular rate (ω) by sensing the magnitude of the Coriolis force acting on a moving proof mass within the gyro. Usually the moving proof mass constantly vibrates within the gyro. Its vibration frequency is also called the resonance frequency of the gyro. The Coriolis force is sensed by measuring its resulting vibration, which is orthogonal to the primary vibration movement. Some gyroscope designs use a single mass to measure the angular rate of different axes, while others use multiple masses. Such a general design is commonly called *vibrating structure gyroscope*.

There are two primary vendors of MEMS gyroscopes for mobile devices: STMicroelectronics [25] and InvenSense [12]. According to a recent survey [30] STMicroelectronics dominates with 80% market share. Teardown analyses show that this vendor's gyros can be found in Apple's iPhones and iPads [29, 13] and also in the latest generations of Samsung's Galaxy-line phones [10, 11]. The second vendor, InvenSense, has the remaining 20% market share [30]. InvenSense gyros can be found in Google's latest generations of Nexus-line phones and tablets [21, 20] as well as in Galaxy-line tablets [9, 8]. These two vendors' gyroscopes have different mechanical designs, but are both noticeably influenced by acoustic noise.

STMicroelectronics

The design of STMicroelectronics 3-axis gyros is based on a single driving (vibrating) mass (shown in Figure 3.1). The driving mass consists of 4 parts M_1 , M_2 , M_3 and M_4 (Figure 3.1(b)). They move inward and outward simultaneously at a certain frequency¹ in the horizontal plane. As shown in Figure 3.1(b), when an angular rate is applied on the Z -axis, due to the Coriolis effect, M_2 and M_4 will move in the same horizontal plane in opposite directions as shown by the red and yellow arrows. When an angular rate is applied on the X -axis, then M_1 and M_3 will move in opposite directions up and down out of the plane due to the Coriolis effect. When an angular rate is applied to the Y -axis, then M_2 and M_4 will move in opposite directions up and down out of the plane. The movement of the driving mass causes a capacitance change relative to stationary plates surrounding it. This change is sensed and translated into the measurement signal.

InvenSense

InvenSense's gyro design is based on the three separate driving (vibrating) masses²; each senses angular rate at a different axis (shown in Figure 3.2(a)). Each mass is a coupled dual-mass that move in opposite directions. The masses that sense the X and Y axes are driven out-of-plane (see Figure 3.2(b)), while the Z -axis mass is driven

¹It is indicated in [1] that STMicroelectronics uses a driving frequency of over 20 KHz.

²According to [143] the driving frequency of the masses is between 25 KHz and 30 KHz.

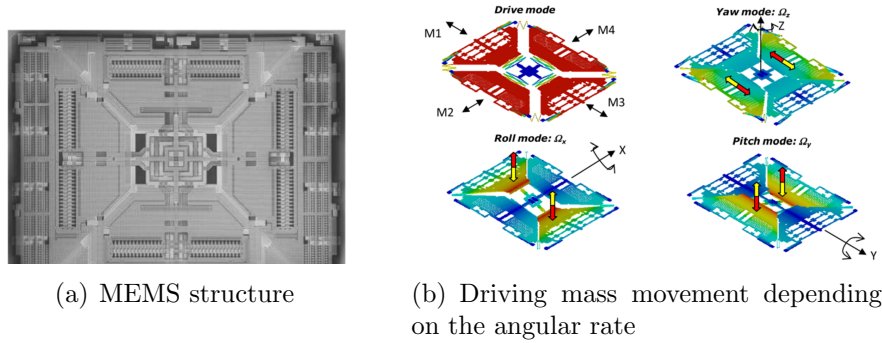


Figure 3.1: STMicroelectronics 3-axis gyro design (Taken from [28]. Figure copyright of STMicroelectronics. Used with permission.)

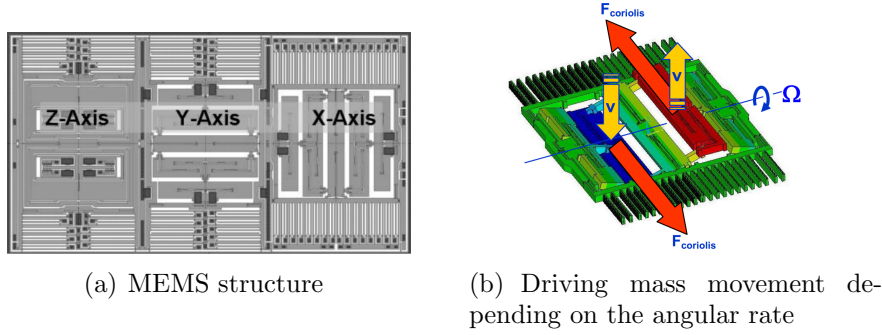


Figure 3.2: InvenSense 3-axis gyro design (Taken from [143]. Figure copyright of InvenSense. Used with permission.)

in-plane. As in the STMicroelectronics design the movement due to the Coriolis force is measured by capacitance changes.

3.1.2 Acoustic Effects

It is a well known fact in the MEMS community that MEMS gyros are susceptible to acoustic noise which degrades their accuracy [56, 65, 64]. An acoustic signal affects the gyroscope measurement by making the driving mass vibrate in the sensing axis (the axis which senses the Coriolis force). The acoustic signal can be transferred to the driving mass in one of two ways. First, it may induce mechanical vibrations to the gyros package. Additionally, the acoustic signal can travel through the gyroscope

packaging and directly affect the driving mass in case it is suspended in air. The acoustic noise has the most substantial effect when it is near the resonance frequency of the vibrating mass. Such effects in some cases can render the gyro's measurements useless or even saturated. Therefore to reduce the noise effects vendors manufacture gyros with a high resonance frequency (above 20 KHz) where acoustic signals are minimal. Nonetheless, in our experiments we found that acoustic signals at frequencies much lower than the resonance frequency still have a measurable effect on a gyro's measurements, allowing one to reconstruct the acoustic signal.

3.1.3 Characteristics of a gyro as a microphone

Due to the gyro's acoustic susceptibility one can treat gyroscope readings as if they were audio samples coming from a microphone. Note that the frequency of an audible signal is higher than 20 Hz, while in common cases the frequency of change of mobile device's angular velocity is lower than 20 cycles per second. Therefore, one can high-pass-filter the gyroscope readings in order to retain only the effects of an audio signal even if the mobile device is moving about. Nonetheless, it should be noted that this filtering may result in some loss of acoustic information since some aliased frequencies may be filtered out (see Section 3.1.3). In the following we explore the gyroscope characteristics from a standpoint of an acoustic sensor, i.e. a microphone. In this section we exemplify these characteristics by experimenting with Galaxy S III which has an STMicroelectronics gyro [11].

Sampling

Sampling resolution is measured by the number of bits per sample. More bits allow us to sample the signal more accurately at any given time. All the latest generations of gyroscopes have a sample resolution of 16 bits [14, 18]. This is comparable to a microphone's sampling resolution used in most audio applications.

Sampling frequency is the rate at which a signal is sampled. According to the Nyquist sampling theorem a sampling frequency f enables us to reconstruct signals at frequencies of up to $f/2$. Hence, a higher sampling frequency allows us to more accurately

		Sampling Freq. [Hz]
Android 4.4	application	200
	Chrome	25
	Firefox	200
	Opera	20
iOS 7	application	100 [6]
	Safari	20
	Chrome	20

Table 3.1: Maximum sampling frequencies on different platforms

reconstruct the audio signal. In most mobile devices and operating systems an application is able to sample the output of a microphone at up to 44.1 KHz. A telephone system (POTS) samples an audio signal at 8000 Hz. However, STMicroelectronics' gyroscope hardware supports sampling frequencies of up to 800 Hz [14], while InvenSense gyros' hardware support sampling frequency up to 8000 Hz [18]. Moreover, all mobile operating systems bound the sampling frequency even further – up to 200 Hz – to limit power consumption. On top of that, it appears that some browser toolkits limit the sampling frequency even further. Table 3.1 summarizes the results of our experiments measuring the maximum sampling frequencies allowed in the latest versions of Android and iOS both for application and for web application running on common browsers. The code we used to sample the gyro via a web page can be found in Appendix A.2. The results indicate that a Gecko based browser does not limit the sampling frequency beyond the limit imposed by the operating system, while WebKit and Blink based browsers does impose stricter limits on it.

Aliasing

As noted above, the sampling frequency of a gyro is uniform and can be at most 200 Hz. This allows us to directly sense audio signals of up to 100 Hz. Aliasing is a phenomenon where for a sinusoid of frequency f , sampled with frequency f_s , the resulting samples are indistinguishable from those of another sinusoid of frequency $|f - N \cdot f_s|$, for any integer N . The values corresponding to $N \neq 0$ are called images or aliases of frequency f . An undesirable phenomenon in general, here aliasing

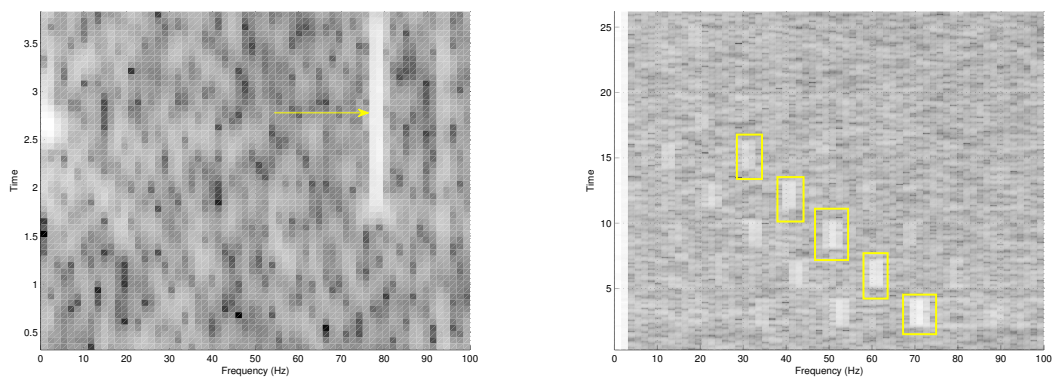
allows us to sense audio signals having frequencies which are higher than 100 Hz, thereby extracting more information from the gyroscope readings. This is illustrated in Figure 3.3.

Using the gyro, we recorded a single 280 Hz tone. Figure 3.3(a) depicts the recorded signal in the frequency domain (x-axis) over time (y-axis). A lighter shade in the spectrogram indicates a stronger signal at the corresponding frequency and time values. It can be clearly seen that there is a strong signal sensed at frequency 80 Hz starting around 1.5 sec. This is an alias of the 280 Hz-tone. Note that the aliased tone is indistinguishable from an actual tone at the aliased frequency. Figure 3.3(b) depicts a recording of multiple short tones between 130 Hz and 200 Hz. Again, a strong signal can be seen at the aliased frequencies corresponding to 130 - 170 Hz³. We also observe some weaker aliases that do not correspond to the base frequencies of the recorded tones, and perhaps correspond to their harmonics. Figure 3.3(c) depicts the recording of a chirp in the range of 420 - 480 Hz. The aliased chirp is detectable in the range of 20 - 80 Hz; however it is a rather weak signal.

Self noise

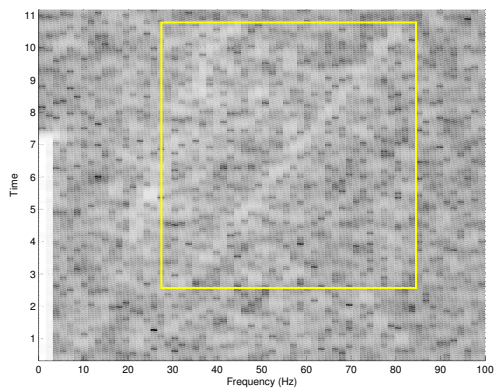
The self noise characteristic of a microphone indicates what is the most quiet sound, in decibels, a microphone can pick up, i.e. the sound that is just over its self noise. To measure the gyroscope's self noise we played 80 Hz tones for 10 seconds at different volumes while measuring it using a decibel meter. Each tone was recorded by the Galaxy S III gyroscope. While analyzing the gyro recordings we realized that the gyro readings have a noticeable increase in amplitude when playing tones with volume of 75 dB or higher which is comparable to the volume of a loud conversation. Moreover, a FFT plot of the gyroscope recordings gives a noticeable peak at the tone's frequency when playing tone with a volume as low as 57 dB which is below the sound level of a normal conversation. These findings indicate that a gyro can pick up audio signals which are lower than 100 HZ during most conversations made over or next to the phone. To test the self noise of the gyro for aliased tones we played 150 Hz

³We do not see the aliases corresponding to 180 - 200 Hz, which might be masked by the noise at low frequencies, i.e., under 20 Hz.



(a) A single 280 Hz tone

(b) Multiple tones in the range of 130 – 170 Hz



(c) A chirp in the range of 420 – 480 Hz

Figure 3.3: Examples of aliasing on a mobile device. Nexus 4 (a,c) and Galaxy SII (b).

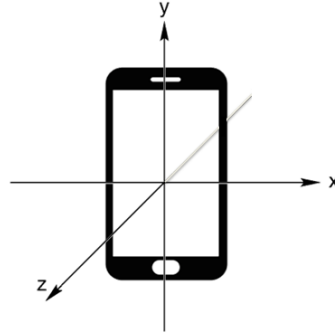


Figure 3.4: Coordinate system of Android and iOS.

Tone direction:	X			Y			Z		
Recording direction:	x	y	z	x	y	z	x	y	z
Amplitude:	0.002	0.012	0.0024	0.01	0.007	0.004	0.007	0.0036	0.0003

Table 3.2: Sensed amplitude for every direction of a tone played at different orientations relative to the phone. For each orientation the dominant sensed directions are emphasized.

and 250 Hz tones. The lowest level of sound the gyro picked up was 67 dB and 77 dB, respectively. These are much higher values that are comparable to a loud conversation.

Directionality

We now measure how the angle at which the audio signal hits the phone affects the gyro. For this experiment we played an 80 Hz tone at the same volume three times. The tone was recorded at each time by the Galaxy S III gyro while the phone rested at a different orientation allowing the signal to hit it parallel to one of its three axes (see Figure 3.4). The gyroscope senses in three axes, hence for each measurement the gyro actually outputs three readings – one per axis. As we show next this property benefits the gyro’s ability to pick up audio signals from every direction. For each recording we calculated the FFT magnitude at 80 Hz. Table 3.2 summarizes the results.

It is obvious from the table that for each direction the audio hit the gyro, there is

at least one axis whose readings are dominant by an order of magnitude compared to the rest. This can be explained by STMicroelectronics gyroscope design as depicted in Figure 3.1⁴. When the signal travels in parallel to the phone's x or y axes, the sound pressure vibrates mostly masses laid along the respective axis, i.e. M_2 and M_4 for x axis and M_1 and M_3 for the y axis; therefore, the gyro primarily senses a rotation at the y or x axes, respectively (see Section 3.1.1). When the signal travels in parallel to the phone's z axis then the sound pressure vibrates all the 4 masses up and down, hence the gyro primarily senses a rotation at both x and y axes.

These findings indicate that the gyro is an omni-directional audio sensor allowing it to pick up audio signal from every direction.

3.2 Speech analysis based on a single gyroscope

In this section we show that the acoustic signal measured by a single gyroscope is sufficient to extract information about the speech signal, such as speaker characteristics and identity, and even recognize the spoken words or phrases. We do so by leveraging the fact that aliasing causes information leaks from higher frequency bands into the sub-Nyquist range.

Since the fundamentals of human voices are roughly in the range of 80 – 1100 Hz [38], we can capture a large fraction of the interesting frequencies, considering the results we observe in 3.1.3. Although we do not delve into comparing performance for different types of speakers, one might expect that given a stronger gyroscope response for low frequencies, typical adult male speech (Bass, Baritone, Tenor) could be better analyzed than typical female or child speech (Alto, Mezzo-Soprano, Soprano)⁵, however our tests show that this is not necessarily the case.

The signal recording, as captured by the gyroscope, is not comprehensible to a human ear, and exhibits a mixture of low frequencies and aliases of frequencies beyond the Nyquist sampling frequency (which is 1/2 the sampling rate of the Gyroscope, i.e. 100 Hz). While the signal recorded by a single device does not resemble speech,

⁴This is the design of the gyro built into Galaxy S III.

⁵For more information about vocal range see http://www.wikipedia.org/wiki/Vocal_range.

it is possible to train a machine to transcribe the signal with significant success.

Speech recognition tasks can be classified into several types according to the setup. Speech recognition can handle fluent speech or isolated words (or phrases); operate on a closed set of words (finite dictionary) or an open set⁶; It can also be speaker dependent (in which case the recognizer is trained per speaker) or speaker independent (in which case the recognizer is expected to identify phrases pronounced by different speakers and possibly ones that were not encountered in the training set). Additionally, speech analysis may be also used to identify the speaker.

We focused on speaker identification (including gender identification of the speaker) and isolated words recognition while attempting both speaker independent and speaker dependent recognition. Although we do not demonstrate fluent speech transcription, we suggest that successful isolated words recognition could be fairly easily transformed into a transcription algorithm by incorporating word slicing and HMM [134]. We did not aim to implement a state-of-the-art speech recognition algorithm, nor to thoroughly evaluate or do a comparative analysis of the classification tests. Instead, we tried to indicate the potential risk by showing significant success rates of our speech analysis algorithms compared to randomly guessing. This section describes speech analysis techniques that are common in practice, our approach, and suggestions for further improvements upon it.

3.2.1 Speech processing: features and algorithms

Features

It is common for various feature extraction methods to view speech as a process that is stationary for short time windows. Therefore speech processing usually involves segmentation of the signal to short (10 – 30 ms) overlapping or non-overlapping windows and operation on them. This results in a time-series of features that characterize the time-dependent behavior of the signal. If we are interested in time-independent properties we shall use spectral features or the statistics of those time-series (such as mean, variance, skewness and kurtosis).

⁶For example by identifying phonemes and combining them to words.

Mel-Frequency Cepstral Coefficients (MFCC) are widely used features in audio and speech processing applications. The Mel-scale basically compensates for the non-linear frequency response of the human ear⁷. The Cepstrum transformation is an attempt to separate the excitation signal originated by air passing through the vocal tract from the effect of the vocal tract (acting as a filter shaping that excitation signal). The latter is more important for the analysis of the vocal signal. It is also common to take the first and second derivatives of the MFCC as additional features, indicative of temporal changes [87].

Short Time Fourier Transform (STFT) is essentially a spectrogram of the signal. Windowing is applied to short overlapping segments of the signal and FFT is computed. The result captures both spectral and time-dependent features of the signal.

Classifiers

Support Vector Machine (SVM) is a general binary classifier, trained to distinguish to groups. We use SVM to distinguish male and female speakers. Multi-class SVMs can be constructed using multiple binary SVMs, to distinguish between multiple groups. We used a multi-class SVM to distinguish between multiple speakers, and to recognize words from a limited dictionary.

Gaussian Mixture Model (GMM) has been successfully used for speaker identification [137]. We can train a GMM for each group in the training stage. In the testing stage we can obtain a match score for the sample using each one of the GMMs and classify the sample according to the group corresponding to the GMM that yields the maximum score.

Dynamic Time Warping (DTW) is a time-series matching and alignment technique [126]. It can be used to match time-dependent features in presence of misalignment or when the series are of different lengths. One of the challenges in word

⁷Approximated as logarithmic by the Mel-scale

recognition is that the samples may differ in length, resulting in different number of segments used to extract features.

3.2.2 Speaker identification algorithm

Prior to processing we converted the gyroscope recordings to audio files in WAV format while upsampling them to 8 KHz⁸. We applied silence removal to include only relevant information and minimize noise. The silence removal algorithm was based on the implementation in [79], which classifies the speech into voiced and unvoiced segments (filtering out the unvoiced) according to dynamically set thresholds for Short-Time Energy and Spectral Centroid features computed on short segments of the speech signal. Note that the gyroscope's zero-offset yields particularly noisy recordings even during unvoiced segments.

We used statistical features based on the first 13 MFCC computed on 40 sub-bands. For each MFCC we computed the mean and standard deviation. Those features reflect the spectral properties which are independent of the pronounced word. We also use delta-MFCC (the derivatives of the MFCC), RMS Energy and Spectral Centroid statistical features. We used MIRToolbox [106] for the feature computation. It is important to note that while MFCC have a physical meaning for real speech signal, in our case of an narrow-band aliased signal, MFCC don't necessarily have an advantage, and were used partially because of availability in MIRToolbox. We attempted to identify the gender of the speaker, distinguish between different speakers of the same gender and distinguish between different speakers in a mixed set of male and female speakers. For gender identification we used a binary SVM, and for speaker identification we used multi-class SVM and GMM. We also attempted gender and speaker recognition using DTW with STFT features. All STFT features were computed with a window of 512 samples which, for sampling rate of 8 KHz, corresponds to 64 ms.

⁸Although upsampling the signal from 200 Hz to 8 KHz does not increase the accuracy of audio signal, it is more convenient to handle the WAV file at higher sampling rate with standard speech processing tools.

3.2.3 Speech recognition algorithm

The preprocessing stage for speech recognition is the same as for speaker identification. Silence removal is particularly important here, as the noisy unvoiced segments can confuse the algorithm, by increasing similarity with irrelevant samples. For word recognition, we are less interested in the spectral statistical features, but rather in the development of the features in time, and therefore suitable features could be obtained by taking the full spectrogram. In the classification stage we extract the same features for a sample y . For each possible label l we obtain a similarity score of the y with each sample X_i^l corresponding to that guess in the training set. Let us denote this similarity function by $D(y, X_i^l)$. Since different samples of the same word can differ in length, we use DTW. We sum the similarities to obtain a total score for that guess

$$S^l = \sum_i D(y, X_i^l)$$

After obtaining a total score for all possible words, the sample is classified according to the maximum total score

$$C(y) = \operatorname{argmax}_l S^l$$

3.2.4 Experiment setup

Our setup consisted of a set of loudspeakers that included a sub-woofer and two tweeters (depicted in Figure 3.5). The sub-woofer was particularly important for experimenting with low-frequency tones below 200 Hz. The playback was done at volume of approximately 75 dB to obtain as high SNR as possible for our experiments. This means that for more restrictive attack scenarios (farther source, lower volume) there will be a need to handle low SNR, perhaps by filtering out the noise or applying some other preprocessing for emphasizing the speech signal.⁹

⁹We tried recording in an anechoic chamber, but it didn't seem to provide better recognition results compared to a regular room. We therefore did not proceed with the anechoic chamber experiments. Yet, further testing is needed to understand whether we can benefit significantly from an anechoic environment.



Figure 3.5: Experimental setup

Data

Due to the low sampling frequency of the gyro, a recognition of speaker-independent general speech would be an ambitious long-term task. Therefore, in this work we set out to recognize speech of a limited dictionary, the recognition of which would still leak substantial private information. For this work we chose to focus on the digits dictionary, which includes the words: zero, one, two..., nine, and "oh". Recognition of such words would enable an attacker to eavesdrop on private information, such as credit card numbers, telephone numbers, social security numbers and the like. This information may be eavesdropped when the victim speaks over or next to the phone.

In our experiments, we use the following corpus of audio signals on which we tested our recognition algorithms.

TIDIGITS This is a subset of a corpus published in [108]. It includes speech of isolated digits, i.e., 11 words per speaker where each speaker recorded each word twice. There are 10 speakers (5 female and 5 male). In total, there are $10 \times 11 \times 2 = 220$ recordings. The corpus is digitized at 20 kHz.

Mobile devices

We primarily conducted our experiments using the following mobile devices:

1. Nexus 4 phone which according to a teardown analysis [20] is equipped with an

InvenSense MPU-6050 [18] gyroscope and accelerometer chip.

2. Nexus 7 tablet which according to a teardown analysis [21] is equipped with an InvenSense MPU-6050 gyroscope and accelerometer.
3. Samsung Galaxy S III phone which according to a teardown analysis [11] is equipped with an STMicroelectronics LSM330DLC [16] gyroscope and accelerometer chip.

3.2.5 Sphinx

We first try to recognize digit pronunciations using general-purpose speech recognition software. We used Sphinx-4 [153] – a well-known open-source speech recognizer and trainer developed in Carnegie Mellon University. Our aim for Sphinx is to recognize gyro-recordings of the TIDIGITS corpus. As a first step, in order to test the waters, instead of using actual gyro recordings we downsampled the recordings of the TIDIGITS corpus to 200 Hz; then we trained Sphinx based on the modified recordings. The aim of this experiment is to understand whether Sphinx detects any useful information from the sub-100 Hz band of human speech. Sphinx had a reasonable success rate, recognizing about 40% of pronunciations.

Encouraged by the above experiment we then recorded the TIDIGITS corpus using a gyro – both for Galaxy S III and Nexus 4. Since Sphinx accepts recording in WAV format we had to convert the raw gyro recordings. Note that at this point for each gyro recording we had 3 WAV files, one for each gyro axis. The final stage is silence removal. Then we trained Sphinx to create a model based on a training subset of the TIDIGITS, and tested it using the complement of this subset.

The recognition rates for either axes and either Nexus 4 or Galaxy S III were rather poor: 14% on average. This presents only marginal improvement over the expected success of a random guess which would be 9%.

This poor result can be explained by the fact that Sphinx’s recognition algorithms are geared towards standard speech recognition tasks where most of the voice-band is present and is less suited to speech with very low sampling frequency.

	SVM	GMM	DTW
Nexus 4	80%	72%	84%
Galaxy S III	82%	68%	58%

Table 3.3: Speaker’s gender identification results

		SVM	GMM	DTW
Nexus 4	Mixed female/male	23%	21%	50%
	Female speakers	33%	32%	45%
	Male speakers	38%	26%	65%
Galaxy S III	Mixed female/male	20%	19%	17%
	Female speakers	30%	20%	29%
	Male speakers	32%	21%	25%

Table 3.4: Speaker identification results

3.2.6 Custom recognition algorithms

In this section we present the results obtained using our custom algorithm. Based on the TIDIGITS corpus we randomly performed a 10-fold cross-validation. We refer mainly to the results obtained using Nexus 4 gyroscope readings in our discussion. We also included in the tables some results obtained using a Galaxy III device, for comparison.

Results for gender identification are presented in Table 3.3. As we see, using DTW scoring for STFT features yielded a much better success rate.

Results for speaker identification are presented in Table 3.4. Since the results for

		SVM	GMM	DTW
Nexus 4	Mixed female/male	10%	9%	17%
	Female speakers	10%	9%	26%
	Male speakers	10%	10%	23%
Galaxy S III	Mixed female/male	7%	12%	7%
	Female speakers	10%	10%	12%
	Male speakers	10%	6%	7%

Table 3.5: Speaker-independent case – isolated words recognition results

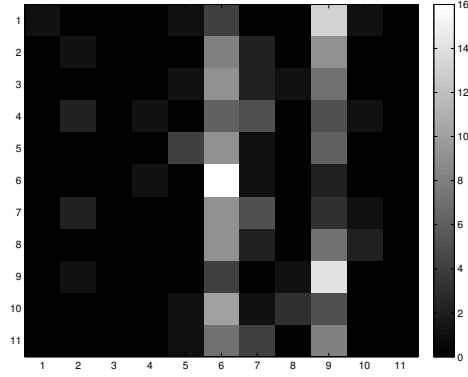


Figure 3.6: Speaker independent word recognition using DTW: confusion matrix as a heat map. $c_{(i,j)}$ corresponds to the number of samples from group i that were classified as j , where i, j are the row and column indices respectively.

SVM	GMM	DTW
15%	5%	65%

Table 3.6: Speaker-dependent case – isolated words recognition for a single speaker. Results obtained via "leave-one-out" cross-validation on 44 recorded words pronounced by a single speaker. Recorded using a Nexus 4 device.

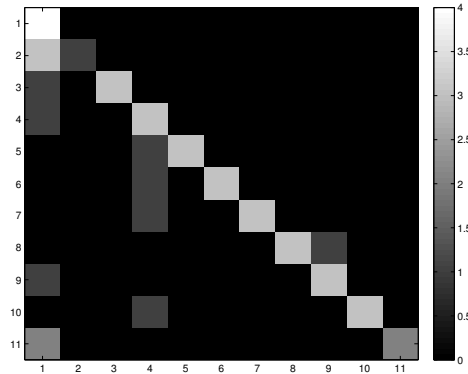


Figure 3.7: Speaker dependent word recognition using DTW: confusion matrix as a heat map.

a mixed female-male set of speakers may be partially attributed to successful gender identification, we tested classification for speakers of the same gender. In this setup we have 5 different speakers. The improved classification rate (except for DTW for female speaker set) can be partially attributed to a smaller number of speakers.

The results for speaker-independent isolated word recognition are summarized in Table 3.5. We had correct classification rate of $\sim 10\%$ using multi-class SVM and GMM trained with MFCC statistical features, which is almost equivalent to a random guess. Using DTW with STFT features we got 23% correct classification for male speakers, 26% for female speakers and 17% for a mixed set of both female and male speakers. The confusion matrix in Figure 3.6, corresponding to the mixed speaker-set recorded on a Nexus 4, explains the not so high recognition rate, exhibiting many false positives for the words "6" and "9". At the same time the recognition rate for these particular words is high, contributing to the correct identification rate.

For a speaker-dependent case one may expect to get better recognition results. We recorded a set of 44 digit pronunciations, where each digit was pronounced 4 times. We tested the performance of our classifiers using "leave-one-out" cross-validation. The results are presented in Table 3.6, and as we expected exhibit an improvement compared to the speaker independent recognition¹⁰ (except for GMM performance that is equivalent to randomly guessing). The confusion matrix corresponding to speaker-dependent word recognition using DTW is presented in Figure 3.7.

DTW method outperforms SVM and GMM in most cases. One would expect that DTW would perform better for word recognition since the changing in time of the spectral features is taken into account. While true for Nexus 4 devices it did not hold for measurements taken with Galaxy III. possible explanation to that is that the low-pass filtering on the Galaxy III device renders all methods quite ineffective resulting in a success rate equivalent to a random guess. For gender and speaker identification, we would expect statistical spectral features based methods (SVM and GMM) to perform at least as good as DTW. It is only true for the Galaxy S III mixed speaker set and gender identification cases, but not for the other experiments. Specifically for gender

¹⁰It is the place to mention that a larger training set for speaker independent word recognition is likely to yield better results. For our tests we used relatively small training and evaluation sets.

identification, capturing the temporal development of the spectral feature wouldn't seem like a clear advantage and is therefore somewhat surprising. One comparative study that supports the advantage of DTW over SVM for speaker recognition is [156]. It doesn't explain though why it outperforms GMM which is a well established method for speaker identification. More experimentation is required to confirm whether this phenomenon is consistent and whether it is related to capturing the high frequencies.

3.2.7 Further improvement

We suggest several possible future improvements on our recognition algorithms. Phoneme recognition instead of whole words, in combination with an HMM could improve the recognition results. This could be more suitable since different pronunciations have different lengths, while an HMM could introduce a better probabilistic recognition of the words. Pre-filtering of the signal could be beneficial and reduce irrelevant noise. It is not clear which frequencies should be filtered and therefore some experimentation is needed to determine it.

For our experiments, we used samples recorded by the gyroscope for training. For speaker-dependent speech recognition we can imagine it may be easier to obtain regular speech samples for a particular speaker than a transcribed recording of gyroscope samples. Even for speaker independent speech recognition, it would be easier to use existing audio corpora for training a speech recognition engine than to produce gyroscope recordings for a large set of words. For that purpose it would be interesting to test how well the recognition can perform when the training set is based on normal audio recordings, downsampled to 200 Hz to simulate a gyroscope recording.

Another possible improvement is to leverage the 3-axis recordings. It is obvious that the three recordings are correlated while the noise of gyro readings is not. Hence, one may take advantage of this to get a composed signal of the three axes to get a better signal-to-noise ratio.

While we suggested that the signal components related to speech, and those related to motion lie in separate frequency bands, the performance of speech analysis in the presence of such noise is yet to be evaluated.

3.3 Reconstruction using multiple devices

In this section we suggest that isolated word recognition can be improved if we sample the gyroscopes of multiple devices that are in close proximity, such that they exhibit a similar response to the acoustic signals around them. This can happen for instance in a conference room where two mobile devices are running malicious applications or, having a browser supporting high-rate sampling of the gyroscope, are tricked into browsing to a malicious website.

We do not refer here to the possibility of using several different gyroscope readings to effectively obtain a larger feature vector, or have the classification algorithm take into account the score obtained for all readings. While such methods to exploit the presence of more than one acoustic side-channel may prove very efficient we leave them outside the scope of this study. It also makes sense to look into existing methods for enhancing speech recognition using multiple microphones, covered in signal processing and machine learning literature (e.g., [60]).

Instead, we look at the possibility of obtaining an enhanced signal by using all of the samples for reconstruction, thus effectively obtaining higher sampling rate. Moreover, we hint at the more ambitious task of reconstructing a signal adequate enough to be comprehensible by a human listener, in a case where we gain access to readings from several compromised devices. While there are several practical obstacles to it, we outline the idea, and demonstrate how partial implementation of it facilitates the automatic speech recognition task.

We can look at our system as an array of time-interleaved data converters (interleaved ADCs). Interleaved ADCs are multiple sampling devices where each samples the signal with a sub-Nyquist frequency. While the ADCs should ideally have time offsets corresponding to a uniform sampling grid (which would allow to simply interleave the samples and reconstruct according to the Whittaker-Shannon interpolation formula [144]), usually there will be small time skews. Also, DC offsets and different input gains can affect the result and must all be compensated.

This problem is studied in a context of analog design and motivated by the need to sample high-frequency signals using low-cost and energy-efficient low-frequency

A/D converters. While many papers on the subject exist, such as [73], the proposed algorithms are usually very hardware centric, oriented towards real-time processing at high-speed, and mostly capable of compensating for very small skews. Some of them require one ADC that samples the signal above the Nyquist rate, which is not available in our case. At the same time, we do not aim for a very efficient, real-time algorithm. Utilizing recordings from multiple devices implies offline processing of the recordings, and we can afford a long run-time for the task.

The ADCs in our case have the same sampling rate $F_s = 1/T = 200$. We assume the time-skews between them are random in the range $[0, T_Q]$ where for N ADCs $T_Q = \frac{T}{N}$ is the Nyquist sampling period. Being located at different distances from the acoustic source they are likely to exhibit considerably different input gains, and possibly have some DC offset. [72] provides background for understanding the problems arising in this configuration and covers some possible solutions.

3.3.1 Reconstruction algorithm

Signal offset correction

To correct a constant offset we can take the mean of the Gyro samples and compare it to 0 to get the constant offset. It is essentially a simple DC component removal.

Gain mismatch correction

Gain mismatch correction is crucial for a successful signal reconstruction. We correct the gain by normalizing the signal to have standard deviation equal to 1. In case we are provided with some reference signal with a known peak, we can adjust the gains of the recordings so that the amplitude at this peak is equal for all of them.

Time mismatch correction

While gyroscope motion events are provided with precise timestamps set by the hardware, which theoretically could have been used for aligning the recordings, in practice, we cannot rely on the clocks of the mobile devices to be synchronized. Even if we take

the trouble of synchronizing the mobile device clock via NTP, or even better, a GPS clock, the delays introduced by the network, operating system and further clock-drift will stand in the way of having clock accuracy on the order of a millisecond¹¹. While not enough by itself, such synchronization is still useful for coarse alignment of the samples.

El-Manar describes foreground and background time-mismatch calibration techniques in his thesis [73]. Foreground calibration means there is a known signal used to synchronize all the ADCs. While for the purpose of testing we can align the recordings by maximizing the cross-correlation with a known signal, played before we start recording, in an actual attack scenario we probably won't be able to use such a marker¹². Nevertheless, in our tests we attempted aligning using a reference signal as well. It did not exhibit a clear advantage over obtaining coarse alignment by finding the maximum of the cross-correlation between the signals. One can also exhaustively search a certain range of possible offsets, choosing the one that results in a reconstruction of a sensible audio signal.

Since this only yields alignment on the order of a sampling period of a single gyroscope (T), we still need to find the more precise time-skews in the range $[0, T]$. We can scan a range of possible time-skews, choosing the one that yields a sensible audio signal. We can think of an automated evaluation of the result by a speech recognition engine or scoring according to features that would indicate human speech, suggesting a successful reconstruction.

This scanning is obviously time consuming. If we have n sources, we set one of the time skews (arbitrary) to 0, and have $n - 1$ degrees of freedom to play with, and the complexity grows exponentially with the number of sources. Nevertheless, in an attack scenario, it is not impossible to manually scan all possibilities looking for the best signal reconstruction, provided the information is valuable to the eavesdropper.

¹¹Each device samples with a period of 5 ms, therefore even 1 ms clock accuracy would be quite coarse.

¹²While an attacker may be able to play using one of the phones' speakers a known tone/chirp (no special permissions are needed), it is unlikely to be loud enough to be picked up well by the other device, and definitely depends on many factors such as distance, position etc.

Signal reconstruction from non-uniform samples

Assuming we have compensated for offset, gain mismatch and found the precise time-skews between the sampling devices, we are dealing with the problem of signal reconstruction from periodic, non-uniform samples. A seminal paper on the subject is [74] by Eldar et al. Among other works in the field are [133, 148] and [97]. Sindhi et al. [145] propose a discrete time implementation of [74] using digital filterbanks. The general goal is, given samples on a non-uniform periodic grid, to obtain estimation of the values on a uniform sampling grid, as close as possible to the original signal.

A theoretic feasibility justification lies in Papoulis' Generalized Sampling theorem [130]. Its corollary is that a signal bandlimited to π/T_Q can be recovered from the samples of N filters with sampling periods $T = NT_Q$.¹³ We suggest using one of the proposed methods for signal reconstruction from periodic non-uniform samples. With only several devices the reconstructed speech will still be narrow-band. While it won't necessarily be easily understandable by a human listener, it could be used for better automated identification. Applying narrowband to wideband speech extension algorithms [123] might provide audio signals understandable to a human listener.

We suggest using one of the methods for signal reconstruction from periodic non-uniform samples mentioned above. With only several devices the reconstructed speech will still be narrow-band. For example, using readings from two devices operating at 200 Hz and given their relative time-skew we obtain an effective sampling rate of 400 Hz. For four devices we obtain a sampling rate of 800 Hz, and so on. While a signal reconstructed using two devices still won't be easily understandable by a human listener, it could be used to improve automatic identification.

We used [74] as a basis for our reconstruction algorithm. The discussion of *re-current non-uniform sampling* directly pertains to our task. It proposes a filterbank scheme to interpolate the samples such that an approximation of the values on the

¹³It is important to note that in our case the signal is not necessarily bandlimited as required. While the base pitch of the speech can lie in the range $[0, 200 \cdot N]$, it can contain higher frequencies that are captured in the recording due to aliasing, and may interfere with the reconstruction. It depends mainly on the low-pass filtering applied by the gyroscope. In InvenSense's MPU-6050, Digital Low-Pass Filtering (DLPF) is configurable through hardware registers [17], so the conditions depend to some extent on the particular driver implementation.

uniform grid is obtained. The derivation of the discrete-time interpolation filters is provided in Appendix A.1.

This method allows us to perform reconstruction with arbitrary time-skews; however we do not have at the time a good method for either a very precise estimation of the time-skews or automatic evaluation of the reconstruction outcome (which would enable searching over a range of possible values). For our experiment we applied this method to the same set of samples used for speaker-dependent speech recognition evaluation, which was recorded simultaneously by two devices. We used the same value for τ , the time-skew for all samples, and therefore chose the expected value $\tau = T/2$ which is equivalent to the particular case of sampling on a uniform grid (resulting in all-pass interpolation filters). It is essentially the same as interleaving the samples from the two readings, and we ended up implementing this trivial method as well, in order to avoid the adverse effects of applying finite non-ideal filters.

It is important to note that while we propose a method rooted in signal processing theory, we cannot confidently attribute the improved performance to obtaining a signal that better resembles the original, until we take full advantage of the method by estimating the precise time-skew for each recording, and applying true non-uniform reconstruction. It is currently left as an interesting future improvement, for which the outlined method can serve as a starting point. In this sense, our actual experiment can be seen as taking advantage of better feature vectors, comprised of data from multiple sources.

Evaluation

We evaluated this approach by repeating the speaker-dependent word recognition experiment on signals reconstructed from readings of two Nexus 4 devices. Table 3.7 summarizes the final results obtained using the sample interleaving method¹⁴.

There was a consistent noticeable improvement compared to the results obtained using readings from a single device, which supports the value of utilizing multiple

¹⁴We also compared the performance of the DTW classifier on samples reconstructed using the filterbank approach. It yielded a slightly lower correct classification rate of 75% which we attribute to the mentioned effects of applying non-ideal finite filters.

SVM	GMM	DTW
18%	14%	77%

Table 3.7: Evaluation of the method of reconstruction from multiple devices. Results obtained via "leave-one-out" cross-validation on 44 recorded words pronounced by a single speaker. Recorded using a Nexus 4 device.

gyroscopes. We can expect that adding more devices to the setup would further improve the speech recognition.

3.4 Further Attacks

In this section we suggest directions for further exploitation of the gyroscopes:

Increasing the gyro’s sampling rate. One possible attack is related to the hardware characteristics of the gyro devices. The hardware upper bound on sampling frequency is higher than that imposed by the operating system or by applications¹⁵. InvenSense MPU-6000/MPU-6050 gyroscopes can provide a sampling rate of up to 8000 Hz. That is the equivalent of a POTS (telephony) line. STMicroelectronics gyroscopes only allow up to 800 Hz sampling rate, which is still considerably higher than the 200 Hz allowed by the operating system (see Appendix A.3). If the attacker can gain a one-time privileged access to the device, she could patch an application, or a kernel driver, thus increasing this upper bound. The next steps of the attack are similar: obtaining gyroscope measurements using an application or tricking the user into leaving the browser open on some website. Obtaining such a high sampling rate would enable using the gyroscope as a microphone in the full sense of hearing the surrounding sounds.

Source separation. Based on experimental results presented in Section 3.1.3 it is obvious that the gyroscope measurements are sensitive to the relative direction from which the acoustic signal arrives. This may give rise to the possibility to detect the

¹⁵As we have shown, the sampling rate available on certain browsers is much lower than the maximum sampling rate enabled by the OS. However, this is an application level constraint.

angle of arrival (AoA) at which the audio signal hits the phone. Using AoA detection one may be able to better separate and process multiple sources of audio, e.g. multiple speakers near the phone.

Ambient sound recognition. There are works (e.g. [139]) which aim to identify a user’s context and whereabouts based on the ambient noise detected by his smart phone, e.g restaurant, street, office, and so on. Some contexts are loud enough and may have distinct fingerprint in the low frequency range to be able to detect them using a gyroscope, for example railway station, shopping mall, highway, and bus. This may allow an attacker to leak more information on the victim user by gaining indications of the user’s whereabouts.

3.5 Defenses

Let us discuss some ways to mitigate the potential risks. As it is often the case, a secure design would require an overall consideration of the whole system and a clear definition of the power of the attacker against whom we defend. To defend against an attacker that has only user-level access to the device (an application or a website), it might be enough to apply low-pass filtering to the raw samples provided by the gyroscope. Judging by the sampling rate available for Blink and WebKit based browsers, it is enough to pass frequencies in the range 0 – 20 Hz. If this rate is enough for most of the applications, the filtering can be done by the driver or the OS, subverting any attempt to eavesdrop on higher frequencies that reveal information about surrounding sounds. In case a certain application requires an unusually high sampling rate, it should appear in the list of permissions requested by that application, or require an explicit authorization by the user. To defend against attackers who gain root access, this kind of filtering should be performed at the hardware level, not being subject to configuration. Of course, it imposes a restriction on the sample rate available to applications.

Another possible solution is some kind of acoustic masking. It can be applied around the sensor only, or possibly on the case of the mobile device.

3.6 Gyrophone Attack Conclusion

We showed that the acoustic signal measured by the gyroscope can reveal private information about the phone’s environment such as who is speaking in the room and, to some extent, what is being said. We use signal processing and machine learning to analyze speech from very low frequency samples. With further work on low-frequency signal processing of this type it should be possible to further increase the quality of the information extracted from the gyro.

This attack demonstrates an unexpected threat resulting from the unmitigated access to the gyro: applications and active web content running on the phone can eavesdrop sound signals, including speech, in the vicinity of the phone. We described several mitigation strategies. Some are backwards compatible for all but a very small number of applications and can be adopted by mobile hardware vendors to block this threat.

A general conclusion we suggest following this work is that access to all sensors should be controlled by the permissions framework, possibly differentiating between low and high sampling rates.

Chapter 4

PowerSpy: Location Tracking using Mobile Device Power Analysis

Our phones are always within reach and their location is mostly the same as our own. In effect, tracking the location of a phone is practically the same as tracking the location of its owner. Since users generally prefer that their location not be tracked by arbitrary third parties, all mobile platforms consider the device’s location as sensitive information and go to considerable lengths to protect it: applications need explicit user permission to access the phone’s GPS and even reading coarse location data based on cellular and WiFi connectivity requires explicit user permission.

In this work we show that despite these restrictions applications can covertly learn the phone’s location. They can do so using a seemingly benign sensor: the phone’s power meter that measures the phone’s power consumption over a period of time. Our work is based on the observation that the phone’s location significantly affects the power consumed by the phone’s cellular radio. The power consumption is affected both by the distance to the cellular base station to which the phone is currently attached (free-space path loss) and by obstacles, such as buildings and trees, between them (shadowing). The closer the phone is to the base station and the fewer obstacles between them the less power the phone consumes. The strength of the cellular signal is a major factor affecting the power used by the cellular radio [142]. Moreover, the cellular radio is one of the most dominant power consumers on the phone [92].

Suppose an attacker measures in advance the power profile consumed by a phone as it moves along a set of known routes or in a predetermined area such as a city. We show that this enables the attacker to infer the target phone’s location over those routes or areas by simply analyzing the target phone’s power consumption over a period of time. This can be done with no knowledge of the base stations to which the phone is attached.

A major technical challenge is that power is consumed simultaneously by many components and applications on the phone in addition to the cellular radio. A user may launch applications, listen to music, turn the screen on and off, receive a phone call, and so on. All these activities affect the phone’s power consumption and result in a very noisy approximation of the cellular radio’s power usage. Moreover, the cellular radio’s power consumption itself depends on the phone’s activity, as well as the distance to the base-station: during a voice call or data transmission the cellular radio consumes more power than when it is idle. All of these factors contribute to the phone’s power consumption variability and add noise to the attacker’s view: the power meter only provides aggregate power usage and cannot be used to measure the power used by an individual component such as the cellular radio.

Nevertheless, using machine learning, we show that the phone’s aggregate power consumption over time completely reveals the phone’s location and movement. Intuitively, the reason why all this noise does not mislead our algorithms is that the noise is not correlated with the phone’s location. Therefore, a sufficiently long power measurement (several minutes) enables the learning algorithm to “see” through the noise. We refer to power consumption measurements as time-series and use methods for comparing time-series to obtain classification and pattern matching algorithms for power consumption profiles.

In this work we use machine learning to identify the routes taken by the victim based on previously collected power consumption data. We study three types of user tracking goals:

1. **Route distinguishability:** First, we ask whether an attacker can tell what route the user is taking among a fixed set of possible routes.

2. **Real-time motion tracking:** Assuming the user is taking a certain known route, we ask whether an attacker can identify her location along the route and track the device's position on the route in real-time.
3. **New route inference:** Finally, suppose a user is moving along an arbitrary (long) route. We ask if an attacker can learn the user's route using the previously measured power profile of many (short) road segments in that area. The attacker composes the power profile of the short road segments to identify the user's route and location at the end of the route.

We emphasize that our approach is based on measuring the phone's aggregate power consumption and nothing else. In particular, we do not use the phone's signal strength as this data is protected on Android and iOS devices and reading it requires user permission. In contrast, reading the phone's power meter requires no special permissions.

On Android reading the phone's aggregate power meter is done by repeatedly reading the following two files:

```
/sys/class/power_supply/battery/voltage_now  
/sys/class/power_supply/battery/current_now
```

Over a hundred applications in the Play Store access these files. While most of these simply monitor battery usage, our work shows that all of them can also easily track the user's location.

This chapter makes the following contributions:

- We show that the power meter available on modern smartphones can reveal potentially private information.
- We develop the machine learning techniques needed to use data collected from the power meter to infer location information. The technical details of our algorithms are presented in sections 4.3, 4.4 and 4.5, followed by experimental results.
- In sections 5.4 and 4.8 we discuss potential continuation to this work, as well as defenses to prevent this type of information leakage.

4.1 Threat Models

We assume a malicious application is installed on the victim’s device and runs in the background. The application has no permission to access the GPS or any other location data such as the cellular or WiFi components. In particular, the application has no permission to query the identity of visible cellular base stations or the SSID of visible WiFi networks.

We only assume access to power data (which requires no special permissions on Android) and permission to communicate with a remote server. Network connectivity is needed to generate dummy low rate traffic to prevent the cellular radio from going into low power state. In our setup we also use network connectivity to send data to a central server for processing. However, it may be possible to do all processing on the phone.¹

As noted earlier, the application can only read the *aggregate* power consumed by the phone. It cannot measure the power consumed by the cellular radio alone. This presents a significant challenge since many components on the phone consume variable amounts of power at any given time. Consequently, all the measurements are extremely noisy and we need a way to “see” through the noise.

To locate the phone, we assume the attacker has prior knowledge of the area or routes through which the victim is traveling. This knowledge allows the attacker to measure the power consumption profile of different routes in that area in advance. Our system correlates this data with the phone’s measured power usage and we show that, despite the noisy measurements, we are able to correctly locate the phone. Alternatively, as for many other machine learning cases, the training data can also be collected after obtaining the unlabeled query data. For instance, an attacker obtained a power consumption profile of a user, the past location of whom it is extremely important to determine. She can still collect, after the fact, reference profiles for a limited area in which the user has likely been driving and carry out the attack.

For this to work we need the tracked phone to be moving by a car or a bus while

¹It is important to mention here that while a network access permission will appear in the permission list for an installed application, it does not currently appear in the list of required permissions prior to application installation.

being tracked. Our system cannot locate a phone that is standing still since that only provides the power profile for a single location. We need multiple adjacent locations for the attack to work.

Given the resources at our disposal, the focus of this work is on locating a phone among a set of local routes in a pre-determined area. A larger effort is needed to scale the system to cover the entire world by pre-measuring the power profile of all road segments worldwide. Nevertheless, our localized experiments already show that tracking users who follow a daily routine is quite possible. For example, a mobile device owner might choose one of a small number of routes to get from home to work. The system correctly identifies what route was chosen and in real-time identifies where the phone is along that route. This already serves as a cautionary note about the type of information that can be leaked by a seemingly innocuous sensor like the power meter.

We note that scaling the system to cover worldwide road segments can be done by crowd-sourcing: a popular app, or perhaps even the core OS, can record the power profile of streets traveled by different users and report the results to a central server. Over time the resulting dataset will cover a significant fraction of the world. On the positive side, our work shows that service providers can legitimately use this dataset to improve the accuracy of location services. On the negative side, tracking apps can use it to covertly locate users. Given that all that is required is one widespread application, many actors in the mobile space are in a position to build the required dataset of power profiles and use it as they will.

4.2 Background

In this section we provide technical background on the relation between a phone's location and its cellular power consumption. We start with a description of how location is related to signal strength, then we describe how signal strength is related to power consumption. Finally, we present examples of this phenomenon, and we demonstrate how obtaining access to power measurements could leak information about a phone's location.

4.2.1 Location affects signal strength and power consumption

Distance to the base station is the primary factor that determines a phone's signal strength. The reason for this is, for signals propagating in free space, the signal's power loss is proportional to the square of the distance it travels over [80]. Signal strength is not only determined by path loss, it is also affected by objects in the signal path, such as trees and buildings, that attenuate the signal. Finally, signal strength also depends on multi-path interference caused by objects that reflect the radio signal back to the phone through various paths having different lengths.

In wireless communication theory signal strength is often modeled as random variation (e.g., log-normal shadowing [80]) to simulate many different environments². However, in one location signal strength can be fairly consistent as base stations, attenuators, and reflectors are mostly stationary.

A phone's received signal strength to its base station affects its cellular modem power consumption. Namely, phone cellular modems consume less instantaneous power when transmitting and receiving at high signal strength compared to low signal strength. Schulman et. al. [142] observed this phenomenon on several different cellular devices operating on different cellular protocols. They showed that communication at a poor signal location can result in a device power draw that is 50% higher than at a good signal location.

The primary reason for this phenomenon is the phone's power amplifier used for transmission which increases its gain as signal strength drops [80]. This effect also occurs when a phone is only receiving packets. The reason for this is cellular protocols which require constant transmission of channel quality and acknowledgments to base stations.

4.2.2 Power usage can reveal location

The following results from driving experiments demonstrate the potential of leaking location from power measurements.

²Parameters of the model can be calibrated to better match a specific environment of interest.

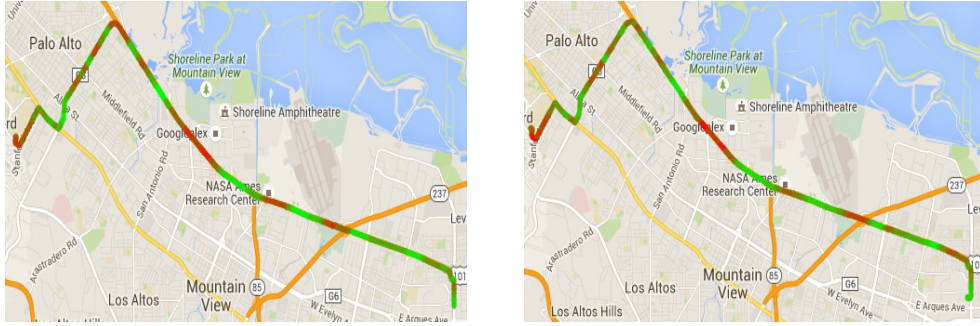


Figure 4.1: Signal strength profiles measured on two different days are stable.

We first demonstrate that signal strength in each location on a drive can be static over the course of several days. We collected signal strength measurements from a smartphone once, and again several days later. In Figure 4.1 we plot the signal strength observed on these two drives. In this figure it is apparent that (1) the segments of the drive where signal strength is high (green) and low (red) are in the same locations across both days, and (2) that the progression of signal strength along the drive appears to be a unique irregular pattern.

Next, we demonstrate that just like signal strength, power measurements of a smartphone, while it communicates, can reveal a stable, unique pattern for a particular drive. Unlike signal strength, power measurements are less likely to be stable across drives because power depends on how the cellular modem reacts to changing signal strength: a small difference in signal strength between two drives may put the cellular modem in a mode that has a large difference in power consumption. For example, a small difference in signal strength may cause a phone to hand-off to a different cellular base station and stay attached to it for some time (Section 4.2.3).

Figure 4.2 shows power measurements for two Nexus 4 phones in the same vehicle, transmitting packets over their cellular link, while driving on the same path. The power consumption variations of the Nexus 4 phones are similar, indicating that power measurements can be mostly stable across devices.

Finally, we demonstrate that power measurements could be stable across different models of smartphones. This stability would allow an attacker to obtain a reference power measurement for a drive without using the same phone as the victim's. We

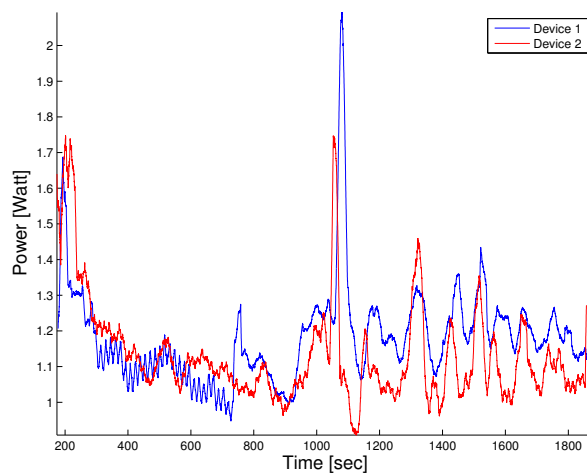


Figure 4.2: For two phones of the same model, power variations on the same drive are similar.

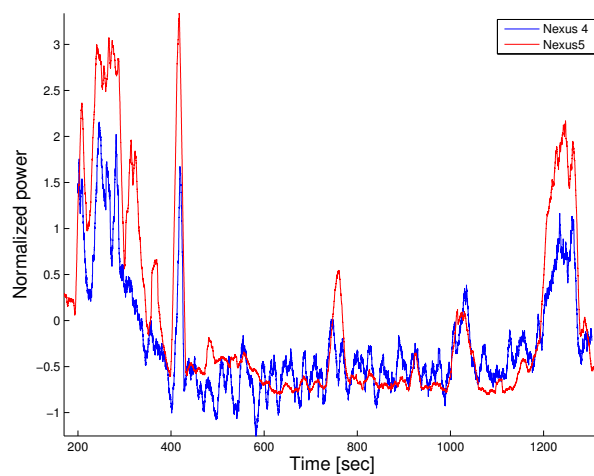


Figure 4.3: For two different phone models, power variations on the same drive are similar.

recorded power measurements, while transmitting packets over cellular, using two different smartphone models (Nexus 4 and Nexus 5) during the same ride, and we aligned the power samples, according to absolute time.

The results presented in Figure 4.3 indicate that there is similarity between different models that could allow one model to be used as a reference for another. This experiment serves as a proof of concept: we leave further evaluation of such an attack scenario, where the attacker and victim use different phone models, to future work. Here we assume that the attacker can obtain reference power measurements using the same phone model as the victim.

4.2.3 Hysteresis

A phone attaches to the base station having the strongest signal. Therefore, one might expect that the base station to which a phone is attached and the signal strength will be the same in one location. Nonetheless, it is shown in [142] that signal strength can be significantly different at a location based on how the device arrived there, for example, the direction of arrival. This is due to the hysteresis algorithm used to decide when to hand-off to a new base station. A phone hands-off from its base station only when its received signal strength dips below the signal strength from the next base station by more than a given threshold [131]. Thus, two phones that reside in the same location can be attached to two different base stations.

Hysteresis has two implications for determining a victim's location from power measurements: (1) an attacker can only use the same direction of travel as a reference power measurement, and (2) it will complicate inferring new routes from power measurements collected from individual road segments (Section 4.5).

4.2.4 Background summary and challenges

The initial measurements in this section suggest that the power consumed by the cellular radio is a side channel that leaks information about the location of a smartphone. However, there are four significant challenges that must be overcome to infer

location from the power meter. First, during the pre-measurement phase the attacker may have traveled at a different speed and encountered different stops than the target phone. Second, the attacker will have to identify the target’s power profile from among many pre-collected power profiles along different routes. Third, once the attacker determines the target’s path, the exact location of the target on the path may be ambiguous because of similarities in the path’s power profile. Finally, the target may travel along a path that the attacker only partially covered during the pre-measurement phase: the attacker may have only pre-collected measurements for a subset of segments in the target’s route. In the following sections we describe techniques that address each of these challenges and experiment with their accuracy.

4.3 Route distinguishability

As a warm-up we show how the phone’s power profile can be used to identify what route the user is taking from among a small set of possible routes (say, 30 routes). Although we view it as a warm-up, building towards our main results, route distinguishability is still quite useful. For example, if the attacker is familiar with the user’s routine then the attacker can pre-measure all the user’s normal routes and then repeatedly locate the user among those routes.

Route distinguishability is a classification problem: we collected power profiles associated with known routes and want to classify new samples based on this training set. We treat each power profile as a time series which needs to be compared to other time series. A score is assigned after each comparison, and based on these scores we select the most likely matching route. Because different rides along the same route can vary in speed at different locations along the ride, and because routes having the same label can vary slightly at certain points (especially before getting to a highway and after exiting it), we need to compare profile features that can vary in time and length and allow for a certain amount of difference. We also have to compensate for different baselines in power consumption due to constant components that depend on the running applications and on differences in device models.

We use a classification method based on Dynamic Time Warping (DTW) [127], an

algorithm for measuring similarity between temporal sequences that are misaligned and vary in time or speed. We compute the DTW distance³ between the new power profile and all reference profiles associated with known routes, selecting the known route that yields the minimal distance. More formally, if the reference profiles are given by sequences $\{X\}_{i=1}^n$, and the unclassified profile is given by sequence Y , we choose the route i such that

$$i = \underset{i}{\operatorname{argmin}} \operatorname{DTW}(Y, X_i)$$

which is equivalent to 1-NN classification given DTW metric.

Because the profiles might have different baselines and variability, we perform the following normalization for each profile prior to computing the DTW distance: we calculate the mean and subtract it, and divide the result by the standard deviation. We also apply some preprocessing in the form of smoothing the profiles using a moving average (MA) filter in order to reduce noise and obtain the general power consumption trend, and we downsample by a factor of 10 to reduce computational complexity.

4.4 Real-time mobile device tracking

In this section we consider the following task: the attacker knows that a mobile user is traveling along a particular route and our objective is to track the mobile device as it is moving along the route. We do not assume a particular starting point along the route, meaning, in probabilistic terms, that our prior on the initial location is uniform. The attacker has reference power profiles collected in advance for the target route, and constantly receives new power measurements from an application installed on the target phone. Its goal is to locate the device along the route, and continue tracking it in real-time as it travels along the route.

³In fact we compute a normalized DTW distance, as we have to compensate for difference in lengths of different routes - a longer route might yield larger DTW distance despite being more similar to the tested sequence.

4.4.1 Tracking via Dynamic Time Warping

This approach is similar to that of route distinguishability, but we use only the measurements collected up to this point, which comprise a sub-sequence of the entire route profile. We use the *Subsequence* DTW algorithm [127], rather than the classic DTW, to search a sub-sequence in a larger sequence, and return a distance measure as well as the corresponding start and end offsets.

We search for the sequence of measurements we have accumulated since the beginning of the drive in all our reference profiles and select the profile that yields the minimal DTW distance. The location estimate corresponds to the location associated with the end offset returned by the algorithm.

4.4.2 Improved tracking via a motion model

While the previous approach can make mistakes in location estimation due to a match with an incorrect location, we can further improve the estimation by imposing rules based on a sensible motion model. We first need to know when we are “locked” on the target. For this purpose we define a similarity threshold so that if the minimal DTW distance is above this threshold, we are in a *locked* state. Once we are locked on the target, we perform a simple sanity check at each iteration: “Has the target displaced by more than X?”

If the sanity check does not pass we consider the estimate unlikely to be accurate, and simply output the previous estimate as the new estimated location. If the similarity is below the threshold, we switch to an *unlocked* state, and stop performing this sanity check until we are “locked” again. Algorithm 1 presents this logic as pseudocode.

4.4.3 Tracking using Optimal Subsequence Bijection

Optimal Subsequence Bijection (OSB) [107] is a technique, similar to DTW, that enables aligning two sequences. In DTW, we align the query sequence with the target sequence without skipping elements in the query sequence, thereby assuming that the query sequence contains no noise. OSB, on the other hand, copes with noise in both

Algorithm 1 Improved tracking using a simple motion model

```

locked  $\leftarrow$  false ▷ Are we locked on the target?
while target moving do
  loc[i], score  $\leftarrow$  estimateLocation()
  d  $\leftarrow$  getDistance(loc[i], loc[i − 1])
  if locked and d > MAX_DISP then
    loc[i]  $\leftarrow$  loc[i − 1] ▷ Reuse previous estimate
  end if
  if score > THRESHOLD then
    locked  $\leftarrow$  true
  end if
end while

```

sequences by allowing to skip elements. A fixed jump-cost is incurred with every skip in either the query or the target sequence. This extra degree of freedom has potential for aligning noisy subsequences more efficiently in our case. In the evaluation section we present results obtained by using OSB and compare them to those obtained using DTW.

4.5 Inference of new routes

In Section 4.3 we addressed the problem of identifying the route traversed by the phone, assuming the potential routes are known in advance. This assumption allowed us to train our algorithm specifically for the potential routes. As previously mentioned, there are indeed many real-world scenarios where it is applicable. Nevertheless, in this section we set out to tackle a broader tracking problem, where the future potential routes are not explicitly known. Here we specifically aim to identify the final location of the phone after it traversed an unknown route. We assume that the area in which the mobile device owner moves is known, however the number of all possible routes in that area may be too large to practically pre-record each one. Such an area can be, for instance, a university campus, a neighborhood, a small town or a highway network.

We address this problem by pre-recording the power profiles of all the road segments within the given area. Each possible route a mobile device may take is a concatenation of some subset of these road segments. Given a power profile of the tracked device, we will reconstruct the unknown route using the reference power profiles corresponding to the road segments. The reconstructed route will enable us to estimate the phone's final location. Note that, due to the hysteresis of hand-offs between cellular base stations, a power consumption is not only dependent on the traveled road segment, but also on the previous road segment the device came from.

In Appendix B.1 we formalize this problem as a hidden Markov model (HMM) [134]. In the following we describe a method to solve the problem using a particle filter. The performance of the algorithm will be examined in the next section.

4.5.1 Particle Filter

A particle filter [39] is a method that estimates the state of a HMM at each step based on observations up to that step. The estimation is done using a Monte Carlo approximation where a set of samples (particles) is generated at each step that approximate the probability distribution of the states at the corresponding steps. A comprehensive introduction to particle filters and their relation to general state-space models is provided in [138].

We implement the particle filter as follows. We denote $O^r = \{o_{xyz}^r\}$, where o_{xyz}^r is a power profile prerecorded over segment (y, z) while the segment (x, y) had been traversed just before it. We use a discrete time resolution $\tau = 3$ seconds. We denote $\Delta_{\min}^{yz}$ and $\Delta_{\max}^{yz}$ to be the minimum and maximum time duration to traverse road segment (y, z) , respectively. We assume these bounds can be derived from prerecordings of the segments. At each iteration i we have a sample set of N routes $P_i = \{(Q, T)\}$. The initial set of routes P_0 are chosen according to Π . At each step, we execute the following algorithm:

At each iteration, we append a new segment, chosen according to the prior A , to each possible route (represented by a particle). Then, the traversal time of the new segment is chosen so that it will have a minimal DTW distance to the respective time

Algorithm 2 Particle filter for new routes estimation

```

for all route  $p$  in  $P$  do
   $t_{\text{end}} \leftarrow$  end time of  $p$ 
   $(x, y) \leftarrow$  last segment of  $p$ 
   $z \leftarrow$  next intersection to traverse (distributed by  $A$ )
  
$$W_p \leftarrow \min_{\substack{t \in [\Delta_{\min}^{yz}, \Delta_{\max}^{yz}] \\ o_{xyz}^r \in O_{xyz}^r}} \{ \text{DTW}(O_{[t_{\text{end}}, t_{\text{end}}+t]}, o_{xyz}^r) \}$$

   $p \leftarrow p || (y, z)$ 
  Update the end time of  $p$ 
end for
Resample  $P$  according to the weights  $W_p$ 

```

interval of the tracked power profile. We take this minimal distance as the weight of the new route. After normalizing the weights of all routes, a resampling phase takes place. N routes are chosen from the existing set of routes according to the particle weights distribution⁴. The new resampled set of routes is the input to the next iteration of the particle filter. The total number of iterations should not exceed an upper bound on the number of segments that the tracked device can traverse. Note however that a route may exhaust the examined power profile before the last iteration (namely, the end time of that route reached t_{max}). In such a case we do not update the route in all subsequent iterations (this case is not described in Algorithm 2 to facilitate fluency of exposition).

Before calculating the DTW distance of a pair of power profiles the profiles are preprocessed to remove as much noise as possible. We first normalize the power profile by subtracting its mean and dividing by the standard deviation of all values included in that profile. Then, we zero out all power values below a threshold percentile. This last step allows us to focus only on the peaks in power consumption where the radio's power consumption is dominant while ignoring the lower power values for which the radio's power has a lesser effect. The percentile threshold we use in this work is 90%.

Upon its completion, the particle filter outputs a set of N routes of various lengths. To select the best estimate route the simple approach is to choose the route that

⁴Note that the resampling of the new routes can have repetitions. Namely, the same route can be chosen more than one time

appears the most number of times in the output set as it has the highest probability to occur. Nonetheless, since a route is composed of multiple segments chosen at separate steps, at each step the weight of a route is determined solely based on the last segment added to the route. Therefore, the output route set is biased in favor of routes ending with segments that were given higher weights, while the weights of the initial segments have a diminishing effect on the route distribution with every new iteration. To counter this bias, we choose another estimate route using a procedure we call *iterative majority vote*, described in Appendix B.2.

4.6 Experiments

4.6.1 Data collection

Our experiments required collecting real power consumption data from smartphone devices along different routes. We developed the PowerSpy android application⁵ that collects various measurements including signal strength, voltage, current, GPS coordinates, temperature, state of discharge (battery level) and cell identifier. The recordings were performed using Nexus 4, Nexus 5 and HTC mobile devices.

4.6.2 Assumptions and limitations

Exploring the limits of our attack, i.e. establishing the minimal necessary conditions for it to work, is beyond our resources. For this reason, we state the assumptions on which we rely in our methods.

We assume there is enough variability in power consumption along a route to exhibit unique features. Lack of variability may be due to high density of cellular antennas that flatten the signal strength profile. We also assume that enough communication is occurring for the signal strength to have an effect on power consumption. This is a reasonable assumption, since background synchronization of data happens frequently in smartphone devices. Moreover, the driver might be using navigation

⁵Source code can be obtained from <https://bitbucket.org/ymcrat/powerspy>.

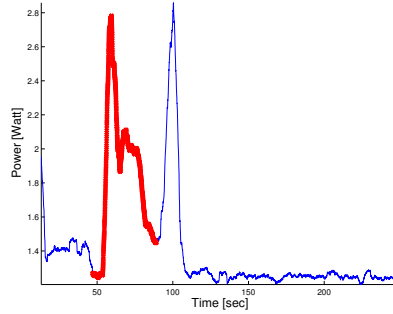


Figure 4.4: Power profile with a phone call occurring between 50-90 seconds. Profile region during phone call is marked in red.

software or streaming music. However, at this stage, it is difficult to determine how inconsistent phone usage across different rides will affect our attacks.

Identifying which route the user took involves understanding which power measurements collected from her mobile device occurred during driving activity. Here we simply assume that we can identify driving activity. Other works (e.g., [125]) address this question by using data from other sensors that require no permissions, such as gyroscopes and accelerometers.

Some events that occur while driving, such as an incoming phone call, can have a significant effect on power consumption. Figure 4.4 shows the power profile of a device at rest when a phone call takes place (the part marked in red). The peak immediately after the phone call is caused by using the phone to terminate the phone call and turn off the display. We can see that this event appears prominently in the power profile and can cope with such transient effects by identifying and truncating peaks that stand out in the profile. In addition, smoothing the profile by a moving average should mitigate these transient effects.

4.6.3 Route distinguishability

To evaluate the algorithm for distinguishing routes (section 4.3) we recorded reference profiles for multiple different routes. The profiles include measurements from both Nexus 4 and Nexus 5 models. In total we had a dataset of 294 profiles, representing 36

# Unique Routes	# Ref. Profiles/Route	# Test Routes	Correct Identification %	Random Guess %
8	10	55	85	13
17	5	119	71	6
17	4	136	68	6
21	3	157	61	5
25	2	182	53	4
29	1	211	40	3

Table 4.1: Route distinguishability evaluation results. First column indicates the number of unique routes in the training set. Second column indicates the number of training samples per route at the attacker’s disposal. Number of test routes indicates the number of power profiles the attacker is trying to classify. Correct identification percentage indicates the percentage of correctly identified routes as a fraction of the third column (test set size), which could be then compared to the expected success of random guessing in the last column.

unique routes. Driving in different directions along the same roads (from point A to B vs. from point B to A) is considered two different routes. We perform cross validation using multiple iterations (100 iterations), each time using a random portion of the profiles as a training set, and requiring equal number of samples for each possible class. The sizes of the training and test sets depend on how many reference routes per profile we require each time. Naturally, the more reference profiles we have, the higher the identification rate.

One evaluation round included 29 unique routes, with only 1 reference profile per route in the training set, and 211 test routes. It resulted in correct identification rate of 40%. That is compared to the random guess probability of only 3%. Another round included 25 unique routes, with 2 reference profiles per route in the training set and 182 routes in the test set, and resulted in correct identification rate of 53% (compared to the random guess probability of only 4%). Having 5 reference profiles per route (for 17 unique routes) raises the identification rate to 71%, compared to the random guess probability of 5.8%. And finally, for 8 reference profiles per route we get 85% correct identification. The results are summarized in table 4.1.

We can see that an attacker can have a significant advantage in guessing the route taken by a user.

4.6.4 Real-time mobile device tracking

We evaluate the algorithm for real-time mobile device tracking (section 4.4) using a set of 10 training profiles and an additional test profile. The evaluation simulates the conditions of real-time tracking by serially feeding samples to the algorithm as if they are received from an application installed on the device. We calculate the estimation error, i.e. the distance between the estimated coordinates and the true location of the mobile device at each step of the simulation. We are interested in the *convergence time*, i.e. the number of samples it takes until the location estimate is close enough to the true location, as well as in the distribution of the estimation errors given by a histogram of the absolute values of the distances.

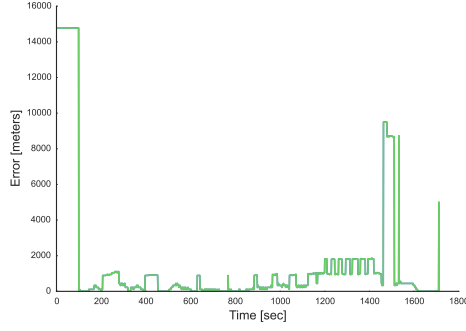
Figure 4.5 illustrates the performance of our tracking algorithm for one of the routes, which was about 19 kilometers long. At the beginning, when there are very few power samples, the location estimation is extremely inaccurate, but after two minutes we lock on the true location. We obtained a precise estimate from 2 minutes up until 20 minutes on the route, where our estimate slightly diverges, due to increased velocity on a freeway segment. Around 26 minutes (in figure 4.5(a)) we have a large estimation error, but as we mentioned earlier, these kind of errors are easy to prevent by imposing a simple motion model (section 4.4.2). Most of the errors are small compared to the length of the route: almost 90% of the estimation errors are less than 1 km.

We also tested the improved tracking algorithm explained in section 4.4.2. Figure 4.5(b) presents the estimation error over time, and we can see that the big errors towards the end of the route that appeared in 4.5(a) are not present in fig. 4.5(b). Moreover, now almost all of the estimation errors are below 3.5 km (fig. 4.6).

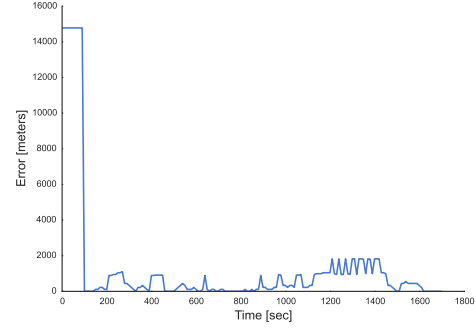
We provide animations visualizing our results for real-time tracking at the following links. The animations, generated using our estimations of the target's location, depict a moving target along the route and our estimation of its location. The first one corresponds to the method described in 4.4.1, and the second to the one described in 4.4.2 that uses the motion model based correction:

crypto.stanford.edu/powerspy/tracking1.mov

crypto.stanford.edu/powerspy/tracking2.mov

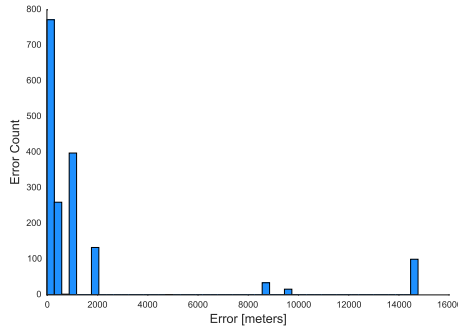


(a) Convergence to true location

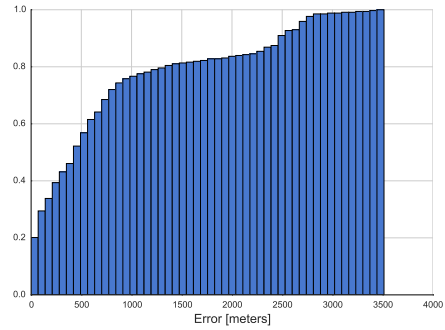


(b) Location estimation error for improved tracking algorithm

Figure 4.5: Location estimation error for online tracking.



(a) Errors histogram. Almost 80% of the errors are less than 1 km



(b) Errors CDF for motion-model tracking: almost 80% are less than 1 km, and all are below 3.5 km

Figure 4.6: Estimation errors distribution for real-time tracking.

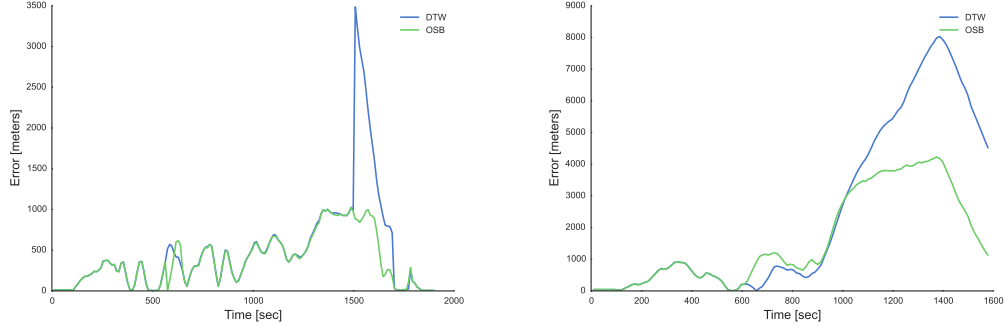


Figure 4.7: Comparison of DTW and OSB for real-time tracking.

OSB vs. DTW

We compare the performance of Dynamic Time Warping to that of Optimal Subsequence Bijection (section 4.4.3). Figure 4.7 present such a comparison for the same route, using two different recordings. The tracking was performed without compensating for errors using a motion model, to evaluate the performance of the subsequence matching algorithms as they are. We can see that, in both cases, Optimal Subsequence Bijection outperforms the standard Subsequence-DTW most of the time. Therefore, we suggest that further experimentation with OSB could potentially be beneficial for this task.

4.6.5 Inference of new routes

Setup

For the evaluation of the particle filter presented in Section 4.5 we considered an area depicted in Figure 4.8. The area has 13 intersections having 35 road segments⁶. The average length of a road segment is about 400 meters. The average travel time over the segments is around 70 seconds. The area is located in the center of Haifa, a city located in northern Israel, having a population density comparable to Philadelphia or Miami. Traffic congestion in this area varies across segments and time of day. For each power recording, the track traversed at least one congested segment. Most of

⁶Three of the segments are one way streets.

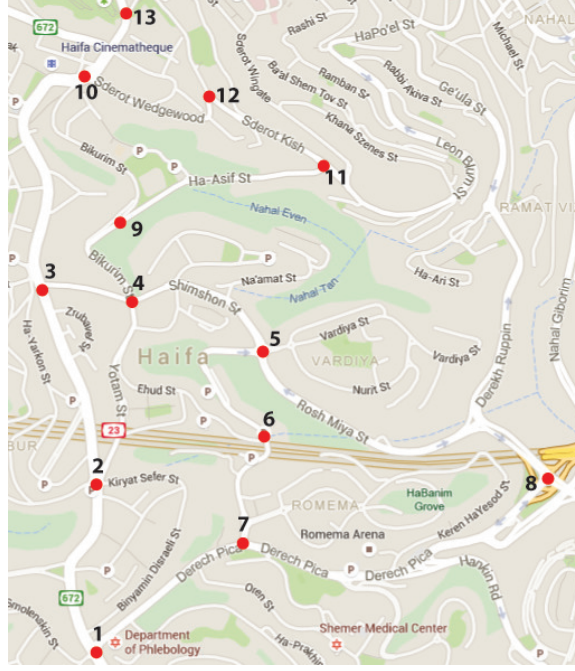


Figure 4.8: Map of area and intersections for route inference.

the 13 intersections have traffic lights, and about a quarter of the road segments pass through them.

We had three pre-recording sessions which in total covered all segments. Each road segment was entered from every possible direction to account for the hysteresis effects. The pre-recording sessions were done using the same Nexus 4 phone.

We set the following parameters of the HMM (as they are defined in Appendix B.1):

1. A – This set defines the transition probabilities between the road segments. We set these probabilities to be uniformly distributed over all possible transitions. Namely, $a_{xyz} = \{1/|I_y| \mid I_y = \{w \mid (y, w) \in R, w \neq x\}\}$.
2. B – This set defines the distribution of power profile observations over each state. These probabilities depend on the road segments and their location relative to the nearby based stations. We do not need an explicit formulation of these probabilities to employ the particle filter. The likelihood of a power profile to be associated with a road segment is estimated by the DTW distance

Phone	Track
Nexus 4 #1	8-5-6-7-1-2-3-4-5-6-4-3-2-1-7-8
Nexus 4 #2	7-1-2-3-4-5-8-7-6-5-4-2-1-7-8
Nexus 5	3-2-4-9-10-12-11-9-4-5-6-4-3-2-1-7-6-5-8-7
HTC Desire	10-12-11-9-4-2-1-7-6-5-8

Table 4.2: Test Routes

of the power profile to prerecorded power profiles of that segment.

3. Π – This set defines the initial state distribution. We assume that the starting intersection of the tracked device is known. This applies to scenarios where the tracking begins from well-known locations, such as the user’s home, office, or another location the attacker knows in advance.

For testing, we used 4 phones: two Nexus 4 (different from the one used for the pre-recordings), a Nexus 5 and an HTC Desire. Each phone was used to record the power profile of a different route. The four routes combined cover almost all of the road segments in the area. Table 4.2 details the routes by their corresponding sequences of intersection identifiers. These route recordings were done on different days, different time of day and varying weather conditions.

As noted, we can only measure the aggregate power consumption which can be significantly affected by applications that run continuously. To have a better sense of the effects of these applications the phones were run with different number of background applications. Nexus 4 #1, Nexus 5 and HTC Desire have a relatively modest number of applications which included (beyond the default Android apps): Email (corporate account), Gmail, and Google Calender. Nexus 4 #2 has a much higher number of application which included on top of the applications of phone #1: Facebook, Twitter, Skype, Waze, and WhatsApp. All those applications periodically send and receive traffic.

For each of the four tracks we derived all possible sub-tracks having 3 to 7 road segments. We estimated each such sub-track. In total we estimated around 200 sub-tracks. For each sub-track we employed Algorithms 2 and 3 to get two best estimates for the sub-track.

	random	frequent	Alg. 3	combined
Nexus 4 #1	33%	65%	48%	80%
Nexus 4 #2	31%	48%	56%	72%
Nexus 5	20%	33%	32%	55%
HTC Desire	22%	40%	41%	65%

Table 4.3: Destination localization

Tables 4.3 to 4.5 summarize the results of route estimation for each of the four phones. For each route we have two alternatives for picking an estimate (1) the most frequent route in the particle set as output by Algorithm 2; (2) the route output by Algorithm 3. For each alternative we note the road segment in which the phone is estimated to be after the completion of its track and compare it with the final road segment of the true route. This allows us to measure the accuracy of the algorithm for estimating the location of the user’s destination (the end of the track). This is the most important metric for many attack scenarios where the attacker wishes to learn the destination of the victim.

In some cases it may also be beneficial for the attacker to know the actual route through which the victim traversed on his way to the destination. For this purpose, we also calculate for each alternative estimate the Levenshtein distance between it and the true route. The Levenshtein distance is a standard metric for measuring the difference between two sequences [109]. It equals the minimum number of updates required in order to change one sequence to the next. In this context, we treat a route as a sequence of intersections. The distance is normalized by the length of the longer route of the two. This allows us to measure the accuracy of the algorithm for estimating the full track the user traversed. For each estimate we also note whether it is an exact fit with the true route (i.e., zero distance). The percentage of successful localization of destination, average Levenshtein distance and percentage of exact full route fits are calculated for each type of estimated route. We also calculate these metrics for both estimates combined while taking into account for each track the best of the two estimates. To benchmark the results we note in each table the performance of a random estimation algorithm which simply outputs a random, albeit feasible, route.

	random	frequent	Alg. 3	combined
Nexus 4 #1	0.61	0.38	0.27	0.24
Nexus 4 #2	0.63	0.61	0.59	0.52
Nexus 5	0.68	0.6	0.55	0.45
HTC Desire	0.65	0.59	0.5	0.45

Table 4.4: Levenshtein distance

The results in Table 4.3 show the accuracy of destination identification. It is evident that the performance of the most frequent route output by the particle filter is comparable to the performance of the best estimate output by Algorithm 3. However, their combined performance is significantly better than either estimates alone and predicts more accurately the final destination of the phone. This result suggests that Algorithm 3 extracts significant amount of information from the routes output by the particle filter beyond the information gleaned from the most frequent route.

Table 4.3 indicates that for Nexus 4 #1 the combined route estimates were able to identify the final road segment for 80% of all scenarios. For Nexus 4 #2 which was running many applications the final destination estimates are somewhat less accurate (72%). This is attributed to the more noisy measurements of the aggregate power consumption. The accuracy for the two models – Nexus 5 and HTC Desire – is lower than the accuracy achieved for Nexus 4. Remember that all our pre-recordings were done using a Nexus 4. These results may indicate that the power consumption profile of the cellular radio is dependent on the phone’s model. Nonetheless, for both phones we achieve significantly higher accuracy of destination localization (55% and 65%) as compared to the random case (about 20%).

Tables 4.4 and 4.5 present measures – Levenshtein distance and exact full route fit – of the accuracy of estimates for the full route the phone took to its destination. Here, again, the algorithm presented for Nexus 4 #1 superior performance. It was able to exactly estimate 45% of the full route to the destination. On the other hand, for the more busy Nexus 4 #2 and the other model phones the performance was worse. It is evident from the results that for these three phones the algorithm had difficulties producing an accurate estimate of the full route. Nonetheless, in all cases the accuracy is always markedly higher than that of the random case.

	random	frequent	Alg. 3	combined
Nexus 4 #1	4%	38%	22%	45%
Nexus 4 #2	5%	8.5%	5%	15%
Nexus 5	3%	15%	9%	20%
HTC Desire	5%	10%	12%	17%

Table 4.5: Exact full route fit

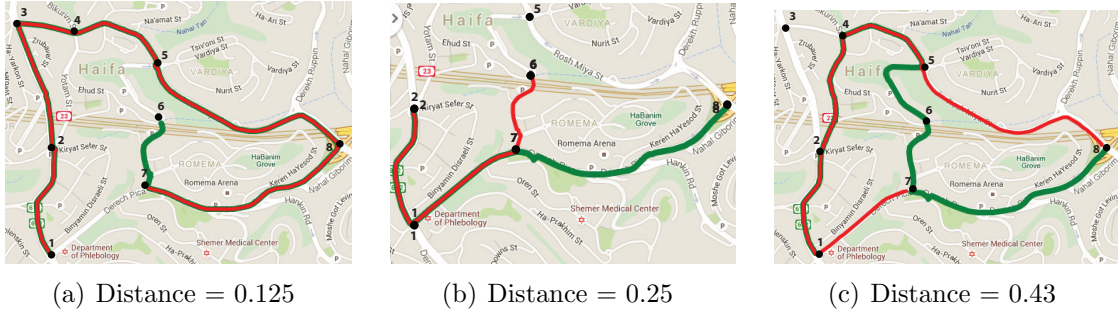


Figure 4.9: Examples of estimation errors and their corresponding distances (partial map is depicted). The true route is green and the estimated route is red.

To have a better sense of the distance metric used to evaluate the quality of the estimated routes, Figure 4.9 depicts three cases of estimation errors and their corresponding distance values in increasing order. We can see that even estimation errors having relatively high distances can have a significant amount of information regarding the true route.

4.7 Future directions

In this section we discuss ideas for further research, improvements, and additions to our method.

4.7.1 Power consumption inference

While new (yet very common) smartphone models contain an internal ampere-meter and provide access to current data, other models (for instance Galaxy S III) supply voltage but not current measurements. Therefore on these models we cannot directly

calculate the power consumption. V-edge [154] proposes using voltage dynamics to model a mobile device's power consumption. That and any other similar technique would extend our method and make it applicable to additional smartphone models.

Ref. [160] presents PowerTutor, an application that estimates power consumption by different components of the smartphone device based on voltage and state of discharge measurements. Isolating the power consumed by the cellular connectivity will improve our method by eliminating the noise introduced by other components such as audio/Bluetooth/WiFi etc. that do not directly depend on the route.

4.7.2 State of Discharge (SOD)

The time derivative of the State-of-Discharge (the battery level) is basically a very coarse indicator of power consumption. While it seemed to be too inaccurate for our purpose, there is a chance that extracting better features from it or having few possible routes may render distinguishing routes based on SOD profiles feasible. Putting it to the test is even more interesting given the HTML 5 Battery API that enables obtaining certain battery statistics from a web-page via JavaScript. Our findings demonstrate how future increases in the sampling resolution of the battery stats may turn this API even more dangerous, allowing web-based attacks.

4.7.3 Choice of reference routes

Successful classification depends among other factors on good matching between the power profile we want to classify and the reference power profiles. Optimal matching might be a matter of month, time of day, traffic on the road, and more. We can possibly improve our classification if we tag the reference profiles with those associated conditions and select reference profiles matching the current conditions when trying to distinguish a route. That of course requires collecting many more reference profiles.

4.7.4 Collecting a massive dataset

Collecting a massive dataset of power profiles associated with GPS coordinates is a feasible task given vendors' capability to legally collect analytics about users' use of their smartphones. Obtaining such big dataset will enable us to better understand how well our approach can scale and whether it can be used with much less prior knowledge about the users.

4.8 Defenses

4.8.1 Non-defenses

One might think that by adding noise or limiting the sampling rate or the resolution of the voltage and current measurements one could protect location privacy. However, our method does not rely on high sampling frequency or resolution. In fact, our method works well with profiles much coarser than what we can directly get from the raw power data, and for the route distinguishing task we actually performed smoothing and downsampling of the data yet obtained good results. Our method also works well with signal strength, which is provided with much lower resolution and sampling frequency⁷.

4.8.2 Risky combination of power data and network access

One way of reporting voltage and current measurements to the attacker is via a network connection to the attacker's server. Warning the user of this risky combination may somewhat raise the bar for this attack. There are of course other ways to leak this information. For instance, a malicious application disguised as a diagnostic software can access power data and log it to a file, without attempting to make a network connection, while another, seemingly unrelated, application reads the data from that

⁷In fact, since it reflects more directly the environmental conditions, signal strength data can provide even better route identification and tracking. We did not focus on signal strength since accessing it requires access permissions and has already drawn research attention to it as useful for localization.

file and sends it over the network.

4.8.3 Secure hardware design

The problem with access to total power consumption is that it leaks the power consumed by the transceiver circuitry and communication related tasks that indicate signal strength. While power measurements can be useful for profiling applications, in many cases, examining the power consumed by the processors executing the software logic might be enough. We therefore suggest that supplying only measurements of the power consumed by the processors (excluding the power consumed by the TX/RX chain) could be a reasonable trade-off between functionality and privacy.

4.8.4 Requiring superuser privileges

A simple yet effective prevention may be requiring superuser privileges (or being root) to access power supply data on the phone. Thus, developers and power-users can install diagnostic software or run a version of their application that collects power data on a rooted phone, whereas the release version of the software excludes this functionality. This would of course prevent the collection of anonymous performance statistics from the install-base, but as we have shown, such data can indicate much more than performance.

4.8.5 Power consumption as a coarse location indicator

Same as the cell identifier is defined as a coarse location indicator, and requires appropriate permissions to be accessed, power consumption data can also be defined as one. The user will then be aware, when installing applications that access voltage and current data, of the application's potential capabilities, and the risk potentially posed to her privacy. This defense may actually be the most consistent with the current security policies of smartphone operating systems like Android and iOS, and their current permission schemes.

4.9 Related work

Power analysis is known to be a powerful side-channel. The most well-known example is the use of high sample rate (~ 20 MHz) power traces from externally connected power monitors to recover private encryption keys from a cryptographic system [101]. Prior work has also established the relationship between signal strength and power consumption in smartphones [54, 142]. Further, Bartendr [142] demonstrated that paths of signal strength measurements are stable across several drives.

PowerSpy combines these insights on power analysis and improving smartphone energy efficiency to reveal a new privacy attack. Specifically, we demonstrate that an attacker can determine a user’s location simply by monitoring the cellular modem’s changes in power consumption with the smartphone’s alarmingly unprotected ~ 100 Hz internal power monitor.

4.9.1 Many sensors can leak location

Prior work has demonstrated that data from cellular modems can be used to localize a mobile device (an extensive overview appears in Gentile et al. [78]). Similar to PowerSpy, these works fingerprint the area of interest with pre-recorded radio maps. Others use signal strength to calculate distances to base stations at known locations. All of these methods [128, 104, 146, 129] require signal strength measurements and base station ID or WiFi network name (SSID), which is now protected on Android and iOS. Our work does not rely on the signal strength, cell ID, or SSID. PowerSpy only requires access to power measurements, which are currently unprotected on Android.

PowerSpy builds on a large body of work that has shown how a variety of unprotected sensors can leak location information. Zhou et al. [162] reveal that audio on/off status is a side-channel for location tracking without permissions. In particular, they extract a sequence of intervals where audio is on and off while driving instructions are being played by Google’s navigation application. By comparing these intervals with reference sequences, the authors were able to identify routes taken by the user. *SurroundSense* [43] demonstrates that ambient sound and light can be used for mobile phone localization. They focus on legitimate use-cases, but the same methods could

be leveraged for breaching privacy. *ACComplice* [85] demonstrates how continuous measurements from unprotected accelerometers in smartphones can reveal a user’s location. Hua et al. [90] extend *ACComplice* by showing that accelerometers can also reveal where a user is located in a metropolitan train system.

4.9.2 Other private information leaked from smartphone sensors

An emerging line of work shows that various phone sensors can leak private information other than location. In future work we will continue analyzing power measurements to determine if other private information is leaked.

Prior work has demonstrated how smartphone sensors can be used to fingerprint specific devices. *AccelPrint* [68] shows that smartphones can be fingerprinted by tracking imperfections in their accelerometer measurements. Fingerprinting of mobile devices by the characteristics of their loudspeakers is proposed in [58, 61]. Further, Bojinov et. al. [47] showed that various sensors in smartphones can be used to identify a mobile device by its unique hardware characteristics. Lukas et. al. [112] proposed a method for digital camera fingerprinting by noise patterns present in the images. [111] enhances the method enabling identification of not only the model but also particular cameras.

Sensors can also reveal a user’s input such as speech and touch gestures. Several works [41, 53, 155] have shown that the accelerometer and gyroscope can leak information about touch and swipe inputs to a foreground application.

4.10 PowerSpy Attack Conclusion

The PowerSpy attack shows that applications with access to a smartphone’s power monitor can gain information about the location of a mobile device – without accessing the GPS or any other coarse location indicators. Our approach enables known route identification, real-time tracking, and identification of a new route by only analyzing the phone’s power consumption. We evaluated PowerSpy on real-world data

collected from popular smartphones that have a significant mobile market share, and demonstrated their effectiveness. We believe that with more data, our approach can be made more accurate and reveal more information about the phone's location.

This attack is an example of the unintended consequences of giving third party applications access to sensors. It suggests that even seemingly benign sensors need to be protected by permissions, or at the very least, that more security modeling needs to be done before giving 3rd party applications access to sensors.

Chapter 5

MASHaBLE: Mobile Applications of Secret Handshakes over BLE

Recent revelations about the extent of surveillance, as well as rising awareness regarding digital privacy, have increased the demand for means to communicate privately and prevent disclosure of user data. Many mobile applications geared to this purpose have been released in the past two years, including Telegram [26], Signal Private Messenger (by Open Whisper Systems) [22], Yik-Yak [27], After School [3], and LegaTalk [15]. They aim to provide different privacy guarantees against different adversaries.

Telegram and Signal rely on cryptographic methods for establishing a shared secret key between two parties. The shared key is used for end-to-end encryption of communication between two users. While messages can be routed through a central server, no one can decipher the content of the messages. However, the origin and destination (which reveal the communicating parties) are known to the server, and by extension, to any adversary that can obtain control of it. Yik-Yak and similar apps provide anonymity, to some extent, as well as message dissemination to nearby peers. The messages are broadcast through the server to unknown users, and are therefore not end-to-end encrypted. Therefore, an adversary that has access to the server can obtain the content, and also de-anonymize the origin.

These applications have a high demand: as of July 2016, Telegram messenger for Android has between 100 to 500 million downloads on the Google Play store. Yik-Yak

has between 1 and 5 million downloads, and the After School app has hundreds of thousands of downloads. These applications' iOS versions are also popular, judging by thousands of reviews in the Apple Store. This confirms a substantial and growing interest in private/anonymous messaging, and forming anonymous communities.

Inspired by Yik-Yak and similar applications attempting to provide anonymity and location-based message propagation, we propose a mobile application that enables the creation of secret virtual communities, protects geo-location privacy and provides anonymity.

More specifically, our application enables mobile users to enroll in a community and obtain credentials by which they can present themselves to other members of the secret community. These credentials allow them to perform a mutual authentication procedure called *secret handshake* (SH), which is reminiscent in its properties to a secret handshake in the physical world.

For instance, imagine a fraternity where membership is kept secret. Its members want to identify fellow members as they walk around campus, and to communicate privately with nearby members without being exposed. In a setting where one party is willing to disclose its identity first, it could use a signed certificate to authenticate itself. Unfortunately, in our setting, when two fraternity members encounter each other, neither is willing to provide a traditional signed certificate, as it may expose them to a non-member.

One possible solution is for each member to have a mobile application that reports GPS coordinates to a server maintained by the fraternity. All communication with the server is encrypted. The server identifies two users in proximity of one another and notifies them. In addition, the two users use the server to exchange messages.

The above approach has several disadvantages. First, anyone who can eavesdrop on the mobile communication and is able to identify the source and origin learns the affiliation of the user from the fact that it sends encrypted messages to the organizational server. This could be prevented by anonymity solutions such as ToR. However, unless there is a large number of users, a powerful adversary can mount timing attacks.

Second, interaction involving a central server assumes having internet connectivity

- an assumption that is often problematic for mobile devices. It also complicates the setup and requires securing the server against attackers. In addition, it requires entrusting location information and message content to a third party.

Finally, there is the power efficiency consideration. Using GPS for high-precision localization is energy consuming, as is using cellular communication for interacting with the server.

The proposals in this chapter aim to eliminate unnecessary interaction with a third party, leveraging physical proximity and minimizing energy consumption, while providing strong privacy and anonymity guarantees. To achieve this we use cryptographic secret handshakes over Bluetooth Low Energy (BLE) protocol.

Modern smartphone devices are equipped with BLE chips that enable transmission and scanning with very low power cost. These transceivers implement the Bluetooth Smart standard and enable a device to constantly transmit short messages, known as advertising, or scan for such advertisements from other devices. One notable example of using BLE advertisements is iBeacon, standardized and promoted by Apple [31].

The basic idea is that each mobile device, affiliated with the secret community in our example, constantly transmits and scans for advertisements. The advertisements seem completely random to any eavesdropper whereas, in fact, they are attempts to initiate secret handshakes with nearby devices. Nowadays, we are already surrounded by many devices that use BLE advertisements. It is therefore possible to go unnoticed as yet another device.

There are more interesting applications to secret handshakes by mobile devices over BLE. One of them is headcounting of attendants to a secret event. Some commercial products, like Doubledutch.me [7], support headcounting by putting a BLE beacon transmitter at a venue that asks nearby mobile users to confirm attendance. However, if we want the event to be private, we need to prevent users that are not affiliated with the event from identifying that beacon. We also want to protect the privacy of the attendants by making sure their reply to the broadcast cannot identify them as attending this event. Once again we can use a secret handshake between a device transmitting the event-advertising beacon and nearby mobile users to perform mutual authentication and enable headcounting, attendance confirmation or any

other private data exchange related to the event.

Another application is BLE-based car locking and unlocking using a mobile device. Using a BLE beacon, an owner can unlock the car when in proximity to it. In addition, the car can transmit a beacon when parked, to make it easier to find it. The car can also lock automatically once it stops receiving a beacon from the mobile device, making sure the car is locked once the driver walks away. *Complete Keyless* [4] and *Connect2Car* [5] are two examples of vendors addressing this need. However, if the beacons can be associated with the vehicle or the mobile device, they enable tracking. An unlinkable secret handshake¹ between the two eliminates this concern.

These examples make a strong case for enabling secret handshakes over BLE on modern smartphones and mobile platforms. BLE opens a new venue of possible private communication and IoT applications. However, it also comes with certain constraints, most importantly the limited amount of data that can be exchanged over BLE packets. We note that, for some applications of interest, exchanging small amounts of data after the handshake is useful, and for others, like headcounting or car unlocking, the success of a handshake protocol itself may be enough, without any further data transmission.

This work aims to demonstrate the practicality of frequently performing secret handshakes between mobile devices, using BLE technology. By implementing a library for secret handshakes we hope to open the door for building various interesting applications. We determined a suitable cryptographic scheme for implementing secret handshakes under the constraints imposed by BLE. We also address the issue of tracking mobile devices that transmit the same or linkable beacons over time by discussing ways to support *unlinkability*, based on existing cryptographic techniques and on features supported by BLE.

As a proof of concept we developed MASHaBLE, a mobile application for performing secret handshakes between Windows Phone devices. As part of its development we also implemented a .NET wrapper for pairing-based cryptography. This is a useful tool for any pairing-based crypto applications for the Windows Universal platform, including mobile and desktop.

¹We explain unlinkability in 5.1.2.

While we design a protocol suitable for BLE, and implement it, we claim no novelty in the underlying cryptography. We base our protocol on proven cryptographic constructions.

To summarize the main contributions of this chapter:

- We propose novel applications for mobile devices supporting BLE, based on a scheme for cryptographic secret handshakes that has previously been mostly theoretic.
- We provide technical details of and identify constraints imposed by the BLE stack, and pick a suitable SH scheme. We also identify computational optimizations applicable to our setting.
- We provide a prototype implementation of the secret handshake scheme for Windows Phone devices. We make our implementation available as an open-source library that can facilitate development of additional pairing-based cryptographic applications for Windows Universal framework, including desktop and Windows phone devices.
- Evaluate our implementation and show that it is practical for use on mobile devices in terms of battery consumption and performance.

In the following section we provide some necessary technical background. We give an introduction to BLE and explain cryptographic secret handshakes constructed from pairing-based key agreements. We then discuss the specifics of implementing the cryptographic scheme over BLE and our prototype implementation for Windows Phone. Finally, we discuss related work and possible future directions for research.

5.1 Background

5.1.1 Bluetooth Low Energy

Bluetooth Low Energy (BLE) was introduced as part of the Bluetooth 4.0 specification. This radio standard was designed to minimize power consumption [150] and be

suitable for low cost and low bandwidth transmitters. BLE devices can operate for extended periods of time, powered by a tiny energy source such as a coin cell. BLE chips are nowadays present in most modern smartphones, such as Windows phone, Google Nexus, and iPhone. It has been widely adopted by smartphone manufacturers.

Bluetooth 4.1 is an update to the specification and is the current reference for developing BLE applications. It is important to note that BLE is not directly compatible with classic Bluetooth and these are, despite the similarities, two different protocols.

Low power consumption by design comes with a cost. The modulation frequency of 1 Mbps [150] imposes an upper limit on throughput. The specification states minimal and maximal intervals of 7.5 ms and 4 s respectively, before entering an idle state after a connection (to save power). This sets up an upper limit for the number of connections per second. Some implementations, for instance nRF51822 by Nordic Semiconductor, impose additional restrictions: limited number of packets can be transmitted per each connection (6 in the case of nRF51822), and each packet contains only a small number of bytes (20 bytes for nRF51822). The latter is an important constraint that influenced our choice of secret-handshake scheme, among the possible variants.

BLE operates well for short ranges. While it's possible to configure devices to operate on a range of tens of meters (line-of-sight), the typical operation range would be several meters.

All of these are important considerations to take into account when designing a protocol to be executed over BLE.

Two modes of communication are available: *broadcast* and *connected* modes. Broadcast mode enables a device to send data to any other device listening for transmissions. As the name suggests, this is the way to transmit to multiple devices at once. Sending broadcasts is called *advertising* and the broadcast packets *advertisements*. A device that listens to advertisements is called a *scanner*. In our context, using this mode enables attempting a handshake with all scanners in proximity of the advertiser. Figure 5.1 depicts the structure of a BLE advertisement packet.

If two devices need to exchange data they can use the connected mode. The

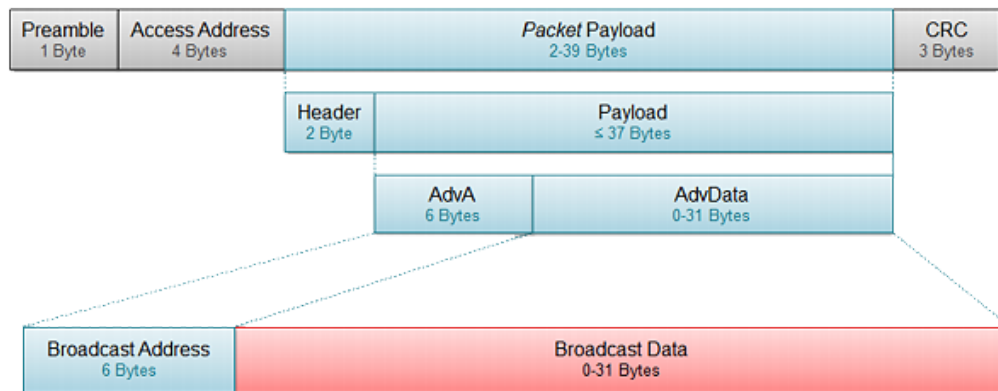


Figure 5.1: Structure of a BLE advertisement packet. Total size of an advertisement packet is 47 bytes.

broadcast indicates that the device can be connected to, and the scanner can initiate a connection following a received broadcast. Once a connection is established the devices can exchange data.

On Windows Phone, it is possible to do both advertising and scanning at the same time. We make use of this property to implement our message exchange.

Tracking prevention

Bluetooth packets include *Bluetooth device address* fields, similar to an Ethernet MAC address. It is a 6 byte number that uniquely identifies a device. Two types of addresses exist. The *public device address* is fixed for a device. A *random device address*, on the other hand, can be set dynamically and change across different connections.

If the public device address is used as the source Bluetooth device address, it would be possible to track the device around a certain area by scanning for BLE packets and examining the source address field.

In order to prevent tracking of a certain device by its MAC address, BLE supports randomized MAC addresses. The MAC address in the transmitted packets changes over time.

5.1.2 Cryptographic secret handshakes

Cryptographic secret handshakes, introduced by Balfanz et al. [45], are related to the broader area of Automatic Trust Negotiation (ATN). Secret handshakes enable two parties to establish that they are affiliated with a certain group, and to disclose their respective roles in the group. They guarantee that this information is disclosed only when the handshake succeeds, and no information is obtained by either party when the handshake fails. As such, it is useful when none of the parties are willing to be the first to reveal its affiliation. In this sense, the scheme offers properties similar to a physical secret handshake between two people. However, there is one caveat. One party learns about the success of the handshake before the other, and if it chooses to terminate the communication without responding to the other party's challenge, the other party is left uncertain as to the success of the secret handshake.

Security

The secret handshake has to be secure against the following attacks:

1. Group member impersonation - an adversary who is unaffiliated with the group performs a successful handshake with a group member.
2. Group member detection - an adversary who intercepts a broadcast from a group member identifies the group it is associated with.
3. Tracking - an adversary is able to tell that two different handshake attempts were made by the same party. Resiliency to this attack is called *unlinkability*.

Formal definitions of the security games corresponding to the above properties can be found in [45, 55, 40].

A concrete SH scheme

The scheme proposed in [45] takes a minimum of 3 communication rounds. Two messages are exchanged to establish a shared secret key, after which another is exchanged in order to enable both parties to confirm that they indeed obtained the same shared

key. The scheme is based on pairings over elliptic-curves (EC) and its security relies on the Bilinear Diffie-Hellman assumption (BDH). First, let us briefly explain the notion of pairings.

A pairing [75] e is a bilinear map $\mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_t$ such that

$$e(a \cdot u, b \cdot v) = e(u, v)^{ab}$$

where $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_t$ are cyclic groups, $u \in \mathbb{G}_1, b \in \mathbb{G}_2, a, b \in \mathbb{Z}_n$ and $(\cdot)$ is a group operation, e.g. multiplication over an elliptic-curve. In case $\mathbb{G}_1$ and $\mathbb{G}_2$ are the same group, the pairing is called symmetric. Symmetric pairings are, in fact, slower to compute and we therefore recommend using asymmetric pairings in the implementation. The most known examples are the Weil and Tate pairings, which can be efficiently computed using Miller's algorithm [124].

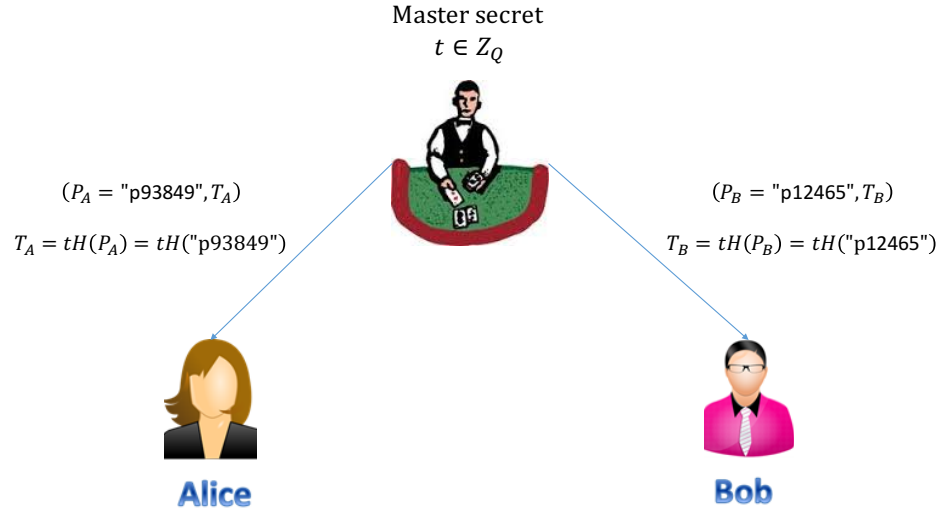
Using pairings for cryptography has resulted in applications such as identity-based and attribute-based encryption [50], 3-party Diffie-Hellman key exchange [98], BLS short signatures [51], secret handshakes and more.

While other options exist, we use a secret handshake constructed from pairing-based key agreements due to Balfanz et al. [45]. The scheme is simple and elegant, and the exchanged credentials are short, making it a suitable candidate for integration with BLE (on which we elaborate in section 5.2).

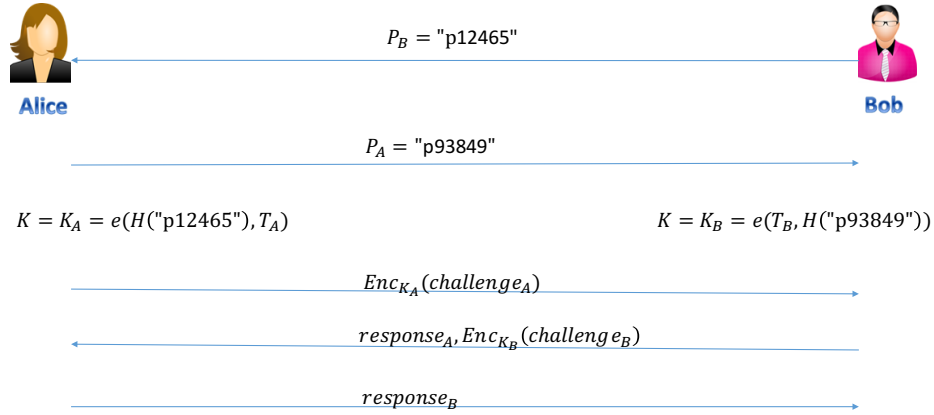
Let us sketch the protocol for performing a secret handshake between two parties. We use a hash-function H modeled as a random oracle that maps arbitrary strings to elliptic-curve points. $\mathbb{Z}_q$ denotes an integer group of large prime order q , where q is 512 bits long.

Two parties, *Alice* and *Bob*, want to perform a handshake. First, there is a setup stage involving interaction of both parties with a credential-authority (CA) service, as depicted in figure 5.2(a).

The CA possesses a master secret key $t \in \mathbb{Z}_q$, called “the group secret”. It issues credentials to *Alice* in the form of a pseudonym P_A and a secret elliptic-curve point T_A , i.e. (P_A, T_A) where $T_A = t \cdot H(P_A)$. Similarly, *Bob* obtains (P_B, T_B) where $T_B = t \cdot H(P_B)$. The two parties exchange their pseudonyms without disclosing their



(a) Setup: CA issues credentials to both parties. T_A and T_B are secret and each party keeps it to itself. Actual pseudonym values were added for illustration.



(b) Handshake. Note that we have redundant rounds here: the challenge message could be sent earlier together with the pseudonym. We present it as separate messages for clarity.

Figure 5.2: Secret handshake protocol.

secret points.

Alice generates a session key by computing

$$K_A = e(H(P_B), T_A) = e(H(P_B), t \cdot H(P_A)) = e(H(P_B), H(P_A))^t \quad (5.1)$$

where e is a bilinear map, i.e. a pairing operation, and *Bob* generates a session key by computing

$$K_B = e(T_B, H(P_A)) = e(t \cdot H(P_B), H(P_A)) = e(H(P_B), H(P_A))^t \quad (5.2)$$

Due to the properties of pairings the two keys are the same, i.e. $K_A = K_B$. By sending a challenge value and getting a response for it, the two can verify that the handshake succeeded. Alternatively, one party can send to the other some content, which will be successfully decrypted if and only if the handshake succeeded. These steps are illustrated in figure 5.2(b).

It is important to include a “direction” bit with the responses to prevent reflection attacks on the challenge–response protocol. A concrete example can be found in [45] (section 4.2).

Note that since this construction involves hashing an arbitrary string to an element of $\mathbb{G}_2$, it is important to choose a type of pairing that indeed supports this kind of hashing. Galbraith et al. [77] present a taxonomy of pairings and their respective properties. According to the properties of the different types of pairings, we need to use Type 1 or Type 3 pairings to enable hashing arbitrary strings onto elliptic-curve group elements.

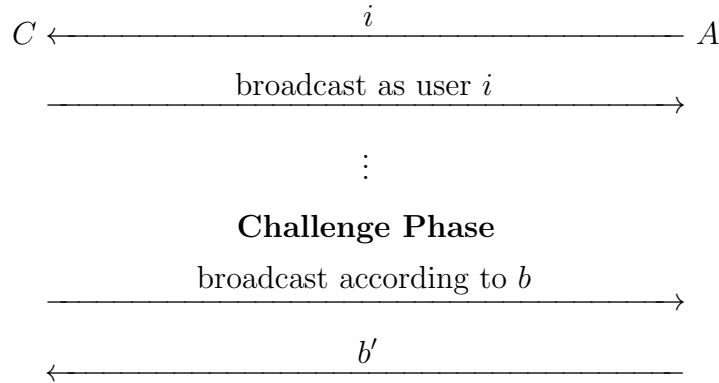
Security of this secret handshake scheme against group member detection and group member impersonation is based on the assumption that Bilinear Diffie-Hellman problem is hard for all probabilistic polynomial-time algorithms. Detailed theorems and proofs of security can be found in section 5.2 of [45]. Security of our protocol implementation against those attacks stems from the security of this underlying cryptographic scheme.

Unlinkable secret handshakes

One problem with the previous scheme is that a powerful eavesdropper who monitors all communication in various locations would be able to track the users. While the eavesdropper doesn't know whether the handshake succeeded or not, it sees the same pseudonym over and over again. We would therefore like to have an untraceable, or unlinkable secret handshake scheme, one that guarantees that an eavesdropper cannot draw a link between two broadcasts, correctly associating them with the same identity. Providing unlinkability addresses the tracking attack, mentioned in section 5.1.2, on the application level, whereas randomized MAC addresses (section 5.1.1) prevent tracking on the datalink level. A combination of both measures has to be used to prevent tracking.

Due to the importance of this property we include a detailed discussion of unlinkability for secret handshakes.

To provide a formal definition of unlinkability, we define the following security game between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$.



The adversary is allowed to perform a polynomial number of broadcast queries to the challenger. It specifies the identity of the user i for which it requests a broadcast. For each query, $\mathcal{C}$ obtains credentials as described in the secret handshake scheme and simulates a new user in the system by sending over a valid advertisement to $\mathcal{A}$. Finally, $\mathcal{A}$ performs another query. The challenger picks at random a bit $b \in \{0, 1\}$.

If $b = 0$, $\mathcal{C}$ responds with a valid broadcast for one of the previously used identities, and if $b = 1$, $\mathcal{C}$ responds with credentials corresponding to a new identity, previously unused in the game. Only $\mathcal{C}$ knows b . $\mathcal{A}$ outputs a guess b' for the value b picked by $\mathcal{C}$.

Definition 1. *A secret handshake scheme is unlinkable if an efficient adversary $\mathcal{A}$ has only negligible advantage in winning the game over randomly guessing b , i.e.*

$$\text{Adv}(\mathcal{A}) = \left| \Pr(b' = b) - \frac{1}{2} \right| < \varepsilon$$

where ε is negligible in the security parameter.

A straightforward approach is generating many pseudonyms and secret points for each client during the enrollment phase. When the client performs handshakes, it uses different credentials each time, which prevents an eavesdropper, or an active attacker, from linking those handshakes to each other, and therefore prevents tracking the client. This naive approach has obvious drawbacks. It increases the amount of storage since the user has to store credentials for all future executions of the protocol. It also limits the number of unlinkable handshakes to the number of pairs of credentials issued at the setup. It might be good enough when a user is unlikely to deplete its pool of credentials, or if it is occasionally possible to communicate with the group administrator to obtain fresh credentials. In addition, this method provides revocation and traitor tracing, which help protect against an adversary that is capable of corrupting users.

We can also achieve unlinkability by a simple modification to the previous scheme, based on an idea introduced by Huang and Cao [91]² and later used by Gu and Xue [83]. Instead of P_A and P_B being strings that are visible during the initial message exchange between the two parties, we denote by P_A and P_B random points on the elliptic curve. t is once again the master secret, and member secrets are computed as

$$T_A = t \cdot P_A \quad T_B = t \cdot P_B$$

²Their particular scheme, however, has a flaw, as pointed out in [149, 159], due to release of the group public-key. Nevertheless, it doesn't affect our use of the pseudonym randomization idea.

P_A and P_B are never sent as is in the clear. Instead, for each handshake, Alice picks a random $r \in \mathbb{Z}_q$ and sends rP_A to Bob. Bob picks a random $s \in \mathbb{Z}_q$ and sends sP_B to Alice. Alice computes

$$K_A = e(sP_B, T_A)^r = e(P_B, P_A)^{rst} \quad (5.3)$$

and Bob computes

$$K_B = e(T_B, rP_A)^s = e(P_B, P_A)^{rst} \quad (5.4)$$

so that $K_A = K_B$.

None of the two learns the permanent pseudonym of the other. An adversary that eavesdrops on the messages cannot link different handshake attempts by the same member. And yet, both parties obtain the same key that can now be used for encrypting communication between the two. This scheme provides information theoretic security against tracking. This has been previously argued in [91, 83], but due to the simplicity of this randomization technique, we include a formal claim, and its trivial proof below.

Claim 1. *The handshake scheme presented in 5.1.2 is unlinkable according to Definition 1.*

Proof. The unlinkability of this scheme is information theoretic with a trivial proof. Since each pseudonym picked by the challenger is multiplied by a uniformly drawn random element in $\mathbb{Z}_q$, each broadcast is indistinguishable from a random element to $\mathcal{A}$. $\square$

It is important to note, that this scheme becomes vulnerable to member impersonation in case an adversary corrupts a member of the group, as pointed out by Yoon [158]. Given a member's permanent credentials, the adversary can generate new ephemeral credentials that will pass a handshake. This, however, poses a problem even for newer SH schemes such as [40]. It is therefore important to avoid this method for achieving unlinkability when corruption is likely, and to examine new schemes that provide revocation and traitor tracing along with unlinkability. In the

meantime, we can stick to the former simple method of issuing multiple credentials to each member.

Security parameters

Security is quantified as the number of basic operations (e.g AES encryptions) needed for recovering the secret key and breaking the cryptographic scheme.

Breaking a symmetric-key scheme with key size k requires 2^k basic operations. Galbraith et al. [77] summarize the equivalence between given symmetric key sizes and the corresponding EC group element sizes, required to provide the same level of security. To enjoy 128-bit security³ we have to use EC subgroups having size of at least 256 bits, i.e. 32 bytes. It means that we need 32-byte long pseudonyms (for unlinkable handshakes). Compromising for 80-bit security, we can use 160-bit (20 byte) elements⁴. For string pseudonyms (first scheme), 20 bytes provide 128-bit security.

5.2 Secret handshakes over BLE

In this section we discuss ways to integrate secret handshakes into BLE. Our goal is to reuse, as much as possible, the existing BLE message exchange. This part examines the ideal vs. the currently available and practical. We look into ways to reconcile the requirements of a protocol for secret handshakes with BLE constraints and with the current state of supported BLE features on popular mobile devices.

5.2.1 Challenges

Implementing secret handshakes over BLE poses several challenges. Addressing those challenges at the design stage is one of the main contributions of this chapter. Some key issues are the limited amount of data that is possible to transfer between devices over BLE, the fact that we are operating within the restrictions of a given standard, and various constraints imposed by existing BLE implementations both in hardware

³That is the expected level of security nowadays.

⁴This compromise, if affordable, can simplify the proposed implementation, as we will see later.

and software. In addition, we had to pick an underlying cryptographic SH scheme that fits those constraints.

1. **Fitting into BLE packet size:** The restriction on the packet sizes influences our choice of the underlying cryptographic scheme. BLE advertisement packets can fit up to 39 bytes of payload, with further restriction on how many of them we can control (31 bytes on the platform we ended up using). The messages exchanged during a pairing process can fit 16 bytes only. This requires choosing a scheme in which little data is exchanged.
2. **Avoiding Bluetooth pairing:** We do not want the devices to “know” each other, i.e. be paired prior to the handshake attempt, as this contradicts our use-cases. Our proposals for integration avoid this. The first one is a new pairing mode that replaces the existing BLE pairing with a secret-handshake. The second uses BLE advertisements, which do not require prior pairing in order to be published or processed by a scanning party.
3. **Fitting within the standard:** BLE communication is standardized and we want to operate within the capabilities and restrictions of the standard. While the proposition we make in section 5.2.3 considers a modification of BLE, we also aim for a prototype that works given the currently available functionality, which we present in section 5.2.4.
4. **Low energy consumption:** Power consumption becomes a major issue since we aim for use-cases where a handshake attempt is frequently initiated. While BLE itself is designed to consume little energy, we also need to make sure that our protocol doesn’t add an energy overhead that is not acceptable for a mobile device.

5.2.2 Choice of the underlying SH scheme

The scheme presented in section 5.1.2 is a good fit for addressing the small packet size, since the parties only exchange a pseudonym in the form of a single string (or single

group element in the case of the modified unlinkable scheme). Some other schemes discussed in section 5.5, such as [40, 55], require sending more data to initiate a handshake. Moreover, this scheme is simple to understand, and given a library for computing pairings⁵, it also results in a relatively simple implementation. Finally, it requires a single pairing computation by each party, compared to [40] which requires three.

While some considerations were presented, surveying and comparing different SH schemes was not the central goal of this work, as much as demonstrating practicality using an existing method. A follow-up work on the subject could greatly benefit from a thorough comparison of the different schemes available, and in particular, comparing their performance and bandwidth requirements.

In the following, we present two methods for integrating the underlying SH scheme into BLE.

5.2.3 A new pairing mode

The first proposition for executing the secret handshake procedure over BLE is to reuse the Bluetooth *pairing* procedure⁶. Pairing, in both classic Bluetooth and BLE, establishes a short-term shared key that can be used for encrypted communication between two Bluetooth devices. It can be used for a short message exchange or to establish a long-term key to be used in subsequent sessions.

BLE currently offers several pairing methods:

- Just Works - requires no user interaction⁷. Does not protect from man-in-the-middle (MITM) attacks.
- Numeric comparison - a 6-digit number is displayed on both devices, which confirm the pairing if they both see the same number. Protects from MITM attacks.

⁵One of the contributions of this work is providing a library wrapper for the platform of choice.

⁶Not to be confused with the mathematical operation over elliptic curves, mentioned in the explanation of secret handshakes.

⁷Sometimes the user will be asked to confirm the pairing, but not to insert a PIN or to compare passkeys.

- Passkey entry - one device displays a 6 digit number, which has to be entered on the other device to confirm the pairing.
- Out-of-band (OOB) - An external method is used to provide information to both devices that is used to complete the pairing. For instance, NFC can be used to provide each device with an encryption key.

The numeric comparison and passkey entry methods are not very secure, as shown by Mike Ryan [140]. Rather than relying on existing pairing methods, we propose the secret handshake for establishing trust and obtaining a shared session key. Therefore we can simply use Just Works to establish the Short Term Key (STK), as we do not care about security at this step. The STK establishment process is followed by a pairing confirmation, in the form of *Pairing Confirm* messages sent by both devices.

We use the *Pairing Confirm* messages to exchange pseudonyms between *Alice* and *Bob* by setting the value of the *Mconfirm* field to be the pseudonym. This implies that we have to use 128-bit pseudonyms. At the next step we use the *Pairing Random* message to send an encrypted challenge from *Alice* to *Bob* by setting *Mrand* to be the challenge. *Bob* computes the response and sends it in the *Srand* field in its *Pairing Random* reply message to *A*, which can now verify the success of the handshake⁸. This process is depicted in figure 5.3.

However, despite using existing messages, this approach requires introducing an additional pairing method and extending the BLE stack with new functionality. And since the pseudonym sizes are limited by the length of the *Mconfirm* messages, it enables less than 80-bits security, which is not acceptable in modern cryptographic implementations. Achieving the desired security levels would require further modifications to the pairing protocol.

5.2.4 Advertiser-scanner interaction

As mentioned in section 5.1, BLE supports broadcasting advertisements. Clients can scan and filter advertisements of specific types. An advertisement packet allows only

⁸Non-resolvable random Bluetooth addresses are used for all communication mentioned above.

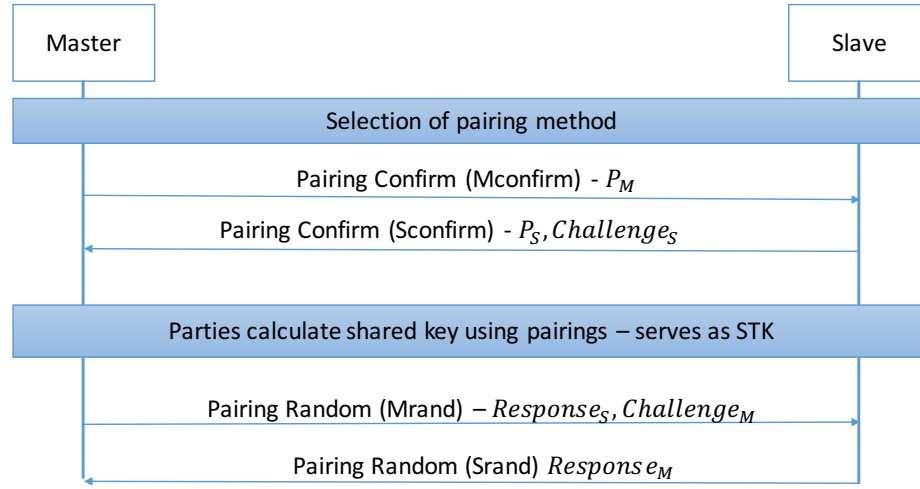


Figure 5.3: Secret handshake as a new pairing mode. Pseudonyms, challenges and responses are embedded in already existing fields of messages exchanged as part of the pairing procedure.

31 bytes of data to be set. With *passive scanning* the advertised data is limited to the contents of one advertisement packet. With *active scanning*, the scanner can request more data and receive another packet from the advertiser in response.

Windows .NET Bluetooth API currently enables controlling only the Manufacturer Specific Data (AD type 0xFF), which is 20 bytes.

This method of performing the handshake doesn't require extending the BLE specification. It relies on transmitting data between an *advertiser Alice* and a *scanner Bob*⁹ in a way that does not require a Bluetooth pairing.

The advertiser broadcasts its pseudonym and, if the client that receives the broadcast in a *scanning* state is interested in performing a handshake, it enters the *initiating* state. The client that enters the *connection* state from an initiating state acts as a Master, while the advertiser that enters it from the advertising state acts as a Slave. The connection state serves for verifying the success of the handshake by exchanging a challenge and a response over the encrypted link.

In section 5.1.2 we mentioned that 32-byte pseudonyms are required. Because an advertisement packet cannot contain more than 31 bytes of payload, we have to submit

⁹Here, advertiser and scanner are terms taken directly from the BLE specification, referring to a party in advertising state and a party in a scanning state respectively.

two packets. By using *Active Scanning*, the scanner can initiate a Scan Request once an advertisement packet is received. The advertiser sends another packet in response, which provides enough space to serve a 32-byte string.

After exchanging pseudonyms, the parties derive a 128-bit (16 bytes) long symmetric key. Any subsequent communication for “challenge-response” can therefore fit in one advertisement packet. As previously mentioned, 80-bit security requires only 20-byte long pseudonyms, which now can fit in a single advertisement packet. This size is also enough for 128-bit security with the first (linkable) scheme.

Note that while the scheme in [45] can ultimately require only 3 messages, it would require packing more data into each message. We avoid it due to the limited advertisement length, and instead exchange pseudonyms first, and then proceed with a challenge and response phase, resulting in 4 messages.

To support simultaneous interaction between many devices, we need to instantiate a separate state machine for each handshake attempt, and associate it with the Bluetooth device address of the other party. That way, advertisements received from different parties do not interfere with each other while executing the protocol. Of course, when using a randomized MAC address, we are required to maintain the same device address across all protocol rounds.

While the pairing mode option (5.2.3) is appealing, it would require adopting our proposal and have it integrated into the BLE standard. Currently, it also restricts us to smaller security parameters. We chose to implement our prototype using advertiser-scanner interaction, as suggested in the last subsection (5.2.4).

The scanner-advertiser interaction does not require a Bluetooth pairing between the devices, and yet enables exchanging small amounts of data by the same method.

5.3 Implementation

In this section we discuss various practical considerations that influenced our prototype implementation. We describe the chosen alternative and walk through the implementation.

5.3.1 Implementation Challenges

Two main factors affect the choice of platform:

- Custom control over BLE communication.
- Convenient implementation of pairings over elliptic curves: pairings are fairly complicated to implement and we would therefore like to begin with an existing, tested implementation.

The current state of BLE deployment on mobile platforms poses challenges to our goals. Only a few smartphone models can broadcast BLE advertisements. In addition, few fields within the advertisement packets can be controlled.

iOS is fairly restrictive in terms of an application's ability to control the data that is transmitted in advertisement packets. While iOS provides support for Apple's own iBeacon protocol, it does not allow customizing the advertisement data, which precludes using iOS as a prototyping platform for testing our protocol.

The Android framework provides the APIs necessary for doing BLE advertising and scanning through *BluetoothLeAdvertiser* and *BluetoothLeScanner* respectively. One particular appeal of Android is the fact that applications are programmed in Java, and there exists a Java library, named JPBC [63], that implements pairing operations over elliptic-curves. It provides the functionality we need for implementing pairing-based secret handshakes.

Unfortunately, most Android phones do not currently support Bluetooth advertisement broadcasting in practice, and we had to defer this option for now. It would be highly useful to enable this capability in future models.

5.3.2 Windows Phone

Windows phones, running Windows 10 OS, support publishing BLE advertisements and scanning. In addition, they enable advertising and scanning simultaneously. We chose to prototype the handshake scheme on Windows Phone platform since at the moment it was the only widespread mobile framework that enables advertising,

provides control of the advertisement packet data, and also enables developing rich user interface using a high-level language.

First, we needed an implementation of elliptic curve operations and pairings that would run on the device. Windows Phone applications are executed on a .NET Runtime and are usually written in C#. It is also possible to integrate native compiled code through *Windows Runtime Components*. One direction was attempting to port the JPBC library, written in Java, to C#. While the task can be partially automated using tools like Sharpen [24], it would still involve a significant effort. Most obstacles arise from the fact that not all Java constructs and syntax are directly translatable to C#.

We adapted the Stanford PBC library [115] to run on ARM and implemented a .NET wrapper to enable using it for Windows Universal applications written in C#. Specifically, we used PBC v0.5.14.

Lynn's PBC library [115] is a C implementation of pairings over elliptic curves. It is easily parameterizable in terms of choosing the algebraic fields to work with, the types of elliptic curves to use, and provides useful functions such as random element generation and hashing arbitrary strings onto elliptic curves.

Its fastest benchmark indicates that a pairing computation can be done in 11 ms [114]. This enables performing about 90 pairing computations per second. Since our handshake protocol requires a single pairing computation (for each party), this is unlikely to become a bottleneck.

Originally, PBC requires the GNU Multi-Precision library (GMP) for its big-integer operations. We used the Multiple-Precision Integers and Rationals library (MPIR) [19], which is compatible with GMP, since it was already adapted for building with Visual Studio. We did the necessary porting to compile it for ARM so that it can be used on a Windows Phone platform.

We replaced the random data generation function originally used in PBC with a modified version to support the new API on Windows 10.

We used *CryptographicBuffer::GenerateRandom* for generating random bytes.

The ported PBC library for ARM is provided in form of a static library that can be linked with a Windows Runtime Component.

PbcProxy - a .NET wrapper for PBC

In order to integrate the ported PBC library with a .NET implementation of the protocol logic, we have written **Pbc-Proxy** - a Windows Runtime Component that links with the library and exposes the interfaces needed for implementing the handshake scheme.

The wrapper is implemented as a Windows Runtime Component, a DLL that exposes some of the functionality of the PBC library by bridging between managed and unmanaged code. All classes reside in the *PbcProxy* namespace. The following interfaces are exposed by the wrapper:

- **Element** - provides multiplication and power operations over group elements, and serializes an element to its byte representation. The serialization is used to obtain an encryption key.
- **Pairing** - provides functions for hashing strings to group elements, obtaining a random element in a given group and applying pairings to pairs of elements, one in $\mathbb{G}_1$ and the other in $\mathbb{G}_2$.
- **Group** - an interface representing an abstract group, and instantiations for representing $\mathbb{G}_1$, $\mathbb{G}_2$ and $\mathbb{Z}_n$.

To use *PbcProxy* we have to first initialize it by calling one of the methods *init()* or *init(seed)*. The first method sets PBC to use a random number generation function that uses *CryptographicBuffer::GenerateRandom*, while the second sets it to use a deterministic generator passing it a seed. Initializing two devices with a common seed is useful for testing by simulating the common dealer using a local instance. In our demo application, we simulated two phones getting issued credentials using the same master secret by independently setting their random seed to the same value.

We also provide a test method *PBC.test()* that implements a simple pairing computation, to test that the PBC library works correctly on the Windows phone platform.

PbcProxy exports the classes *Pairing* and *Element* that expose group element operations and the pairing application. A common interface *GroupIface* requires

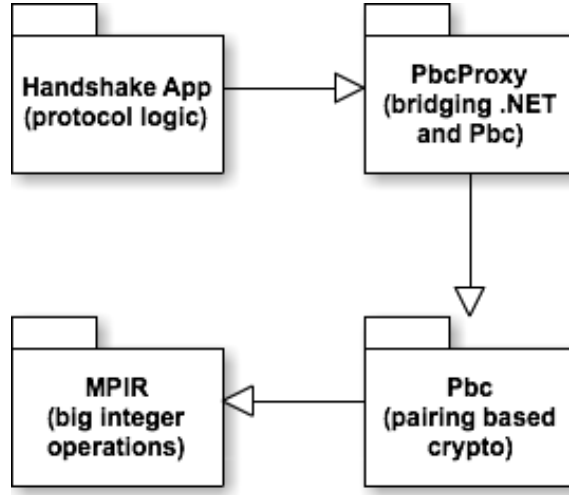


Figure 5.4: Component dependencies: PbcProxy bridges between the protocol logic implementation and the library providing pairing-based crypto, which uses big integer operations provided by MPIR.

implementing an *initElement* method for each group type we want to work with. This interface serves for decoupling the specific groups used from the implementation of the *Pairing* and *Element* classes. We have currently implemented subclasses for the groups $\mathbb{G}_1$, $\mathbb{G}_2$ and $\mathbb{Z}_n$ that allow using $\mathbb{Z}_n$ elements and referring to the respective groups used in a pairing. For example, in the secret handshake scheme, one credential is an element in $\mathbb{G}_1$ and the other is an element in $\mathbb{G}_2$.

The dependencies between the different software components are summarized in figure 5.4.

Handshake scheme prototype

We provide documentation of our prototype application. It is structured as follows. A party participating in the handshake is represented by one of the classes *Advertiser* or *Scanner*. The first, as the name suggests, publishes advertisements and the latter is scanning for advertisement packets. A mobile application, however, can perform both roles simultaneously. The two classes inherit common functionality from *HandshakeParty*. *Advertiser* and *Scanner* subclass *HandshakeParty*. *HandshakeParty* implements the communication protocol, and uses an instance of *HandshakeMember*

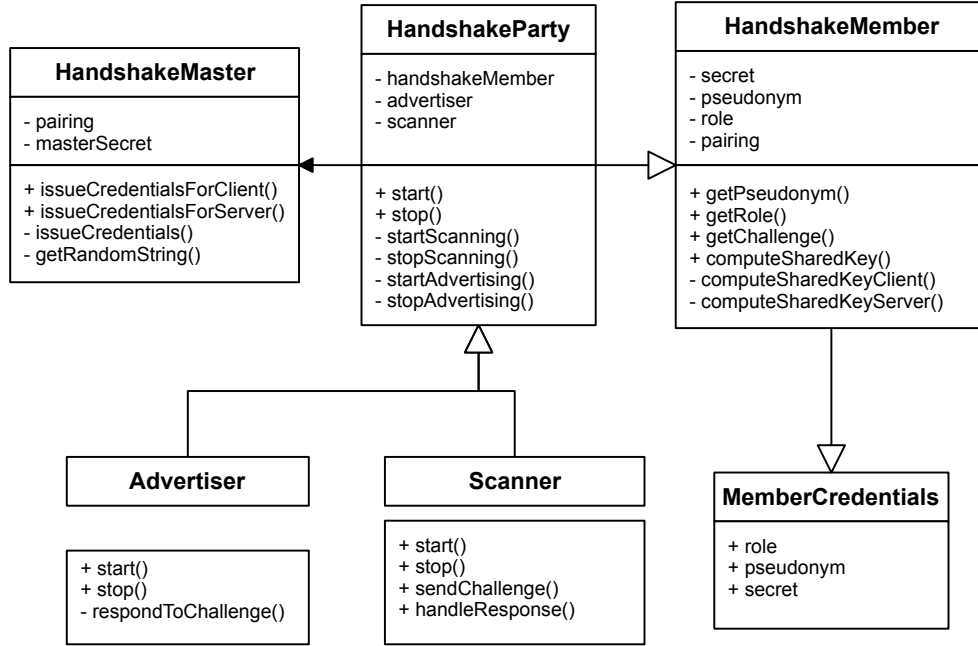


Figure 5.5: Class diagram of the handshake prototype implementation. Each party participating in a handshake instantiates both Advertiser and Scanner and alternates between them to communicate.

that provides the cryptographic functionality and interacts with *PbcProxy*. *HandshakeMaster* simulates the credentials dealer, providing an instance of *MemberCredentials* upon request. The relationships between the classes are summarized in the UML diagram in figure 5.5.

Figure 5.6 illustrates our realization of the secret handshake protocol using interaction between an advertiser and a scanner. It uses the fact that Windows Phone enables performing advertisement and scanning at the same time. Fig. 5.7 illustrates the state transition for the two parties based on the received broadcast.

The *Advertiser* starts publishing its pseudonym P_A , while at the same time scanning for a reply from the *Scanner*. The Scanner, upon receiving an advertisement packet, pauses the scanning, interprets the data as a pseudonym and computes a shared key according to one of Eq. 5.2 or 5.4, depending on whether we use linkable or unlinkable handshakes. It then starts advertising a response packet containing its own pseudonym P_S , and a challenge $Challenge_S$. The challenge is AES-encrypted using the

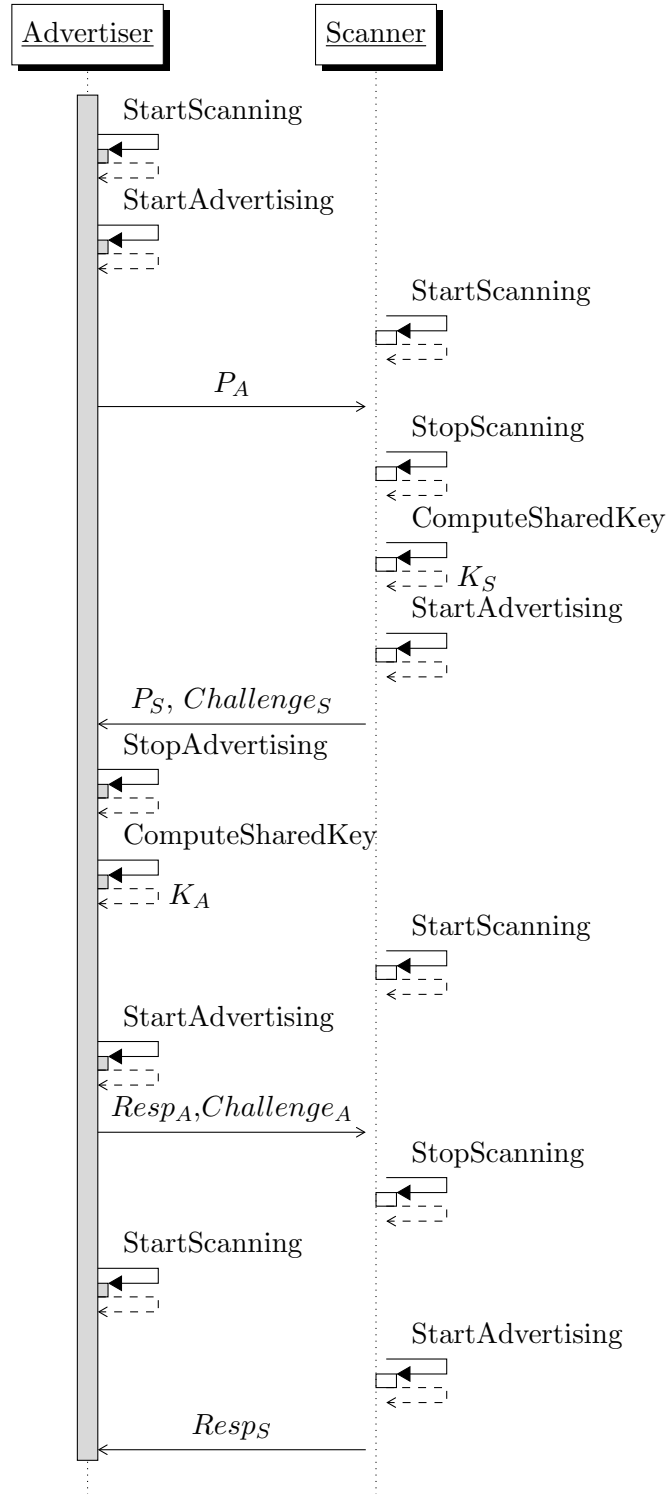
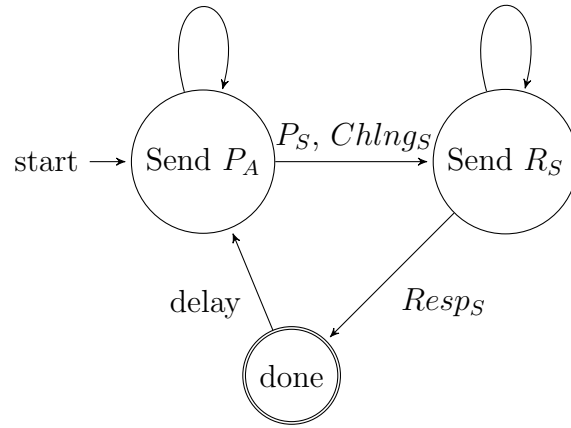
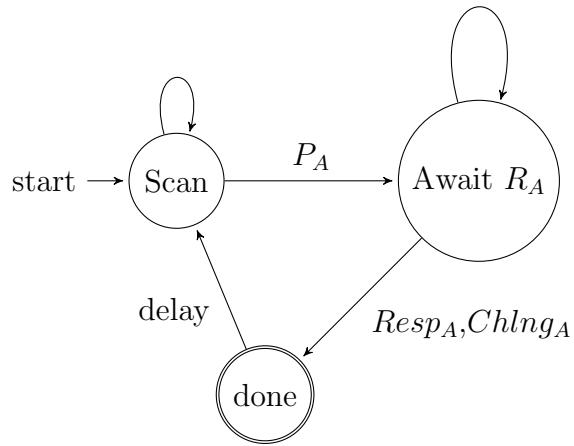


Figure 5.6: Secret handshake on top of Advertiser-Scanner interaction. The diagram illustrates simultaneous advertisement and scanning by the same party at certain stages, and switching between advertising and scanning to communicate.



(a) Advertiser



(b) Scanner

Figure 5.7: State machines for Advertiser and Scanner.

computed symmetric key. Shortly after, it reactivates scanning in order to receive a response from the advertiser. There is a small delay between starting to advertise and reactivating the scanning, in order to avoid receiving an advertisement with a pseudonym, before the Advertiser had a change to receive the Scanner's pseudonym and switch to a new state. Once the Advertiser receives an advertisement with the scanner's pseudonym, it pauses scanning and computes the shared key using one of Eq. 5.1 or 5.3. It then decrypts the challenge, using the computed symmetric key and calculates a response $Resp_A$. It also generates its own challenge $Challenge_A$ and encrypts it using the same key. It resumes advertising to send the response to the Scanner. The Scanner decrypts the advertisement packet using its key, and if it verifies correctly, assumes that the handshake succeeded. It computes and encrypts a response $Resp_S$ to $Challenge_A$ and starts advertising to send it to the Advertiser. Once the Advertiser receives the response $Resp_S$, it verifies it, and if it passes, assumes that the handshake succeeded.

The source code of our implementation is available on <https://github.com/ymcrcat/MASHaBLE>.

5.3.3 Evaluation

For evaluating our implementation we created a simple app that can broadcast, scan, and perform a secret handshake with another phone running the same app.

Functionality testing

Our devices perform simultaneously Bluetooth advertising and scanning. It is important to verify experimentally that this unusual practice results in successful execution of the handshake protocol with high probability. We ran our application on two Windows Phone devices, repeatedly performing handshakes between the two, for 8296 seconds (~ 2 hours 18 seconds). We performed 1 handshake every 8 seconds, resulting in a total number of 1068 handshake attempts. 1025 attempts resulted in a successful shared key establishment, and only 43 handshake attempts have failed.

The failures were due to occasional lack of synchronization between the states of the two devices. Overall, in 96% of the cases the handshake procedure succeeded. This result suggests that our protocol is fairly resistant to synchronization failures.

Energy overhead

We measured the energy overhead of our Windows Phone implementation on a Nokia Lumia 920 phone. In each of the following scenarios, we started with the phone with its battery 100% charged, WiFi and location services off, and screen dimmed to minimum brightness level:

1. Baseline: the phone is not running the app.
2. Advertising: the app is continuously publishing BLE advertisements of its pseudonym (no response is sent meaning no actual handshake process is initiated).
3. Scanning: the app is continuously scanning for advertisements (no advertisement is received meaning no actual handshake process is initiated).
4. Handshake Advertiser: the app, acting as an advertiser, is constantly performing handshakes with another phone acting as a scanner.
5. Handshake Scanner: the app, acting as a scanner, is constantly performing handshakes with another phone acting as an advertiser.

We ran the experiment for 3 hours for each one of the modes, and took note of the battery drain percentage. The results of the experiment are presented in Fig. 5.8. It is interesting to notice that scanning actually consumes more than advertising (despite the fact that advertising involves transmission). This result was previously confirmed in other experiments such as [100]. The reason is the design rational behind BLE modes of operation. Since it is likely that a beacon device has to be in advertising mode for a long period of time, it is designed for maximal energy saving. It transmits a short packet each time and returns to sleep. Scanning is intended to be initiated

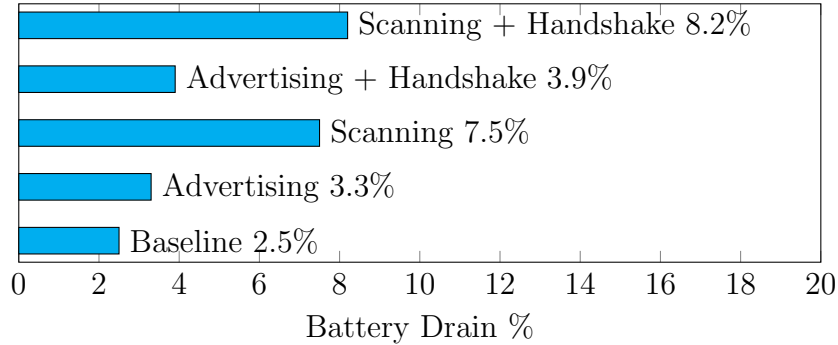


Figure 5.8: Battery drain in different modes after 3 hours. We can see that cryptographic operations add little overhead on top of BLE energy consumption.

less often. While scanning, the device constantly searches for beacons, performing intense processing.

The baseline we compare to is battery drain without any activity, which was measured to be 2.5% per hour. Advertising resulted in additional 0.8% per hour. Scanning consumes 5% per hour on top of the baseline, which is non-negligible, but nevertheless still enables almost 13 hours of operation. Advertising followed by handshake operations resulted in a battery drain of 3.9% per hour, which enables approximately 26 hours of operation. Finally, scanning followed by handshakes resulted in a drain of 8.2% per hour, which enables 12 hours of operation. Overall, these results are encouraging in the sense that they enable operation times that are on the order of what is required to be able to normally use a smartphone before recharging.

By subtracting the results when handshakes are not performed from the results when they are, we get the percentage that the handshake logic adds on top of the power consumption due to BLE scanning or advertising. In the case of advertising, handshakes added 0.6% per hour on top of advertising without proceeding further with the protocol. In the case of scanning, proceeding with a handshake added 0.7% on top of scanning only.

Communication overhead

Each advertisement packet consists of 47 bytes¹⁰. The handshake initiator (Alice) sends 1 packets to scanner (Bob) containing a pseudonym. Bob sends back 1 packet with his pseudonym and a challenge. Alice replies with 1 packet containing her response to Bob’s challenge and a challenge of her own, and finally, Bob responds to Alice’s challenge with 1 packet. Alice sends $47 \times 2 = 94$ bytes total, and Bob sends $47 \times 2 = 94$ bytes. Overall we have 4 communication rounds with 4 packets, which results in 188 bytes.

Depending on Alice and Bob’s decision whether to use the established encrypted channel for further communication, e.g. exchanging messages, more data can be transmitted. However, for all further communication, the overhead will stem solely from the Bluetooth packet structure, since they use the computed symmetric key for encryption, which results in the same length as the plaintext.

5.4 Further improvements

We propose possible enhancements to the current implementation. Some of them are immediately applicable, and some are subject to further research.

5.4.1 Speeding up computation

Chapter 6.11 of Lynn’s thesis [116] covers several possible opportunities for precomputation that can speed up subsequent computation of pairings. Those opportunities arise from the fact that some of the computation steps depend entirely on only one of the two inputs to the pairing. For each handshake initiated using the same credentials, the pairing $\mathbb{G}_1$ input is the same. Therefore, Lynn’s proposals can be applied. Moreover, preprocessing is supported by the PBC library and is part of its API. It would be beneficial to extend PbcProxy to support preprocessed pairings.

¹⁰In our case we use the maximum available size for advertisement packets.

5.4.2 Using BLE identifiers as pseudonyms

As already mentioned, BLE supports using a randomized device address instead of the permanent public address. If we could set the source device address we would be able to use it as a pseudonym, instead of setting it in the advertisement data. We saw that having 128-bit security for the unlinkable scheme requires broadcasting two subsequent advertisement packets in order to transmit the pseudonym. Having additional space in the form of the Bluetooth device address field enables to transmit 32-byte long credentials in a single packet.

5.4.3 Organizational role authentication

The secret handshake scheme described in 5.1.2 supports, with a slight modification, establishing the corresponding roles of the parties within the organization, in addition to proving affiliation. We do not provide the details here, but it is described in [45]. We note, however, that the method in [45] assumes using pseudonyms which are public strings, to which the other party can attach an additional string indicating the role. That doesn't settle with the unlinkable handshake scheme in 5.1.2, proposed as a countermeasure against tracking, because it involves transmitting encodings of elliptic curve elements that mask the pseudonym.

5.4.4 Additional applications of PbcProxy and ported PBC library

PbcProxy, the glue between the .NET framework and a C implementation of pairings compiled for the ARM processor, is an opening for many other Windows Phone applications that rely on pairing-based cryptography. One prominent application is Identity-based Encryption [50, 48, 49]. It enables encrypting sending encrypted messages to another party solely based on its publicly known identifier such as an email or a phone number. For instance, a company or organization's employees have each other's contacts. Using identity-based encryption they can securely communicate with each other without the need to exchange public keys directly. While the concept

itself is not new, and even commercialized by Voltage, the software libraries we provide enable easy development of this application for the Windows Phone platform.

5.4.5 Usability

While we discuss the technical aspects of secret handshakes between mobile devices, usability issues were left outside the scope of this work. For successful adoption of our, or any other library and protocol, there has to be a good understanding of how vendors and users would use this capability in practice. We suggested several use cases in the introduction. It would be beneficial to validate them by surveying vendors and users, as well as to find additional use-case scenarios.

5.5 Related work

We survey related work in the relevant domains, focusing on theoretic work applicable to our setting, and on systems projects with similar goals.

Hiding the Rumor Source by Fanti et al. [76] provides message source obfuscation by using *adaptive diffusion* spreading. They operate under a model that assumes an adversary that has access to metadata, and also to some corrupted nodes. Their method can be complementary to ours. If our application uses the established shared key to diffuse messages among society members, we can apply their scheme to dictate the message diffusion policies, so that an adversary that colludes with corrupted members of the secret group, would not be able to identify the message source.

RevCast by Schulman et al. [141] is related to our proposal in the sense that it disseminates messages over a radio channel. The authors use FM radio broadcasts to spread certificate revocation updates. Thus the message transfer is independent of a central server. It also aims to facilitate clients' privacy by obviating requests to a server, an act that discloses a client's interest in particular data.

Since the time our chosen scheme [45] was proposed, there have been more works

on the subject of secret handshakes. Some of them provide alternatives to pairing-based crypto [55, 152], while others address unlinkability and more flexible policies [40, 95, 105]. We discuss some of them below.

In [55] the authors propose a scheme for secret handshakes based on CA-Oblivious encryption. CA-Oblivious encryption relies on providing a certificate to the user from which one cannot learn about the signing authority. Its security proof relies on the Computational Diffie-Hellman assumption (CDH). This scheme avoids using pairing-based cryptography and can be built based on more standard PKI. This can be useful on platforms on which it is currently difficult to run one of the existing libraries for pairing-based cryptography, such as PBC or JPBC. While this scheme doesn't require pairings, more data has to be sent by the handshake initiating party than in the scheme based on pairings that only requires sending a short pseudonym. Considering the very constrained amount of data we are allowed to send in an advertisement packet, the pairing-based scheme seems more suitable for our task. Additionally, their scheme doesn't provide unlinkability that is not based on issuing multiple credentials, while the scheme of Balfanz et al. [45] is conveniently transformed into an unlinkable SH scheme with reusable credentials.

Secret handshakes with dynamic and fuzzy matching, by Ateniese et al. [40], adds the ability to specify a set of attributes, required from the other party, in order for the handshake to succeed. It is essentially an attribute-based secret handshake scheme which also provides unlinkability and supports roles. The authors integrated the scheme into the IPsec protocol, by extending the Internet Key Exchange protocol [86]. While the scheme we chose is not the state of the art, as of today, it is simpler to understand, and requires sending less data upon handshake initiation. The applications we proposed so far can do without the additional features provided by the dynamic or fuzzy handshake schemes. In addition, our protocol requires performing a single bilinear pairing computation, while that scheme requires performing three pairings. However, providing additional functionality by using more novel schemes would be a desirable follow-up work, as well as identifying how mobile applications can benefit from fuzzy attribute-based secret handshakes.

Hidden Credentials by Holt et al. [89] proposes a method for Bob to send a message

to Alice depending only on Alice’s credentials, without having any credentials of his own. It is based on Boneh-Franklin IBE [50] which is in turn based on the Weil pairing. The notion of hidden credentials could be useful in case there is a messenger, who is an outsider to the organization, but wants to send an encrypted message to any member that has a certain role or clearance in the organization.

[103] is an example of incorporating custom identifiers in the BLE MAC address, as we suggested in 5.4.2. While that work also pertains to protecting user’s privacy, the guarantees provided by their Incognito system are different and are useful under different assumptions.

Prior work also investigated techniques for preserving privacy of end-users by changes of IP and MAC addresses [135, 118], identifier-free link layer protocol [46, 82] and short-lived credentials or pseudonyms [96]. However, all these techniques aim to ensure unlinkability only.

[100] is a useful reference for understanding the intricacies of BLE scanning and advertisement. The power consumption model and design guidelines in that work can be useful for optimizing future versions of our protocol.

5.6 MASHaBLE Conclusion

The mobile application described in this chapter enables members of a secret community to discover other affiliates that are in proximity to their mobile device, and the creation of an authenticated and encrypted communication channel over which the two members can communicate. We covered the background needed to understand our scheme and studied of the technicalities related to implementing it on top of a Bluetooth LE stack on a mobile device. The current state of BLE support poses many challenges which we examine. We discussed the design considerations and alternatives and described our prototype implementation of the secret handshake scheme. The experimental evaluation of our implementation suggests that cryptographic secret handshakes for mobile devices are highly practical. In particular, we showed that our protocol plays well with Bluetooth LE, and its energy overhead is acceptable for mobile devices.

Chapter 6

Conclusions

Fast development of the technology related to smartphones has resulted in many valuable use-cases becoming realistic and, simultaneously, created gaps in the security and privacy of mobile users. Specifically, the approach often taken by the Android platform developers and device manufacturers is to unleash functionality, letting third-party software developers to take advantage of it and create new applications. However, it also results in occasional broadening of the attack surface.

The attacks presented in this thesis expose the danger of enabling direct access to mobile sensors hardware, and the potential threats of being able to access raw sensor measurements. It is difficult to address those threats using the permissions system prior to fully understanding the possible types of leakage. Surprisingly, sensors can reveal more sensitive information than suggested by their name or kind, due to correlation between their measurements and other conditions.

It is worth to point to several works that appeared following the publication of the Gyrophone attack, and cite this work. Some of them, specifically acknowledged Gyrophone for providing inspiration. It is interesting to learn about additional ways to “abuse” motion sensors.

AccelWord by Zhang et al. [161] studied energy efficient hotword recognition. Accelerometers are similarly sensitive to acoustic vibrations, and being highly energy-efficient, provide a way to identify short phrases without turning on the microphone

subsystem. Son et al. [147] have shown that it is possible to confuse a drone’s gyroscopes, causing it to lose stability, by pointing a noise-source (loudspeaker) at it. This allowed them to take down a drone from a distance of several tens of meters. Trippel et al. further explored acoustic injection attacks in WALNUT [151], showing that it is possible to affect the motion of a remotely controlled toy vehicle by influencing the MEMS gyroscopes on the controlling smartphone.

It is often easy to overlook all the potential threats stemming from different system components. One needs to take a step back and examine the whole system, and all possible interactions between its modules in order to assess its overall security. I hope that this work helps accentuate this principle.

On the applied cryptography side, MASHaBLE demonstrates how fairly new cryptographic tools can be harnessed to provide new privacy guarantees for practical applications. Cryptographic constructions are often proposed as is, without evaluation for concrete use-cases, and as such do not evolve into widely used schemes. As new use-case scenarios appear, and performance improves, we should revisit known cryptographic constructions and examine them for practicality.

Appendix A

Gyrophone supplementary materials

A.1 Signal reconstruction from Recurrent Non-Uniform Samples

Here we present the derivation of the discrete-time interpolation filters used in our implementation. The notation in the expressions corresponds to the notation in [74]. The continuous time expression for the interpolation filters according to Eq. 18 in [74] is given by

$$h_p(t) = a_p \text{sinc}\left(\frac{t}{T}\right) \prod_{q=0, q \neq p}^{N-1} \sin\left(\frac{\pi(t + t_p - t_q)}{T}\right)$$

We then sample this expression at times $t = nT_Q - t_p$ and calculate the filter coefficients for 48 taps. Given these filters, the reconstruction process consists of upsampling the input signals by factor N , where $N = T/T_Q$ is the number of ADCs, filtering and summation of the outputs of all filters (as shown in Figure A.1).

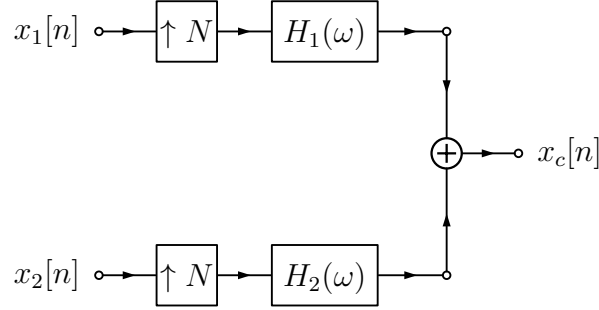


Figure A.1: Filterbank reconstruction scheme

A.2 Code for sampling a gyroscope via a HTML web-page

For a web page to sample a gyro the DeviceMotion class needs to be utilized. In the following we included a JavaScript snippet that illustrates this:

```
if(window.DeviceMotionEvent) {
  window.addEventListener('devicemotion', function(event) {
    var r = event.rotationRate;
    if ( r!=null ) {
      console.log('Rotation at [x,y,z] is: [' +
        r.alpha+', '+r.beta+', '+r.gamma+']\n');
    }
  }
}
```

Figure A.2 depicts measurements of the above code running on Firefox (Android) while sampling an audio chirp 50 – 100 Hz.

A.3 Gyroscope rate limitation on Android

Here we see a code snippet from the Invensense driver for Android, taken from *hardware/invensense/65xx/libensors_iio/MPLSensor.cpp*.

The OS is enforcing a sampling rate of 200 Hz.

```
static int hertz_request = 200;
#define DEFAULT_MPL_GYRO_RATE      (20000L)      //us
...
#define DEFAULT_HW_GYRO_RATE      (100)          //Hz
#define DEFAULT_HW_ACCEL_RATE     (20)           //ms
```

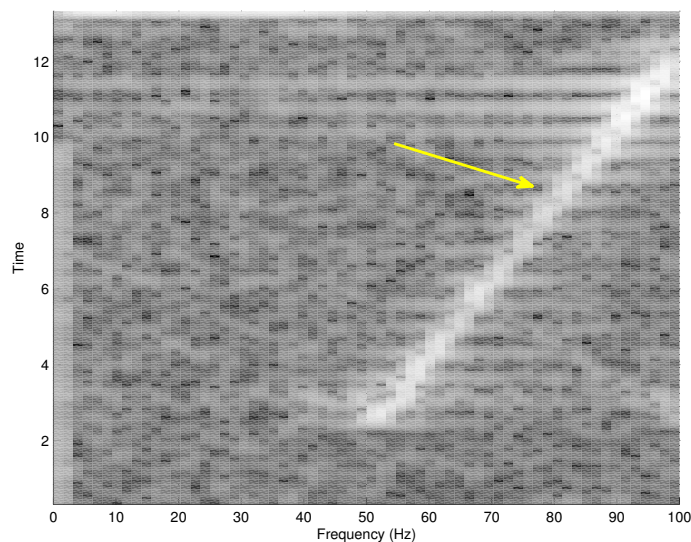


Figure A.2: Recording audio at 200 Hz using JavaScript code on a web-page accessed from the Firefox browser for Android.

```

...
/* convert ns to hardware units */
#define HW_GYRO_RATE_NS      (1000000000LL / rate_request) // to Hz
#define HW_ACCEL_RATE_NS    (rate_request / (1000000L))    // to ms
...
/* convert Hz to hardware units */
#define HW_GYRO_RATE_HZ      (hertz_request)
#define HW_ACCEL_RATE_HZ    (1000 / hertz_request)

```

A.4 Code Release

We provide the source code of the Android application we used for recording the sensor measurements, as well as the Matlab code we used for analyzing the data and training and testing of the speech recognition algorithms. We also provide the gyroscope recordings used for the evaluation of our method. The code and data can be downloaded from the project website at <http://crypto.stanford.edu/gyrophone>. In addition, we provide a web page that records gyroscope measurements if accessed from a device that supports it.

Appendix B

PowerSpy supplementary materials

B.1 Formal model of new route inference

In this section we formalize the problem of the new route inference (Section 4.5) as a hidden Markov model (HMM) [134]. Let I denote the set of intersections in an area in which we wish to track a mobile device. A road segment is given by an ordered pair of intersections (x, y) , defined to be a continuous road between intersection x and intersection y . We denote the set of road segments as R .

We assume that once a device starts to traverse a road segment it does not change the direction of its movement until it reaches the end of the segment. We define a state for each road segment. We say that the tracked device is in state s_{xy} if the device is currently traversing a road segment (x, y) , where $x, y \in I$. We denote the route of the tracked device as a (Q, T) , where

$$Q = \{q_1 = s_{x_1x_2}, q_2 = s_{x_2x_3}, \dots\} \quad T = \{t_1, t_2, \dots\}$$

For such a route the device has traversed from x_i to x_{i+1} during time interval $[t_{i-1}, t_i]$ ($t_0 = 0, t_{i-1} < t_i \forall i > 0$).

Let $A = \{a_{xyz} | \forall x, y, z \in I\}$ be the state transition probability distribution, where

$$a_{xyz} = p\{q_{i+1} = s_{yz} | q_i = s_{xy}\} \tag{B.1}$$

Note that $a_{xyz} = 0$ if there is no road between intersections x and y or no road between intersections y and z . A traversal of the device over a road segment yields a power consumption profile of length equal to the duration of that movement. We denote a power consumption profile as an observation o . Let B be the probability distribution of yielding a given power profile while the device traversed a given segment. Due to the hysteresis of hand-offs between cellular base stations, this probability depends on the previous segment the device traversed. Finally, let $\Pi = \{\pi_{xy}\}$ be the initial state distribution, where π_{xy} is the probability that the device initially traversed segment (x, y) . If there is no road segment between intersections x and y , then $\pi_{xy} = 0$. In our model we treat this initial state as the state of the device *before* the start of the observed power profile. We need to take this state into account due to the hysteresis effect. Note that an HMM is characterized by A , B , and Π .

The route inference problem is defined as follows. Given an observation of a power profile O over time interval $[0, t_{\max}]$, and given a model A , B and Π , we need to find a route (Q, T) such that $p\{(Q, T)|O\}$ is maximized. In the following we denote the part of O which begins at time t' and ends at time t'' by $O_{[t', t'']}$. Note that $O = O_{[0, t_{\max}]}$. We consider the time interval $[0, t_{\max}]$ as having a discrete resolution of τ .

B.2 Choosing the best inferred route

Upon its completion, the particle filter described in section 4.5.1 outputs a set of N routes of various lengths. We denote this set by P_{final} . This set exhibits an estimate of the distribution of routes given the power profile of the tracked device. The simple approach to select the best estimate is to choose the route that appears most frequently in P_{final} as it has the highest probability to occur. Nonetheless, since a route is composed of multiple segments chosen at separate steps, at each step the weight of a route is determined solely based on the last segment added to the route. Therefore, in P_{final} there is a bias in favor of routes ending with segments that were given higher weights, while the weights of the initial segments have a diminishing effect on the route distribution with every new iteration.

To counter this bias, we choose another estimate using a procedure we call *iterative*

majority vote. This procedure ranks the routes based on the prevalence of their prefixes. At each iteration i the procedure calculates – Prefix[i] – a list of prefixes of length i ranked by their prevalence out of all the routes that has a prefix in Prefix[$i-1$]. Prefix[i][n] denotes the prefix of rank n . The operation $p||j$ – where p is a route and j is an intersection – denotes the appending of j to p . At each iteration i algorithm 3 is executed. In the following we denote RoutePrefixed(R , p) to be the subset of routes out of the set R having p as their prefix.

Algorithm 3 Iterative majority vote

```

 $I' \leftarrow I$ 
while not all prefixes found do
  Prf  $\leftarrow$  next prefix from Prefix[ $i$ ].
  Find  $j \in I'$  that maximizes
    RoutePrefixed(RoutePrefixed( $P_{\text{final}}$ , Prf), Prf|| $j$ )
  if no such  $j$  is found then
     $I' = I$ 
    continue loop
  end if
  Prefix[ $i + 1$ ]  $\leftarrow$  Prefix[ $i + 1$ ]  $\cup$  {Prf|| $j$ }
   $I' = I' - \{j\}$ 
end while

```

At each iteration i we rank the prefixes based on the ranks of the previous iteration. Namely, prefixes which are extensions of a shorter prefix having a higher rank in a previous iteration will always get higher ranking over prefixes which are extensions of a lower rank prefix. At each iteration the we first find the most common prefixes of length $i + 1$, which start with the most common prefix of length i found in the previous iteration, and rank them according to their prevalence. Then we look for common prefixes of length $i + 1$, that start with the second most common prefix of length i found in the previous iteration, and so on until all prefixes of length $i + 1$ are found. The intuition is as follows. The procedure prefers routes traversing segments that are commonly traversed by other routes. Those received a high score when were chosen. Since we cannot pick the most common segments separately from each step (a continuous route is unlikely to emerge), we iteratively pick the most common segment out of the routes that are prefixed with the segments that were already chosen.

Bibliography

- [1] 3-axis digital gyroscopes. http://www.st.com/st-web-ui/static/active/en/resource/sales_and_marketing/promotional_material/flyer/fl3axdigitalgyro.pdf.
- [2] Accelerometer errors. <http://www.kionix.com/sites/default/files/AN012%20Accelerometer%20Errors.pdf>.
- [3] Afterschool, <http://afterschoolapp.com/app>.
- [4] Complete keyless, www.completekeyless.com.
- [5] Connect2car, www.connect2car.com.
- [6] Corona SDK API reference. <http://docs.coronalabs.com/api/library/system/setGyroscopeInterval.html>.
- [7] Dubbledutch.me, <http://doubledutch.me/attendance-tracking.html>.
- [8] Galaxy Tab 7.7. <http://www.techrepublic.com/blog/cracking-open/galaxy-tab-77-teardown-reveals-lots-of-samsungs-homegrown-hardware/588/>.
- [9] Inside the Latest Galaxy Note 3. <http://www.chipworks.com/en/technical-competitive-analysis/resources/blog/inside-the-galaxy-note-3/>.

- [10] Inside the Samsung Galaxy S4. <http://www.chipworks.com/en/technical-competitive-analysis/resources/blog/inside-the-samsung-galaxy-s4/>.
- [11] Inside the Samsung Galaxy SIII. <http://www.chipworks.com/en/technical-competitive-analysis/resources/blog/inside-the-samsung-galaxy-siii/>.
- [12] InvenSense Inc. <http://www.invensense.com/>.
- [13] iPad Mini Retina Display Teardown. <http://www.ifixit.com/Teardown/iPad+Mini+Retina+Display+Teardown/19374>.
- [14] L3G4200D data sheet. <http://www.st.com/st-web-ui/static/active/en/resource/technical/document/datasheet/CD00265057.pdf>.
- [15] Legatalk, <http://legatalk.com>.
- [16] LSM330DLC data sheet. <http://www.st.com/st-web-ui/static/active/en/resource/technical/document/datasheet/DM00037200.pdf>.
- [17] MPU-6000 and MPU-6050 Register Map and Descriptions. <http://invensense.com/mems/gyro/documents/RM-MPU-6000A.pdf>.
- [18] MPU-6050 product specification. <http://www.invensense.com/mems/gyro/documents/PS-MPU-6000A-00v3.4.pdf>.
- [19] Multiple precision integers and rationals library, mpir.org.
- [20] Nexus 4 Teardown. <http://www.ifixit.com/Teardown/Nexus+4+Teardown/11781>.
- [21] Nexus 7 Teardown. <http://www.ifixit.com/Teardown/Nexus+7+Teardown/9623>.
- [22] Open whisper systems, <https://whispersystems.org>.
- [23] Sensor id experiment website. <http://sensor-id.com>.

- [24] Sharpen - Automated Java to C# coversion.
- [25] STMicroelectronics Inc. <http://www.st.com/>.
- [26] Telegram messenger, <https://telegram.org>.
- [27] Yik yak, <https://www.yikyak.com/home>.
- [28] Everything about STMicroelectronics 3-axis digital MEMS gyroscopes. http://www.st.com/web/en/resource/technical/document/technical_article/DM00034730.pdf, July 2011.
- [29] iPhone 5S MEMS Gyroscope STMicroelectronics 3x3mm - Reverse Costing Analysis. http://www.researchandmarkets.com/research/lxrnrn/iphone_5s_mems, October 2013.
- [30] MEMS for Cell Phones and Tablets. http://www.i-micronews.com/upload/Rapports/Yole_MEMS_for_Mobile_June_2013_Report_Sample.pdf, July 2013.
- [31] "iBeacon" technology that will make possible Internet of Things. pages 159–165. Institution of Engineering and Technology, 2014.
- [32] Ahmed Al-Haiqi, Mahamod Ismail, and Rosdiadee Nordin. On the best sensor for keystrokes inference attack on android. *Procedia Technology*, 11:989–995, 2013.
- [33] Android Developers. Build Class. <http://developer.android.com/reference/android/os/Build.html>, January 2013.
- [34] Android Developers. Manifest.permission. <http://developer.android.com/reference/android/Manifest.permission.html>, January 2013.
- [35] Android Developers. Settings.Secure Class. <http://developer.android.com/reference/android/provider/Settings.Secure.html>, January 2013.

- [36] Android Developers. TelephonyManager Class. <http://developer.android.com/reference/android/telephony/TelephonyManager.html>, January 2013.
- [37] Android Developers Blog. Identifying App Installations. <http://android-developers.blogspot.se/2011/03/identifying-app-installations.html>, March 2011.
- [38] D.R. Appelman. *The Science of Vocal Pedagogy: Theory and Application*. Midland book. Indiana University Press, 1967.
- [39] M Sanjeev Arulampalam, Simon Maskell, Neil Gordon, and Tim Clapp. A tutorial on particle filters for online nonlinear/non-gaussian bayesian tracking. *Signal Processing, IEEE Transactions on*, 50(2):174–188, 2002.
- [40] Giuseppe Ateniese, Marina Blanton, and Jonathan Kirsch. Secret handshakes with dynamic and fuzzy matching. In *Network and Distributed System Security Symposium*, pages 159–177, 2007.
- [41] Adam J Aviv, Benjamin Sapp, Matt Blaze, and Jonathan M Smith. Practicality of accelerometer side channels on smartphones. In *Proceedings of the 28th Annual Computer Security Applications Conference*, pages 41–50. ACM, 2012.
- [42] M. Ayenson, D. Wambach, A. Soltani, N. Good, and C. Hoofnagle. Flash cookies and privacy ii: Now with html5 and etag respawning. *Available at SSRN 1898390*, 2011.
- [43] Martin Azizyan, Ionut Constandache, and Romit Roy Choudhury. Surround-sense: mobile phone localization via ambience fingerprinting. In *Proceedings of the 15th annual international conference on Mobile computing and networking*, pages 261–272. ACM, 2009.
- [44] A. Baharev. Gyroscope calibration. In *Proceedings of TMOP 4.2.2 Workshop*, 9 2010.

- [45] D. Balfanz, G. Durfee, N. Shankar, D. Smetters, J. Staddon, and Hao-Chi Wong. Secret handshakes from pairing-based key agreements. *2003 Symposium on Security and Privacy, 2003.*, 2003.
- [46] Kevin Bauer, Damon McCoy, Ben Greenstein, Dirk Grunwald, and Douglas Sicker. Physical layer attacks on unlinkability in wireless lans. In *International Symposium on Privacy Enhancing Technologies Symposium*, pages 108–127. Springer, 2009.
- [47] Hristo Bojinov, Yan Michalevsky, Gabi Nakibly, and Dan Boneh. Mobile device identification via sensor fingerprinting. *arXiv preprint arXiv:1408.1416*, 2014.
- [48] Dan Boneh and Xavier Boyen. Secure identity based encryption without random oracles. In *Advances in Cryptology–Crypto 2004*, pages 443–459. Springer, 2004.
- [49] Dan Boneh, Xavier Boyen, and Eu-Jin Goh. Hierarchical identity based encryption with constant size ciphertext. In *Advances in Cryptology–EUROCRYPT 2005*, pages 440–456. Springer, 2005.
- [50] Dan Boneh and Matt Franklin. Identity-based encryption from the weil pairing. *Advances in Cryptology - CRYPTO 2001*, 2001.
- [51] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. *Advances in Cryptology - ASIACRYPT 2001*, 2001.
- [52] V. Brik, S. Banerjee, M. Gruteser, and S. Oh. Wireless device identification with radiometric signatures. In *Proceedings of the 14th ACM international conference on Mobile computing and networking*, pages 116–127, 2008.
- [53] Liang Cai and Hao Chen. Touchlogger: inferring keystrokes on touch screen from smartphone motion. In *Proceedings of the 6th USENIX conference on Hot topics in security*, pages 9–9. USENIX Association, 2011.
- [54] Aaron Carroll and Gernot Heiser. An analysis of power consumption in a smartphone. In *USENIX Annual Technical Conference*, 2010.

- [55] Claude Castelluccia, Stanislaw Jarecki, and Gene Tsudik. Secret Handshakes from CA-Oblivious Encryption. In *Advances in Cryptology - ASIACRYPT 2004, 10th International Conference on the Theory and Application of Cryptology and Information Security, Jeju Island, Korea, December 5-9, 2004, Proceedings*, volume 3329, pages 293–307, 2004.
- [56] Simon Castro, Robert Dean, Grant Roth, George T Flowers, and Brian Grantham. Influence of acoustic noise on the dynamic performance of mems gyroscopes. In *ASME 2007 International Mechanical Engineering Congress and Exposition*, pages 1825–1831, 2007.
- [57] O. Celiktutan, B. Sankur, and I. Avcibas. Blind identification of source cell-phone model. *Trans. Info. For. Sec.*, 3(3):553–566, September 2008.
- [58] William Banks Clarkson and Edward W Felten. Breaking assumptions: distinguishing between seemingly identical items using cheap sensors. Technical report, 2012.
- [59] J. Cooke. Flash memory technology direction. *Micron Applications Engineering Document*, 2007.
- [60] Koby Crammer, Michael Kearns, and Jennifer Wortman. Learning from multiple sources. *The Journal of Machine Learning Research*, 9:1757–1774, 2008.
- [61] Anupam Das and Nikita Borisov. Poster: Fingerprinting smartphones through speaker. In *Poster at the IEEE Security and Privacy Symposium*, 2014.
- [62] Anupam Das, Nikita Borisov, Edward Chou, and Muhammad Haris Mughees. Smartphone fingerprinting via motion sensors: Analyzing feasibility at large-scale and studying real usage patterns. *arXiv preprint arXiv:1605.08763*, 2016.
- [63] Angelo De Caro and Vincenzo Iovino. jpbcc: Java pairing based cryptography. In *Proceedings of the 16th IEEE Symposium on Computers and Communications, ISCC 2011*, pages 850–855. IEEE, 2011.

- [64] Robert N Dean, George T Flowers, A Scotte Hodel, Grant Roth, Simon Castro, Ran Zhou, Alfonso Moreira, Anwar Ahmed, Rifki Rifki, Brian E Grantham, et al. On the degradation of mems gyroscope performance in the presence of high power acoustic noise. In *Industrial Electronics, 2007. ISIE 2007. IEEE International Symposium on*, pages 1435–1440. IEEE, 2007.
- [65] Robert Neal Dean, Simon Thomas Castro, George T Flowers, Grant Roth, Anwar Ahmed, Alan Scottedward Hodel, Brian Eugene Grantham, David Allen Bittle, and James P Brunsch. A characterization of the performance of a mems gyroscope in acoustically harsh environments. *Industrial Electronics, IEEE Transactions on*, 58(7):2591–2596, 2011.
- [66] L.C.C. Desmond, C.C. Yuan, T.C. Pheng, and R.S. Lee. Identifying unique devices through wireless fingerprinting. In *Proceedings of the first ACM conference on Wireless network security*, pages 46–55, 2008.
- [67] Sanorita Dey, Nirupam Roy, Wenyuan Xu, Romit Roy Choudhury, and Srihari Nelakuditi. Accelprint: Imperfections of accelerometers make smartphones trackable. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2014.
- [68] Sanorita Dey, Nirupam Roy, Wenyuan Xu, Romit Roy Choudhury, and Srihari Nelakuditi. Accelprint: Imperfections of accelerometers make smartphones trackable. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2014.
- [69] J. Doscher. Accelerometer design and applications. *Analog Devices*, 3, 1998.
- [70] P. Eckersley. How unique is your web browser? In *Privacy Enhancing Technologies*, pages 1–18, 2010.
- [71] Gerry Eisenhaur, Michael Gagnon, Tufan Demir, and Neil Daswani. Mobile malware madness and how to cap the mad hatters, 2011.

- [72] Manar El-Chammas and Boris Murmann. *Background calibration of time-interleaved data converters*. Springer, 2012.
- [73] Manar Ibrahim El-Chammas. *Background Calibration of Timing Skew in Time-Interleaved A/D Converters*. Stanford University, 2010.
- [74] Yonina C Eldar and Alan V Oppenheim. Filterbank reconstruction of bandlimited signals from nonuniform and generalized samples. *Signal Processing, IEEE Transactions on*, 48(10):2864–2875, 2000.
- [75] Andreas Enge. Bilinear pairings on elliptic curves, 2013.
- [76] Giulia Fanti, Peter Kairouz, Sewoong Oh, Kannan Ramchandran, and Pramod Viswanath. Hiding the rumor source. *arXiv preprint arXiv:1509.02849*, 2015.
- [77] Steven D. Galbraith, Kenneth G. Paterson, and Nigel P. Smart. Pairings for cryptographers, 2008.
- [78] Camillo Gentile, Nayef Alsindi, Ronald Raulefs, and Carole Teolis. *Geolocation Techniques*. Springer New York, New York, NY, 2013.
- [79] Theodoros Giannakopoulos. A method for silence removal and segmentation of speech signals, implemented in matlab.
- [80] Andrea Goldsmith. *Wireless communications*. Cambridge university press, 2005.
- [81] Google Play. Mac Address Ghost. <https://play.google.com/store/apps/details?id=diewland.changemac>.
- [82] Ben Greenstein, Damon McCoy, Jeffrey Pang, Tadayoshi Kohno, Srinivasan Seshan, and David Wetherall. Improving wireless privacy with an identifier-free link layer protocol. In *Proceedings of the 6th international conference on Mobile systems, applications, and services*, pages 40–53. ACM, 2008.
- [83] Jie Gu and Zhi Xue. An Improved Efficient Secret Handshakes Scheme with Unlinkability. *IEEE Communications Letters*, 15(2):259–261, feb 2011.

- [84] J. Hall, M. Barbeau, and E. Kranakis. Detection of transient in radio frequency fingerprinting using signal phase. *Wireless and Optical Communications*, pages 13–18, 2003.
- [85] Jun Han, Emmanuel Owusu, Le T. Nguyen, Adrian Perrig, and Joy Zhang. ACComplice: Location inference using accelerometers on smartphones. In *Proceedings of the 2012 International Conference on COMmunication Systems & NETworkS*, 2012.
- [86] Dan Harkins and Dave Carrel. Rfc 2409: The internet key exchange (ike), november 1998. *Status: Proposed Standard*.
- [87] Md Rashidul Hasan, Mustafa Jamil, and Md Golam Rabbani Md Saifur Rahman. Speaker identification using mel frequency cepstral coefficients. *variations*, 1:4, 2004.
- [88] G.C. Holst. *CCD arrays, cameras, and displays*. SPIE Optical Engineering Press, 1998.
- [89] Jason E. Holt, Robert W. Bradshaw, Kent E. Seamons, and Hilarie Orman. Hidden Credentials. *Proceeding of the ACM workshop on Privacy in the electronic society - WPES '03*, page 1, 2003.
- [90] Jingyu Hua, Zhenyu Shen, and Sheng Zhong. We can track you if you take the metro: Tracking metro riders using accelerometers on smartphones. *arXiv:1505.05958*, 2015.
- [91] Hai Huang and Zhenfu Cao. A Novel and Efficient Unlinkable Secret Handshakes Scheme. *Ieee Communications Letters*, 13(5):363–365 ST – A Novel and Efficient Unlinkable Sec, 2009.
- [92] Junxian Huang, Feng Qian, Alexandre Gerber, Z. Morley Mao, Subhabrata Sen, and Oliver Spatscheck. A close examination of performance and power characteristics of 4G LTE networks. In *MobiSys*, 2012.

- [93] iOS Developer Library. ASIdentifierManager Class. http://developer.apple.com/library/ios/#documentation/AdSupport/Reference/ASIdentifierManager_Ref/ASIdentifierManager.html, September 2012.
- [94] iOS Developer Library. UIDevice Class. http://developer.apple.com/library/ios/#documentation/uikit/reference/UIDevice_Class/Reference/UIDevice.html, September 2012.
- [95] Stanislaw Jarecki and Xiaomin Liu. Unlinkable Secret Handshakes and Key-Private Group Key Management Schemes. In *Applied Cryptography and Network Security*, volume 4521, pages 270–287. 2007.
- [96] Qi Jiang, Jianfeng Ma, Guangsong Li, and Li Yang. An efficient ticket based authentication protocol with unlinkability for wireless access networks. *Wireless personal communications*, 77(2):1489–1506, 2014.
- [97] Håkan Johansson and Per Löwenborg. *Reconstruction of periodically nonuniformly sampled bandlimited signals using time-varying FIR filters*. 2005.
- [98] Antoine Joux. A one round protocol for tripartite diffie–hellman. In *Algorithmic number theory*, pages 385–393. Springer, 2000.
- [99] Arnold Kim. Apple begins rejecting apps for using the unique device identifier (UDID), 2012.
- [100] Philipp Kindt, Daniel Yunge, Robert Diemer, and Samarjit Chakraborty. Precise Energy Modeling for the Bluetooth Low Energy Protocol. mar 2014.
- [101] Paul Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In *Advances in Cryptology – CRYPTO’99*, pages 388–397. Springer, 1999.
- [102] T. Kohno, A. Broido, and K.C. Claffy. Remote physical device fingerprinting. *Dependable and Secure Computing, IEEE Transactions on*, 2(2):93–108, 2005.
- [103] Robin Kravets, Güliz Seray Tuncay, and Hari Sundaram. For your eyes only. In *Proceedings of the 6th International Workshop on Mobile Cloud Computing and Services*, pages 28–35. ACM, 2015.

- [104] John Krumm and Eric Horvitz. Locadio: Inferring motion and location from wi-fi signal strengths. In *MobiQuitous*, pages 4–13, 2004.
- [105] Preeti Kulshrestha and Arun Kumar. A New Unlinkable Secret Handshakes Scheme based on ZSS.
- [106] Olivier Lartillot, Petri Toivainen, and Tuomas Eerola. A matlab toolbox for music information retrieval. In *Data analysis, machine learning and applications*, pages 261–268. Springer, 2008.
- [107] L.J. Latecki, Qiang Wang, S. Koknar-Tezel, and V. Megalooikonomou. Optimal subsequence bijection. In *Data Mining, 2007. ICDM 2007. Seventh IEEE International Conference on*, pages 565–570, Oct 2007.
- [108] R. Gary Leonard and George Doddington. TIDIGITS. <http://catalog.ldc.upenn.edu/LDC93S10>, 1993.
- [109] Vladimir I Levenshtein. Binary codes capable of correcting deletions, insertions and reversals. In *Soviet physics doklady*, volume 10, page 707, 1966.
- [110] Chang-Tsun Li. Source camera identification using enhanced sensor pattern noise. *IEEE Transactions on Information Forensics and Security*, 5(2):280–287, 2010.
- [111] Chang-Tsun Li. Source camera identification using enhanced sensor pattern noise. *Information Forensics and Security, IEEE Transactions on*, 5(2):280–287, 2010.
- [112] Jan Lukas, Jessica Fridrich, and Miroslav Goljan. Digital camera identification from sensor pattern noise. *Information Forensics and Security, IEEE Transactions on*, 1(2):205–214, 2006.
- [113] Jan Lukás, Jessica J. Fridrich, and Miroslav Goljan. Digital camera identification from sensor pattern noise. *IEEE Transactions on Information Forensics and Security*, 1(2):205–214, 2006.

- [114] Ben Lynn. Pbc benchmarks, <https://crypto.stanford.edu/pbc/times.html>.
- [115] Ben Lynn. Pbc library, <https://crypto.stanford.edu/pbc>.
- [116] Ben Lynn. *On the Implementation of Pairing-Based Cryptosystems*. PhD thesis, 2007.
- [117] Jani Mantyjarvi, Mikko Lindholm, Elena Vildjiounaite, S-M Makela, and HA Ailisto. Identifying users of portable devices from gait pattern with accelerometers. In *Acoustics, Speech, and Signal Processing, 2005. Proceedings (ICASSP'05). IEEE International Conference on*, volume 2, pages ii–973. IEEE, 2005.
- [118] Shrirang Mare, Jacob Sorber, Minho Shin, Cory Cornelius, and David Kotz. Hide-n-sense: preserving privacy efficiently in wireless mhealth. *Mobile Networks and Applications*, 19(3):331–344, 2014.
- [119] Philip Marquardt, Arunabh Verma, Henry Carter, and Patrick Traynor. (sp) iphone: decoding vibrations from nearby keyboards using mobile phone accelerometers. In *Proceedings of the 18th ACM conference on Computer and communications security*, pages 551–562. ACM, 2011.
- [120] Yan Michalevsky, Dan Boneh, and Gabi Nakibly. Gyrophone: Recognizing speech from gyroscope signals. In *Proc. 23rd USENIX Security Symposium (SEC14)*, USENIX Association, 2014.
- [121] Yan Michalevsky, Suman Nath, and Jie Liu. Mashable: Mobile applications of secret handshakes over bluetooth le. In *Proceedings of the 22nd Annual International Conference on Mobile Computing and Networking*, pages 387–400. ACM, 2016.
- [122] Yan Michalevsky, Aaron Schulman, Gunaa Arumugam Veerapandian, Dan Boneh, and Gabi Nakibly. Powerspy: Location tracking using mobile device power analysis. In *USENIX Security Symposium*, pages 785–800, 2015.

- [123] Gilles Miet. Towards wideband speech by narrowband speech bandwidth extension: magic effect or wideband recovery? *These de doctorat, University of Maine*, 2001.
- [124] Victor S Miller. The weil pairing, and its efficient calculation. *Journal of Cryptology*, 17(4):235–261, 2004.
- [125] Prashanth Mohan, Venkata N. VN Padmanabhan, and Ramachandran Ramjee. Nericell: rich monitoring of road and traffic conditions using mobile smart-phones. In *... of the 6th ACM conference on ...*, page 323, New York, New York, USA, November 2008. ACM Press.
- [126] Meinard Müller. Dynamic time warping. *Information retrieval for music and motion*, pages 69–84, 2007.
- [127] Meinard Müller. *Information Retrieval for Music and Motion*. Springer Berlin Heidelberg, Berlin, Heidelberg, 2007.
- [128] Kavitha Muthukrishnan, Berend Jan van der Zwaag, and Paul Havinga. Inferring motion and location using WLAN RSSI. In *Mobile Entity Localization and Tracking in GPS-less Environments*, pages 163–182. Springer, 2009.
- [129] Robin Wentao Ouyang, AK-S Wong, Chin-Tau Lea, and Victoria Ying Zhang. Received signal strength-based wireless localization via semidefinite programming. In *Global Telecommunications Conference, 2009. GLOBECOM 2009. IEEE*, pages 1–6. IEEE, 2009.
- [130] Athanasios Papoulis. Generalized sampling expansion. *Circuits and Systems, IEEE Transactions on*, 24(11):652–654, 1977.
- [131] Gregory P Pollini. Trends in handover design. *Communications Magazine, IEEE*, 34(3):82–90, 1996.
- [132] Pravin Prabhu, Ameen Akel, Laura M. Grupp, Wing-Kei S. Yu, G. Edward Suh, Edwin Kan, and Steven Swanson. Extracting device fingerprints from flash

- memory by exploiting physical variations. In *Proceedings of the 4th international conference on Trust and trustworthy computing*, TRUST'11, pages 188–201, Berlin, Heidelberg, 2011. Springer-Verlag.
- [133] Ryan S Prendergast, Bernard C Levy, and Paul J Hurst. Reconstruction of band-limited periodic nonuniformly sampled signals through multirate filter banks. *Circuits and Systems I: Regular Papers, IEEE Transactions on*, 51(8):1612–1622, 2004.
- [134] L Rabiner. A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 1989.
- [135] Barath Raghavan, Tadayoshi Kohno, Alex C Snoeren, and David Wetherall. Enlisting isps to improve online privacy: Ip address mixing by default. In *International Symposium on Privacy Enhancing Technologies Symposium*, pages 143–163. Springer, 2009.
- [136] V. Renaudin, M.H. Afzal, and G. Lachapelle. Complete triaxis magnetometer calibration in the magnetic domain. *Journal of Sensors*, 2010.
- [137] Douglas A Reynolds and Richard C Rose. Robust text-independent speaker identification using gaussian mixture speaker models. *Speech and Audio Processing, IEEE Transactions on*, 3(1):72–83, 1995.
- [138] Branko Ristic, Sanjeev Arulampalam, and Neil Gordon. Beyond the kalman filter. *IEEE AEROSPACE AND ELECTRONIC SYSTEMS MAGAZINE*, 19(7):37–38, 2004.
- [139] Mirco Rossi, Sebastian Feese, Oliver Amft, Nils Braune, Sandro Martis, and G Troster. Ambientsense: A real-time ambient sound recognition system for smartphones. In *Pervasive Computing and Communications Workshops (PERCOM Workshops), 2013 IEEE International Conference on*, pages 230–235. IEEE, 2013.

- [140] Mike Ryan. Bluetooth: With Low Energy Comes Low Security. In *Proceedings of the 7th USENIX Conference on Offensive Technologies*, page 4, 2013.
- [141] Aaron Schulman, Dave Levin, and Neil Spring. Revcast: Fast, private certificate revocation over fm radio. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, pages 799–810. ACM, 2014.
- [142] Aaron Schulman, Neil Spring, Vishnu Navda, Ramachandran Ramjee, Pralhad Deshpande, Calvin Grunewald, Venkata N. Padmanabhan, and Kamal Jain. Bartendr: a practical approach to energy-aware cellular data scheduling. *MOBICOM*, 2010.
- [143] Joe Seeger, Martin Lim, and Steve Nasiri. Development of high-performance high-volume consumer MEMS gyroscopes. <http://www.invensense.com/mems/gyro/documents/whitepapers/Development-of-High-Performance-High-Volume-Consumer-MEMS-Gyroscopes.pdf>.
- [144] Claude Elwood Shannon. Communication in the presence of noise. *Proceedings of the IRE*, 37(1):10–21, 1949.
- [145] SK Sindhi and KMM Prabhu. Reconstruction of N-th Order Nonuniformly Sampled Signals Using Digital Filter Banks. *commsp.ee.ic.ac.uk*, 2012.
- [146] Timothy Sohn, Alex Varshavsky, Anthony LaMarca, Mike Y Chen, Tanzeem Choudhury, Ian Smith, Sunny Consolvo, Jeffrey Hightower, William G Griswold, and Eyal De Lara. Mobility detection using everyday gsm traces. In *UbiComp 2006: Ubiquitous Computing*, pages 212–224. Springer, 2006.
- [147] Yunmok Son, Hocheol Shin, Dongkwan Kim, Young-Seok Park, Juhwan Noh, Kibum Choi, Jungwoo Choi, Yongdae Kim, et al. Rocking drones with intentional sound noise on gyroscopic sensors. In *USENIX Security Symposium*, pages 881–896, 2015.

- [148] T Strohmer and J Tanner. Fast reconstruction algorithms for periodic nonuniform sampling with applications to time-interleaved adcs. In *Acoustics, Speech and Signal Processing, 2007. ICASSP 2007. IEEE International Conference on*, volume 3, pages III–881. IEEE, 2007.
- [149] Renwang Su. On the security of a novel and efficient unlinkable secret handshakes scheme. *IEEE Communications Letters*, 13(9):712–713, sep 2009.
- [150] Kevin Townsend, Carles Cufi, and Robert Davison. Getting Started with Bluetooth Low Energy, 2014.
- [151] Timothy Trippel, Ofir Weisse, Wenyuan Xu, Peter Honeyman, and Kevin Fu. Walnut: Waging doubt on the integrity of mems accelerometers with acoustic injection attacks. In *Security and Privacy (EuroS&P), 2017 IEEE European Symposium on*, pages 3–18. IEEE, 2017.
- [152] Damien Vergnaud. RSA-Based Secret Handshakes. pages 252–274. 2006.
- [153] Willie Walker, Paul Lamere, Philip Kwok, Bhiksha Raj, Rita Singh, Evandro Gouvea, Peter Wolf, and Joe Woelfel. Sphinx-4: A flexible open source framework for speech recognition. Technical report, 2004.
- [154] F Xu, Y Liu, Q Li, and Y Zhang. V-edge: fast self-constructive power modeling of smartphones based on battery voltage dynamics. *Presented as part of the 10th USENIX . . .*, 2013.
- [155] Zhi Xu, Kun Bai, and Sencun Zhu. Taplogger: Inferring user inputs on smartphone touchscreens using on-board motion sensors. In *Proceedings of the fifth ACM conference on Security and Privacy in Wireless and Mobile Networks*, pages 113–124. ACM, 2012.
- [156] LM Yee and AM Ahmad. Comparative Study of Speaker Recognition Methods: DTW, GMM and SVM. *comp.utm.my*.

- [157] T.F. Yen, Y. Xie, F. Yu, R.P. Yu, and M. Abadi. Host fingerprinting and tracking on the web: Privacy and security implications. In *Proceedings of NDSS*, 2012.
- [158] Eun-Jun Yoon. Cryptanalysis of an Efficient Secret Handshakes Scheme with Unlinkability. *Procedia Engineering*, 24:128–132, 2011.
- [159] Taek-Young Youn and Young-Ho Park. Security analysis of an unlinkable secret handshakes scheme. *IEEE Communications Letters*, 14(1):4–5, jan 2010.
- [160] L Zhang, B Tiwana, Z Qian, and Z Wang. Accurate online power estimation and automatic battery behavior based power model generation for smartphones. *Proceedings of the ...*, 2010.
- [161] Li Zhang, Parth H Pathak, Muchen Wu, Yixin Zhao, and Prasant Mohapatra. Accelword: Energy efficient hotword detection through accelerometer. In *Proceedings of the 13th Annual International Conference on Mobile Systems, Applications, and Services*, pages 301–315. ACM, 2015.
- [162] Xiaoyong Zhou, Soteris Demetriou, Dongjing He, Muhammad Naveed, Xiaorui Pan, XiaoFeng Wang, Carl A. Gunter, and Klara Nahrstedt. Identity, location, disease and more: inferring your secrets from android public resources. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security - CCS '13*, pages 1017–1028, 2013.